

Documentación Técnica y Diseño de Pruebas del Sistema SmartComedor

Sandro Jaramillo y Anuar Sierra y Gabriel Macias

Universidad Popular del Cesar

Facultad de Ingenierías y Tecnologías

Ingeniería de Software II

Docente: Maribel Romero

Valledupar, 2026

Documentación Técnica y Diseño de Pruebas del Sistema SmartComedor

Este documento presenta la documentación técnica integral del sistema **SmartComedor**, una aplicación web de tres capas para la gestión del comedor universitario (registro y validación de elegibilidad de estudiantes, gestión de pagos, control de asistencia, retroalimentación del servicio y auditoría). La primera parte describe el problema, el sistema, el modelo de requerimientos (funcionales y no funcionales), el modelo de casos de uso y el modelo de diseño (clases, secuencias, entidad-relación y componentes). La segunda parte documenta el proceso de verificación y validación: planificación, pruebas unitarias (caja negra y caja blanca), pruebas de integración (incremental y basada en hilos), pruebas de sistema (seguridad, rendimiento, usabilidad y portabilidad) y pruebas de aceptación. La tercera parte mide el software (atributos internos de tamaño y atributos externos de calidad según ISO/IEC 25010) y la cuarta lo estima con cinco técnicas (puntos de función, puntos de caso de uso, puntos objeto, puntos de historia y COCOMO). La suite automatizada ejecuta **196 pruebas con 100 % de éxito** y una cobertura global aproximada del **80,4 %** de sentencias. El trabajo se apoya en estándares y técnicas reconocidas de la ingeniería de software (IEEE, 2008; International Organization for Standardization, 2011; Myers y col., 2011; Pressman & Maxim, 2014).

Palabras clave: pruebas de software, verificación y validación, casos de uso, requerimientos, calidad de software, JWT, SISBÉN.

Índice

PRIMERA PARTE. Descripción del Sistema	8
Descripción del sistema	8
Identificación del problema	8
Contexto	8
Problemas concretos detectados	8
Planteamiento del problema	9
Justificación	9
Descripción detallada del sistema o aplicación	10

Propósito y alcance	10
Actores y roles	10
Arquitectura general	10
Stack tecnológico	11
Reglas de negocio principales	12
Modelo de requerimientos	12
Descripción	12
Requisitos funcionales	12
Requisitos no funcionales	16
Modelo de casos de uso	18
Diagramas de caso de uso	18
Descripción de casos de uso	20
CU-01 — Registrar estudiante	21
CU-02 — Iniciar sesion	24
CU-03 — Recuperar y restablecer contrasena	27
CU-04 — Configurar y verificar 2FA	30
CU-05 — Gestionar estudiantes (CRUD)	33
CU-06 — Importar estudiantes desde Excel	36
CU-07 — Validar SISBEN	39
CU-08 — Gestionar supervisores	42
CU-09 — Invitar supervisor / unirse por invitacion	45
CU-10 — Asignar estudiantes a supervisor	48
CU-11 — Subir pago y comprobantes	50
CU-12 — Verificar pago	53
CU-13 — Calcular almuerzos por monto	56
CU-14 — Registrar almuerzo	59
CU-15 — Consultar historial y asistencia del dia	63
CU-16 — Consultar analitica del tablero	66

CU-17 — Gestionar ciclos y cierre automatico	69
CU-18 — Calificar servicio	72
CU-19 — Enviar y responder quejas	75
CU-20 — Gestionar noticias	78
CU-21 — Consultar auditoria	81
Modelo de diseño del sistema	84
Diagrama de clases detallado	84
Diagramas de secuencias	87
Diagrama entidad–relación	92
Diagrama de componentes	94
Producto del software	95
Despliegue	95
Estructura del producto	96
Características destacadas	96
SEGUNDA PARTE. Pruebas del Software	96
Introducción	96
Planificación de las pruebas	97
Objetivos de las pruebas	97
Alcance de las pruebas	97
Estrategias de pruebas	98
Ambiente de pruebas	99
Pruebas unitarias	99
Análisis de las pruebas	99
Catálogo detallado de casos de prueba	100
AuthController (<code>unit/authController.test.ts</code>)	100

StudentsController (unit/studentsController.test.ts)	106
PaymentsController (unit/paymentsController.test.ts)	109
SupervisorsController (unit/supervisorsController.test.ts)	112
Ratings/Complaints/News (unit/ratingsComplaintsNews.test.ts)	113
Middleware de seguridad (unit/middleware.test.ts)	115
MealsController (unit/mealsController.test.ts)	116
SisbenValidationService (unit/sisbenValidationService.test.ts)	118
Integración incremental (bottom-up) (integration/incremental.test.ts)	118
Integración basada en hilos (integration/threads.test.ts)	120
Diseño de casos de prueba	121
Caja negra — Clases de equivalencia (Login)	121
Caja blanca — Cobertura de caminos (login)	122
Caja negra — Valores límite (registro y montos)	123
Registro de almuerzo y middleware	124
Ejecución y evaluación de las pruebas	125
Resultado de ejecución	125
Cobertura de código	126
Evaluación	127
Pruebas de integración	128
Estrategia de pruebas incrementales	128
Descripción	128
Diseño y ejecución de casos	128
Estrategia de pruebas basadas en hilos	129
Descripción	129
Diseño y ejecución de casos	129
Pruebas de sistema	130
Pruebas de seguridad	130
Diseño	130

Ejecución y evaluación	131
Pruebas de rendimiento	132
Diseño	132
Ejecución y evaluación	132
Pruebas de usabilidad	132
Diseño	132
Ejecución y evaluación	133
Pruebas de portabilidad	133
Diseño	133
Ejecución y evaluación	134
Pruebas de aceptación	134
Diseño de casos de prueba	134
Ejecución y evaluación de la prueba	135
Conclusiones de las pruebas	136
 TERCERA PARTE. Medición del Software	 137
Introducción a la medición	137
Objetivos	137
Alcance	137
Tipos y herramientas de métricas	137
Medición del software — atributos internos	138
Métricas de tamaño del código fuente	138
Métricas de tamaño orientadas a clases	139
Medición de la calidad — atributos externos	140
Métricas de usabilidad	140
Métricas de seguridad	141

Métricas de portabilidad	142
Métricas de mantenibilidad	143
Métricas de rendimiento	143
Métricas de fiabilidad	144
Métricas de compatibilidad	144
Métricas de adecuación funcional	145
Conclusiones de la medición	145
 CUARTA PARTE. Estimación del Software	 146
Introducción a la estimación	146
Estimación por puntos de función (PF)	146
Estimación por puntos de caso de uso (PCU)	147
Estimación por puntos objeto	148
Estimación por puntos de historia	148
Estimación con una herramienta (COCOMO básico)	149
Análisis de los resultados	149
Conclusiones de la estimación	150
 QUINTA PARTE. Conclusiones Generales	 150
Conclusiones generales	150

PRIMERA PARTE. Descripción del Sistema

Descripción del sistema

Identificación del problema

Contexto

El programa de bienestar universitario ofrece un servicio de comedor (almuerzos subsidiados) dirigido a estudiantes en condición de vulnerabilidad socioeconómica. La elegibilidad para el subsidio depende de la clasificación del estudiante en el SISBÉN (Sistema de Identificación de Potenciales Beneficiarios de Programas Sociales del Estado colombiano), de su situación académica (carrera, semestre) y de su situación personal. Históricamente la operación se gestionaba mediante procesos manuales y en papel: el estudiante diligenciaba formularios físicos y entregaba fotocopias; un funcionario validaba manualmente que la cédula y los nombres del certificado SISBÉN correspondieran al documento de identidad; el control de asistencia se llevaba en planillas impresas, y el pago/recarga de almuerzos se soportaba con comprobantes físicos. No existía trazabilidad ni un canal estructurado de retroalimentación.

Problemas concretos detectados

La Tabla 1 resume los problemas identificados y sus consecuencias.

Tabla 1*Problemas identificados en la operación manual del comedor*

#	Problema	Consecuencia
P1	Validación manual de elegibilidad (SISBÉN vs. cédula)	Lentitud, error humano y riesgo de fraude documental
P2	Control de asistencia en papel	Doble cobro, suplantación y pérdida de información
P3	Conciliación manual de pagos/recargas	Descuadres contables y comprobantes extra-viados
P4	Ausencia de trazabilidad/auditoría	Imposibilidad de investigar incidentes
P5	Falta de canal de retroalimentación	Desconocimiento de la percepción del servicio
P6	Gestión de cupos y ciclos sin automatizar	Beneficios vencidos que siguen activos
P7	Información dispersa entre actores	Ausencia de una fuente única de verdad

Planteamiento del problema

¿Cómo digitalizar y automatizar la gestión integral del comedor universitario —registro y validación de elegibilidad, control de asistencia, gestión de pagos, retroalimentación del servicio y auditoría— de manera segura, trazable y verificable, reduciendo el trabajo manual y el riesgo de fraude?

Justificación

Un sistema de información web que centralice estos procesos reduce el fraude (validación automática del SISBÉN por OCR/parseo de PDF e identificación con código QR único por estudiante), garantiza trazabilidad (bitácoras `audit_logs` y `supervisor_logs`), acelera la operación (importación masiva desde Excel, cálculo automático de almuerzos y cierre automático de ciclos) y mejora la calidad del servicio (calificaciones y quejas/sugerencias/comentarios).

Descripción detallada del sistema o aplicación

Propósito y alcance

SmartComedor permite registrar estudiantes, gestionar pagos, supervisar almuerzos y administrar el servicio de comedor universitario, con control de acceso por roles, validación automática de elegibilidad y auditoría completa. El alcance funcional abarca nueve módulos: autenticación y cuentas; gestión de estudiantes; gestión de supervisores; pagos/recargas; asistencia (almuerzos); ciclos; retroalimentación; noticias; y auditoría.

Actores y roles

El sistema define cuatro roles (enum `UserRole`), descritos en la Tabla 2.

Tabla 2

Actores y roles del sistema

Rol	Valor	Descripción
Administrador	<code>admin</code>	Configura el sistema; gestiona estudiantes, supervisores, pagos, ciclos y noticias; responde quejas; valida SISBÉN.
Supervisor	<code>supervisor</code>	Registra la asistencia/almuerzo de los estudiantes; busca estudiantes y consulta su historial.
Estudiante	<code>student</code>	Se registra; consulta perfil y almuerzos disponibles; sube pagos; califica el servicio; envía quejas; lee noticias.
Auditor externo	<code>external_auditor</code>	Acceso de solo lectura a auditoría, supervisores, pagos, asistencia y analítica.

Arquitectura general

El sistema sigue una arquitectura cliente-servidor de tres capas desplegada con Docker Compose (Figura 1). El frontend (React + Vite + Material UI) consume una API REST (Express + TypeScript) que persiste en PostgreSQL mediante el ORM Sequelize. El backend integra servicios de correo (Nodemailer), validación SISBÉN (pdf-parse y OCR con tesseract.js),

auditoría, backups (`pg_dump`) y automatización de ciclos (`node-cron`).

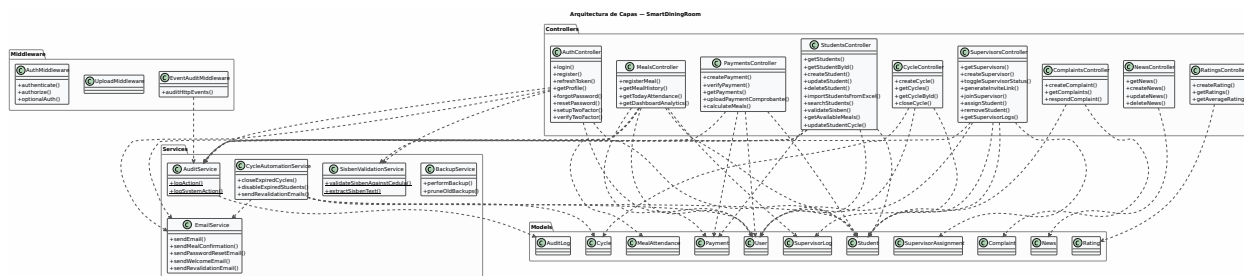


Figura 1

Arquitectura general del sistema SmartComedor.

Stack tecnológico

La Tabla 3 resume las tecnologías por capa.

Tabla 3

Stack tecnológico

Capa / aspecto	Tecnología
Frontend	React 18 + TypeScript + Vite + Material UI (MUI)
Backend	Node.js + Express + TypeScript (API REST bajo <code>/api</code>)
Base de datos	PostgreSQL 14 con ORM Sequelize
Autenticación	JWT (access + refresh); contraseñas con bcrypt
Segundo factor	TOTP (speakeasy + qrcode)
Correo	Nodemailer (SMTP)
OCR / PDF	tesseract.js, pdf-parse
Archivos / importación	multer; xlsx
Programación de tareas	node-cron
Contenedores	Docker Compose (perfiles dev y prod, nginx en prod)

Reglas de negocio principales

- **Precio del almuerzo:** `MEAL_PRICE` = 2000 (COP). Los almuerzos incluidos en un pago se calculan como $\lfloor \text{monto} / 2000 \rfloor$; el residuo ($\text{monto} \bmod 2000$) no genera almuerzo.
- **Monto mínimo de recarga:** un pago debe ser ≥ 2000 ; en caso contrario se rechaza con HTTP 400.
- **Almuerzos disponibles:** se calculan sobre los pagos *verificados* como $\sum(\text{mealsIncluded} - \text{mealsUsed})$.
- **Un almuerzo por día:** un estudiante no puede registrar dos almuerzos el mismo día.
- **Días autorizados:** el almuerzo solo se registra en los días del comedor asignados al estudiante (`diasComedor`).
- **Elegibilidad SISBÉN:** las categorías válidas pertenecen a los grupos A, B y C; la validación coteja cédula, nombre y apellido contra la cédula frontal.
- **Autorización de cuentas:** un usuario con `isAuthorized` = `false` no puede iniciar sesión.
- **Invitación de supervisores:** el enlace de invitación es válido por 48 horas.
- **Ciclos:** un ciclo vencido (`endDate` < hoy) con estado **Activo** se cierra automáticamente.

Modelo de requerimientos

Descripción

El modelo de requerimientos especifica qué debe hacer el sistema (requisitos funcionales, RF) y bajo qué condiciones de calidad (requisitos no funcionales, RNF) (Sommerville, 2011). Los requisitos se derivaron del análisis del problema, de las rutas REST implementadas, de los controladores y de las reglas de negocio verificadas por las pruebas. Se emplea la convención RF-XX y RNF-XX, con prioridad Alta, Media o Baja.

Requisitos funcionales

Tabla 4*Requisitos funcionales del sistema*

ID	Requisito	Prioridad
RF-01	Registro de estudiante con datos personales/académicos y carga de archivos (SISBÉN, cédula frontal, horario PDF, recibo de pago).	Alta
RF-02	Iniciar sesión con email, contraseña y rol, retornando access token y refresh token JWT.	Alta
RF-03	Rechazar el inicio de sesión ante campos faltantes (400), credenciales inválidas (401), usuario inactivo (401), rol no coincidente (403) o cuenta no autorizada (403).	Alta
RF-04	Refrescar el access token a partir de un refresh token válido.	Alta
RF-05	Consultar y actualizar el perfil del usuario autenticado.	Media
RF-06	Cambiar la contraseña validando la contraseña actual.	Media
RF-07	Recuperar la contraseña por correo con token, sin revelar si el email existe.	Alta
RF-08	Restablecer la contraseña con un token válido y no expirado.	Alta
RF-09	Configurar y verificar 2FA (TOTP) exclusivamente para estudiantes.	Media
RF-10	Listar estudiantes con paginación y búsqueda.	Alta
RF-11	Consultar un estudiante por ID, incluyendo sus almuerzos disponibles.	Alta
RF-12	Crear un estudiante y su usuario asociado.	Alta
RF-13	Actualizar y deshabilitar (soft delete) un estudiante.	Alta
RF-14	Importar estudiantes masivamente desde Excel (.xlsx).	Alta

continúa en la siguiente página

Tabla 4 – *continuación*

ID	Requisito	Prioridad
RF-15	Buscar estudiantes (por cédula y otros filtros) para el registro de almuerzo.	Alta
RF-16	Calcular los almuerzos disponibles a partir de los pagos verificados.	Alta
RF-17	Validar el SISBÉN cotejando cédula, nombre y apellido contra la cédula frontal.	Alta
RF-18	Actualizar el ciclo de un estudiante (revalidación/deshabilitación).	Media
RF-19	Listar supervisores y sus bitácoras.	Media
RF-20	Crear un supervisor con contraseña temporal.	Alta
RF-21	Actualizar, deshabilitar y cambiar el estado (activo/autorizado) de un supervisor.	Media
RF-22	Generar un enlace de invitación válido por 48 h para auto-registro de supervisor.	Media
RF-23	Unirse mediante invitación validando el token.	Media
RF-24	Asignar y remover estudiantes a un supervisor, evitando duplicados.	Media
RF-25	Consultar los estudiantes asignados a un supervisor.	Media
RF-26	Crear un pago asociado a un estudiante, calculando los almuerzos incluidos.	Alta
RF-27	Cargar los comprobantes del pago (comprobante, recibo universidad, recibo banco).	Alta
RF-28	Calcular los almuerzos de un monto, rechazando montos < \$2000.	Alta

continúa en la siguiente página

Tabla 4 – *continuación*

ID	Requisito	Prioridad
RF-29	Listar pagos con paginación y filtros (por estudiante y por estado).	Alta
RF-30	Verificar un pago (marcar <code>isVerified</code> , registrar <code>verifiedBy</code> y <code>verifiedAt</code>).	Alta
RF-31	Registrar el almuerzo validando estudiante activo, día autorizado, no haber comido hoy y disponibilidad.	Alta
RF-32	Decrementar la disponibilidad y registrar un <code>SupervisorLog</code> al registrar el almuerzo.	Alta
RF-33	Consultar el historial de asistencia con filtros por estudiante y fechas.	Media
RF-34	Consultar la asistencia del día.	Media
RF-35	Ofrecer analítica del tablero (métricas agregadas).	Media
RF-36	Crear, listar y consultar ciclos.	Media
RF-37	Cerrar un ciclo manualmente.	Media
RF-38	Cerrar automáticamente los ciclos vencidos y gestionar la revalidación.	Media
RF-39	Calificar el servicio (1–5 estrellas + comentario opcional).	Media
RF-40	Consultar calificaciones y su promedio/distribución.	Baja
RF-41	Crear quejas, sugerencias o comentarios, de forma anónima o identificada.	Media
RF-42	Responder y marcar como resueltas las quejas.	Media
RF-43	Publicar, editar y eliminar (soft delete) noticias, y listarlas.	Baja
RF-44	Registrar en auditoría las acciones y eventos HTTP sensibles.	Alta

continúa en la siguiente página

Tabla 4 – *continuación*

ID	Requisito	Prioridad
RF-45	Consultar y filtrar la auditoría (acción, método, código, ruta, rol, fechas) con paginación.	Alta

Requisitos no funcionales

Los RNF se organizan según el modelo de calidad ISO/IEC 25010 (International Organization for Standardization, [2011](#)).

Tabla 5

Requisitos no funcionales del sistema

ID	Categoría	Requisito
RNF-01	Seguridad	Contraseñas almacenadas cifradas con bcrypt (hash + salt).
RNF-02	Seguridad	Control de acceso por JWT (access 15 min, refresh 7 días) y autorización por rol.
RNF-03	Seguridad	Rechazo de peticiones sin token (401) y de rol no permitido (403).
RNF-04	Seguridad	No filtrar trazas de error (stack traces); mensajes genéricos.
RNF-05	Seguridad	Recuperación de contraseña sin revelar si un correo existe.
RNF-06	Seguridad	Cabeceras de seguridad (nosniff, anti-clickjacking) y sin X-Powered-By.
RNF-07	Seguridad	Validación de filtros de auditoría contra listas blancas.
RNF-08	Seguridad	2FA (TOTP) como segundo factor para estudiantes.
RNF-09	Rendimiento	Consultas críticas y autenticación responden en menos de 3 s.

Tabla 5 – *continuación*

ID	Categoría	Requisito
RNF-10	Rendimiento	Soportar pruebas de carga/estrés (p. ej. 200 hilos) con umbral configurable.
RNF-11	Rendimiento	Listados paginados para acotar el volumen por respuesta.
RNF-12	Disponibilidad	Healthcheck de PostgreSQL y perfiles dev/prod.
RNF-13	Disponibilidad	Endpoint de salud GET /api/health.
RNF-14	Disponibilidad	Backups diarios (<code>pg_dump</code>) con retención de 30 días.
RNF-15	Disponibilidad	Cierre de ciclos y revalidación automáticos y programados.
RNF-16	Usabilidad	UI con componentes MUI responsive, labels, estados y mensajes guiados.
RNF-17	Usabilidad	Experiencia diferenciada por rol.
RNF-18	Usabilidad	Formularios con label/aria-label y landmark main.
RNF-19	Portabilidad	Funcionar en Chrome, Firefox, Edge y Safari (escritorio).
RNF-20	Portabilidad	Desplegable en cualquier entorno con Docker.
RNF-21	Portabilidad	Operar con APIs estándar del navegador sin polyfills.
RNF-22	Mantenibilidad	Código en TypeScript (tipado estático) en cliente y servidor.
RNF-23	Mantenibilidad	Auditoría de acciones sensibles y bitácora de supervisor.
RNF-24	Mantenibilidad	Identificador legible (<code>uid</code>) por entidad con prefijo.
RNF-25	Mantenibilidad	Batería de pruebas automatizadas y reportes de cobertura.

Modelo de casos de uso

Diagramas de caso de uso

La Figura 2 presenta el diagrama general de casos de uso. La Tabla 6 lista el catálogo de casos de uso del sistema.

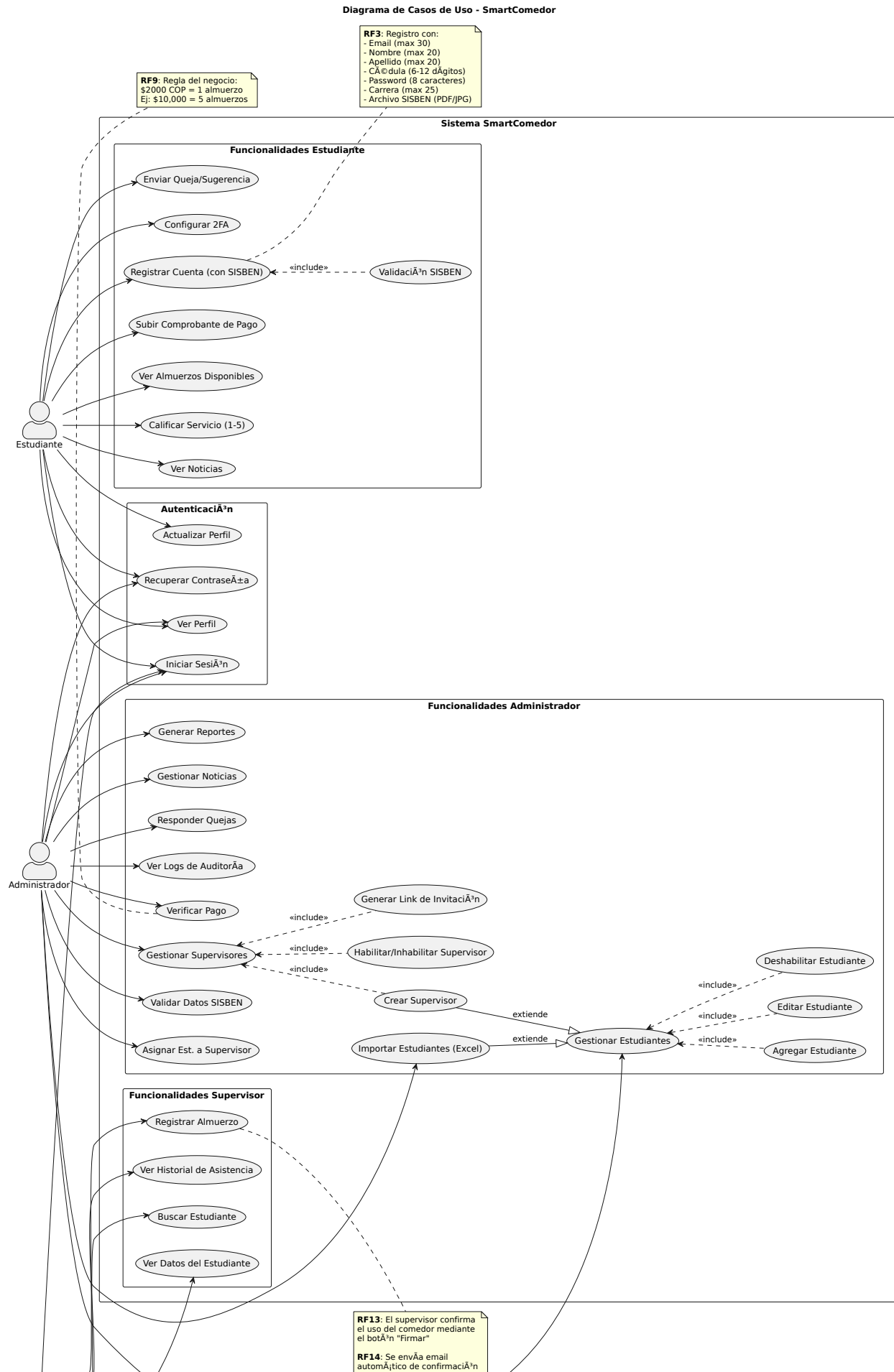


Tabla 6*Catálogo de casos de uso*

ID	Caso de uso	Actor principal
CU-01	Registrar estudiante	Estudiante
CU-02	Iniciar sesión	Todos
CU-03	Recuperar/restablecer contraseña	Todos
CU-04	Configurar y verificar 2FA	Estudiante
CU-05	Gestionar estudiantes (CRUD)	Administrador
CU-06	Importar estudiantes desde Excel	Administrador
CU-07	Validar SISBÉN	Administrador
CU-08	Gestionar supervisores	Administrador
CU-09	Invitar supervisor / unirse por invitación	Admin / Supervisor
CU-10	Asignar estudiantes a supervisor	Administrador
CU-11	Subir pago y comprobantes	Estudiante
CU-12	Verificar pago	Administrador
CU-13	Calcular almuerzos por monto	Autenticado
CU-14	Registrar almuerzo	Supervisor
CU-15	Consultar historial / asistencia del día	Autenticado
CU-16	Consultar analítica del tablero	Admin / Auditor
CU-17	Gestionar ciclos / cierre automático	Admin / Sistema
CU-18	Calificar servicio	Estudiante
CU-19	Enviar y responder quejas	Estudiante / Admin
CU-20	Gestionar noticias	Administrador
CU-21	Consultar auditoría	Admin / Auditor

Descripción de casos de uso

Esta sección documenta los veintinueve casos de uso del sistema con la plantilla extendida (objetivo, precondiciones, condiciones de éxito y fracaso, actores y flujo de eventos). Cada caso se

acompana de su diagrama de caso de uso, su diagrama de secuencia y su diagrama de actividades, derivados de los controladores reales y verificados por las pruebas de la Segunda Parte.

CU-01 — Registrar estudiante

Endpoint(s): POST /api/auth/register

Tabla 7

Ficha del caso de uso CU-01 — Registrar estudiante

Campo	Descripcion
Objetivo del contexto	Permitir que un aspirante cree su cuenta de estudiante en SmartComedor adjuntando los documentos requeridos (SISBEN, cedula y horario), para acceder al servicio de comedor.
Precondiciones	El correo electronico no debe estar registrado previamente.
Condicion de exito	Se crea el usuario (rol estudiante) y su perfil de estudiante con SISBEN sin validar.
Condicion de fracaso	El registro se rechaza por falta de documento SISBEN o por correo ya existente.
Actores principales	Estudiante (no autenticado)
Actores secundarios	Servicio de almacenamiento de archivos

Tabla 8

Flujo principal de CU-01

Paso	Accion
1	El estudiante diligencia el formulario con sus datos personales y academicos.
2	Adjunta el documento SISBEN, la cedula frontal y el horario.
3	El sistema valida la presencia del archivo SISBEN.
4	El sistema verifica que el correo no exista.
5	El sistema cifra la contrasena (bcrypt) y crea el usuario y el estudiante.
6	El sistema responde 201 Created.

Tabla 9

Extensiones (flujos alternativos) de CU-01

Paso	Accion
3.1	Falta el archivo SISBEN → HTTP 400.
4.1	Correo ya registrado → HTTP 400.

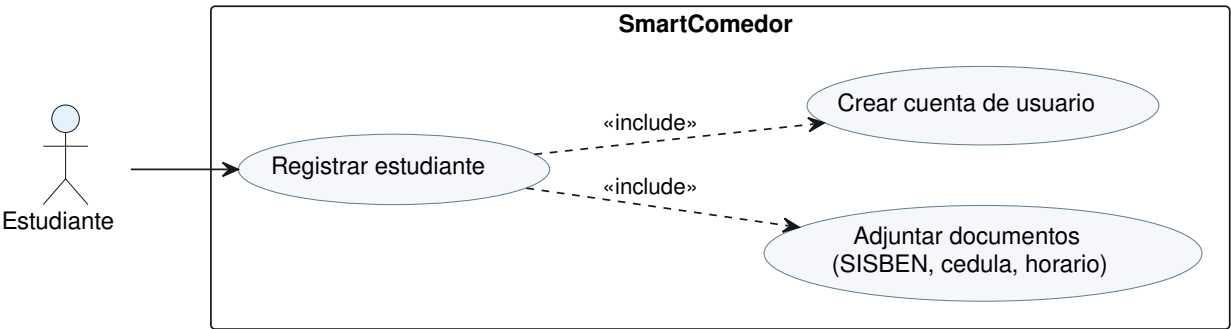
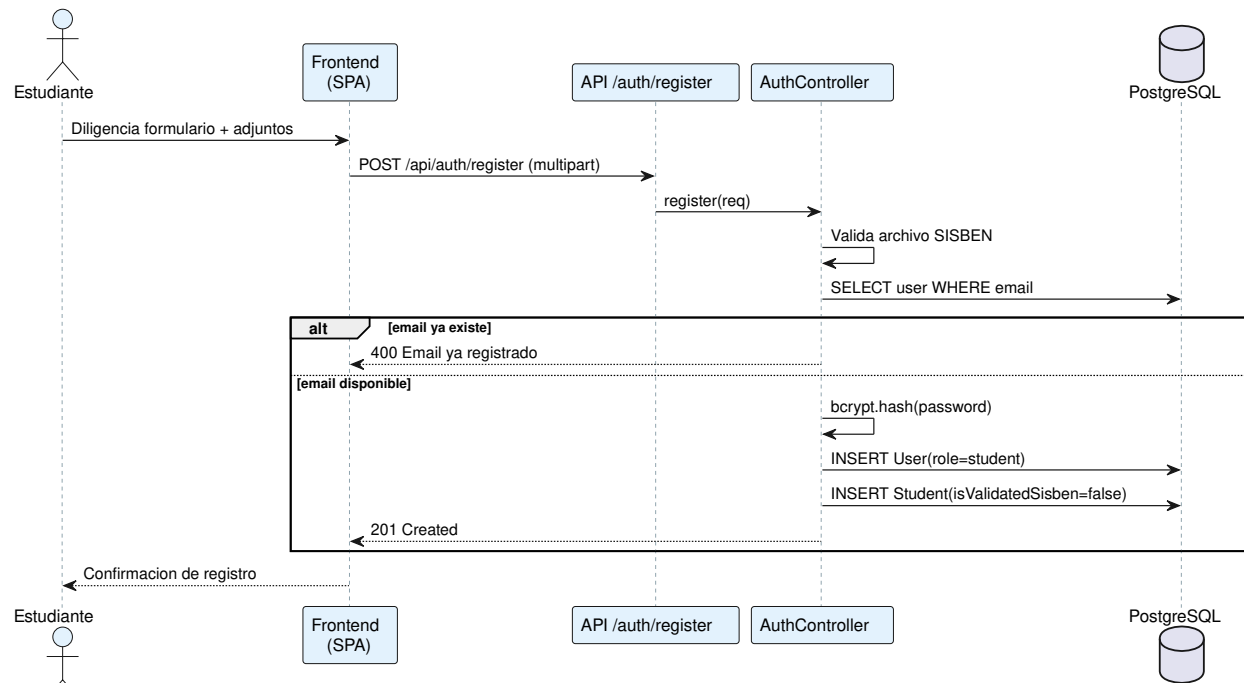


Figura 3

Diagrama de caso de uso — CU-01

**Figura 4***Diagrama de secuencia — CU-01*

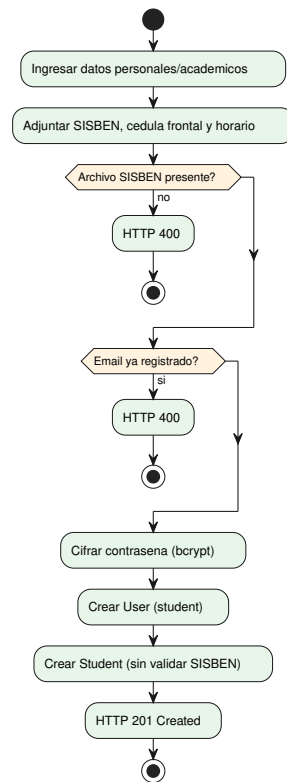
**Figura 5**

Diagrama de actividades — CU-01

CU-02 — Iniciar sesion

Endpoint(s): POST /api/auth/login

Tabla 10*Ficha del caso de uso CU-02 — Iniciar sesion*

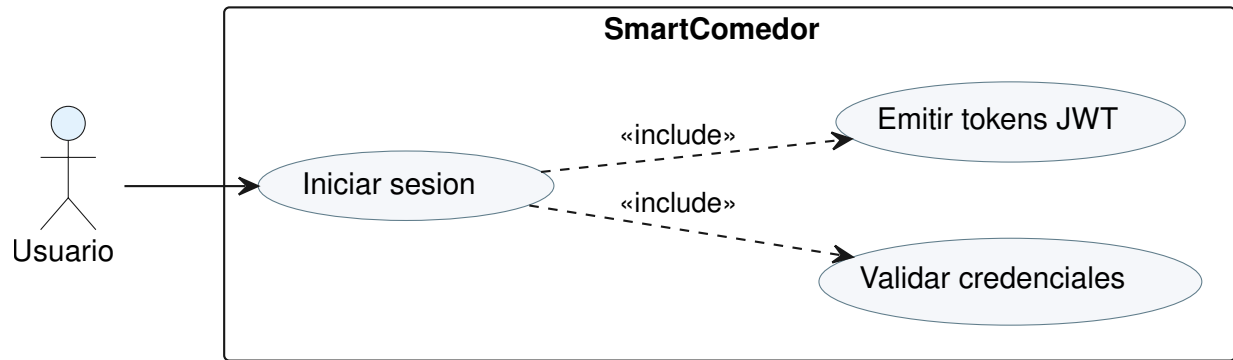
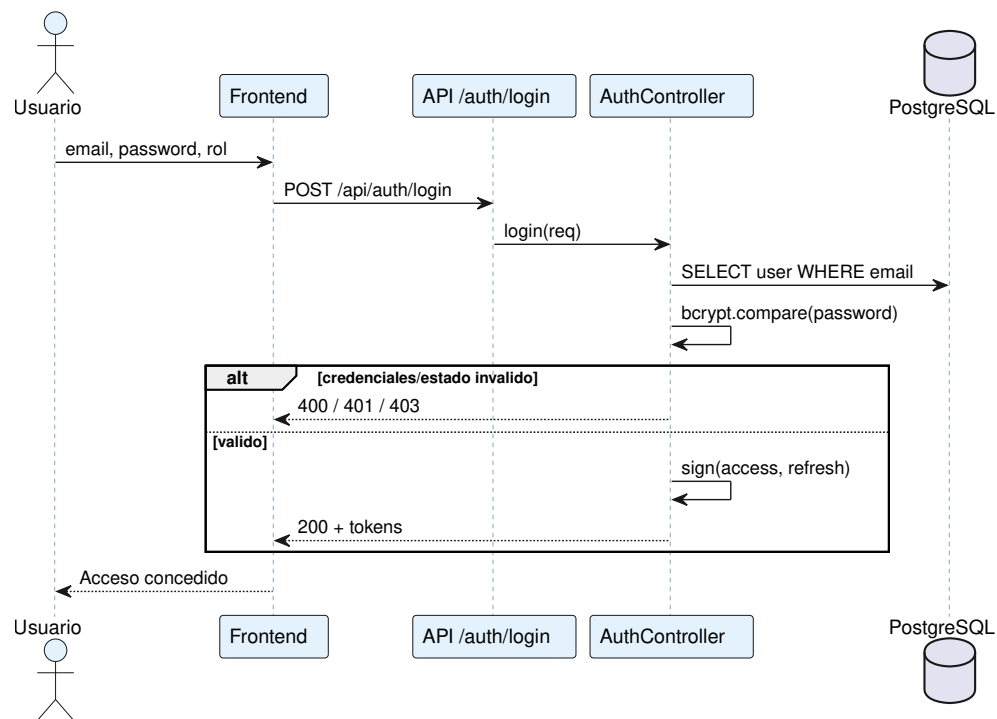
Campo	Descripcion
Objetivo del contexto	Permitir a un usuario autenticarse en el sistema mediante correo, contraseña y rol, obteniendo los tokens de acceso.
Precondiciones	El usuario debe estar registrado y su cuenta activa.
Condicion de exito	Se emiten los tokens de acceso y refresco; el usuario accede segun su rol.
Condicion de fracaso	Acceso denegado por credenciales, estado o rol invalidos.
Actores principales	Usuario (estudiante, supervisor, administrador, auditor)
Actores secundarios	Servicio de autenticacion JWT

Tabla 11*Flujo principal de CU-02*

Paso	Accion
1	El usuario ingresa correo, contraseña y rol.
2	El sistema valida que los campos esten completos.
3	El sistema busca al usuario por correo.
4	El sistema verifica la contraseña con bcrypt.compare.
5	El sistema valida que la cuenta este activa, autorizada y que el rol coincida.
6	El sistema emite access y refresh token y responde 200.

Tabla 12*Extensiones (flujos alternativos) de CU-02*

Paso	Accion
2.1	Campos faltantes → HTTP 400.
3.1	Usuario inexistente → HTTP 401.
4.1	Contraseña incorrecta → HTTP 401.
5.1	Cuenta inactiva → 401; rol no coincide o no autorizado → 403.

**Figura 6***Diagrama de caso de uso — CU-02***Figura 7***Diagrama de secuencia — CU-02*

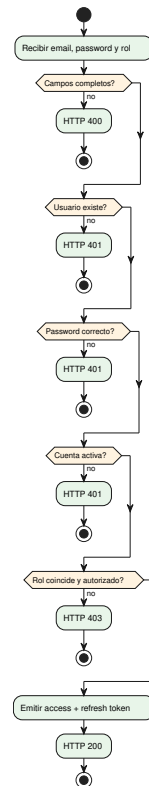
**Figura 8**

Diagrama de actividades — CU-02

CU-03 — Recuperar y restablecer contraseña

Endpoint(s): POST /api/auth/forgot-password, POST /api/auth/reset-password

Tabla 13*Ficha del caso de uso CU-03 — Recuperar y restablecer contraseña*

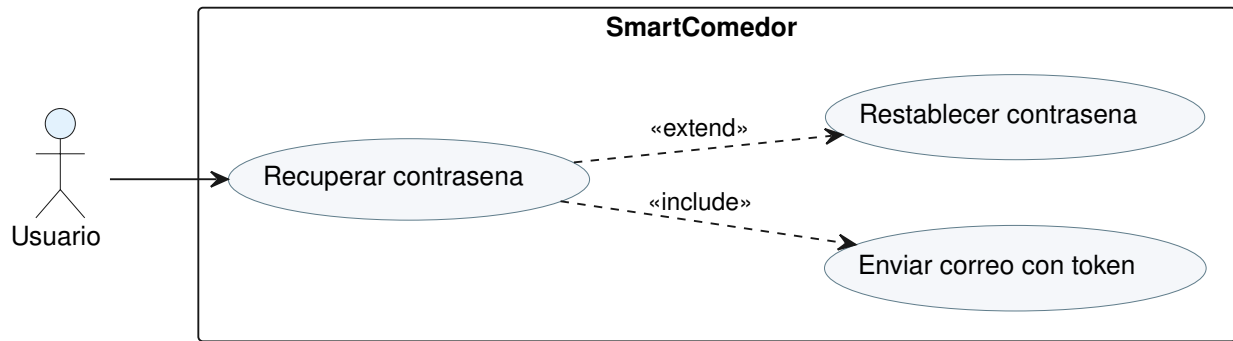
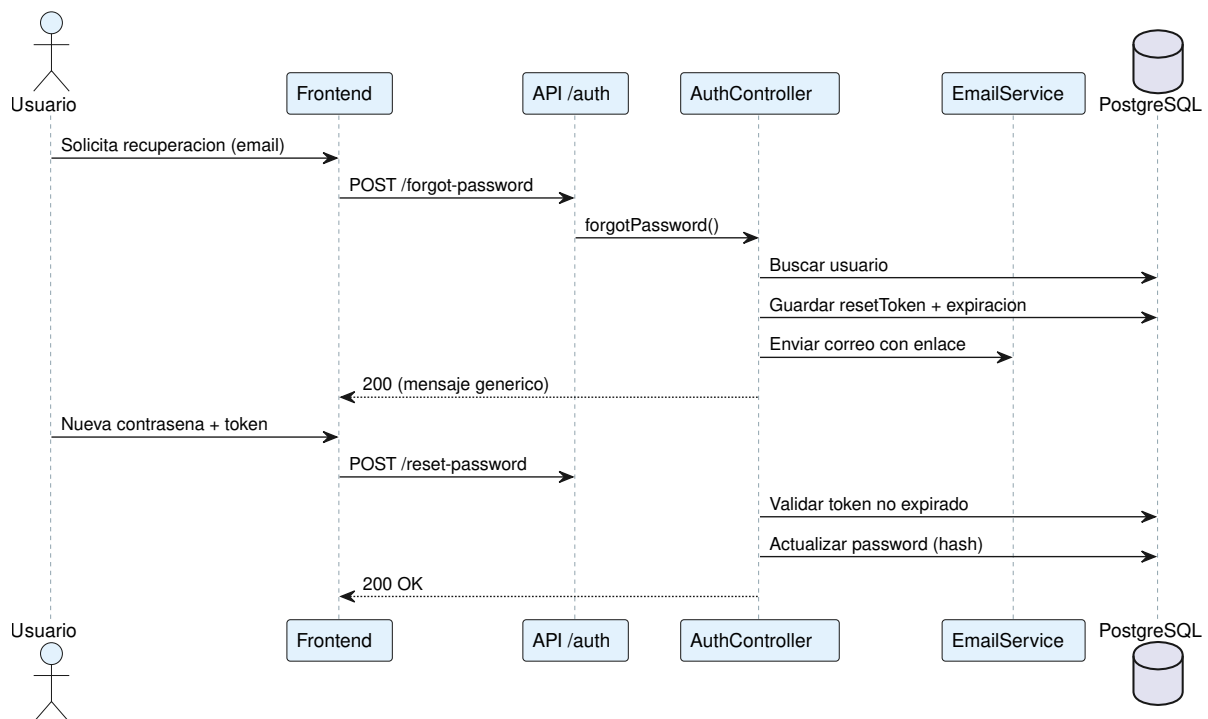
Campo	Descripcion
Objetivo del contexto	Permitir al usuario recuperar el acceso cuando olvida su contraseña, mediante un enlace con token enviado por correo.
Precondiciones	El usuario debe tener un correo registrado.
Condicion de exito	La contraseña se actualiza y el usuario puede iniciar sesion.
Condicion de fracaso	El token es invalido o esta expirado.
Actores principales	Usuario
Actores secundarios	Servicio de correo (Nodemailer)

Tabla 14*Flujo principal de CU-03*

Paso	Accion
1	El usuario solicita recuperacion indicando su correo.
2	El sistema genera un token con expiracion y lo almacena.
3	El sistema envia un correo con el enlace (mensaje generico).
4	El usuario abre el enlace e ingresa la nueva contraseña.
5	El sistema valida el token y actualiza la contraseña cifrada.
6	El sistema responde 200.

Tabla 15*Extensiones (flujos alternativos) de CU-03*

Paso	Accion
3.1	Por seguridad, el mensaje no revela si el correo existe.
5.1	Token invalido o expirado → HTTP 400.

**Figura 9***Diagrama de caso de uso — CU-03***Figura 10***Diagrama de secuencia — CU-03*

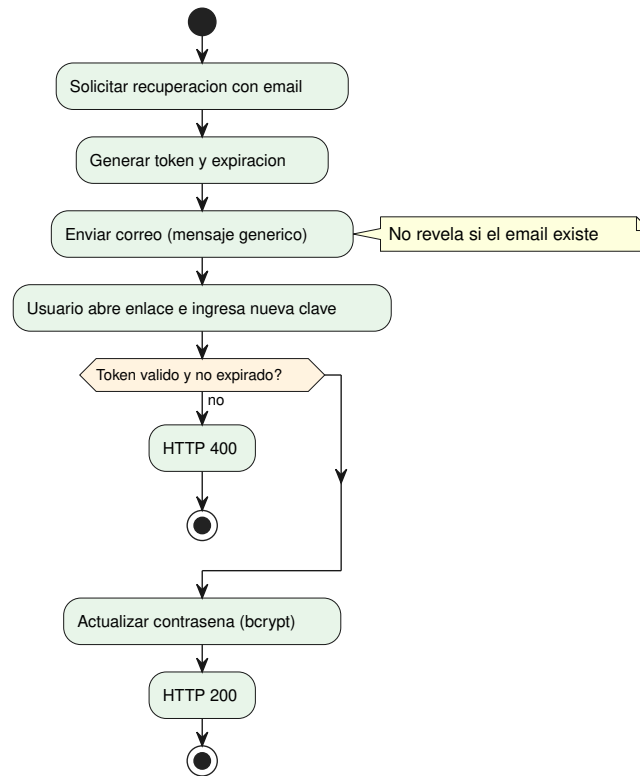
**Figura 11**

Diagrama de actividades — CU-03

CU-04 — Configurar y verificar 2FA

Endpoint(s): POST /api/auth/setup-2fa, POST /api/auth/verify-2fa

Tabla 16*Ficha del caso de uso CU-04 — Configurar y verificar 2FA*

Campo	Descripcion
Objetivo del contexto	Permitir al estudiante activar un segundo factor de autenticacion (TOTP) para reforzar la seguridad de su cuenta.
Precondiciones	El usuario debe estar autenticado.
Condicion de exito	El segundo factor queda activo y se exige en posteriores inicios de sesion.
Condicion de fracaso	El codigo TOTP ingresado es invalido.
Actores principales	Estudiante
Actores secundarios	Aplicacion autenticadora (TOTP)

Tabla 17*Flujo principal de CU-04*

Paso	Accion
1	El usuario solicita activar 2FA.
2	El sistema genera un secreto TOTP y un codigo QR.
3	El usuario escanea el QR en su aplicacion autenticadora.
4	El usuario ingresa el codigo de 6 digitos.
5	El sistema verifica el codigo con speakeasy y activa el 2FA.
6	El sistema responde 200.

Tabla 18*Extensiones (flujos alternativos) de CU-04*

Paso	Accion
5.1	Codigo TOTP invalido → HTTP 400.

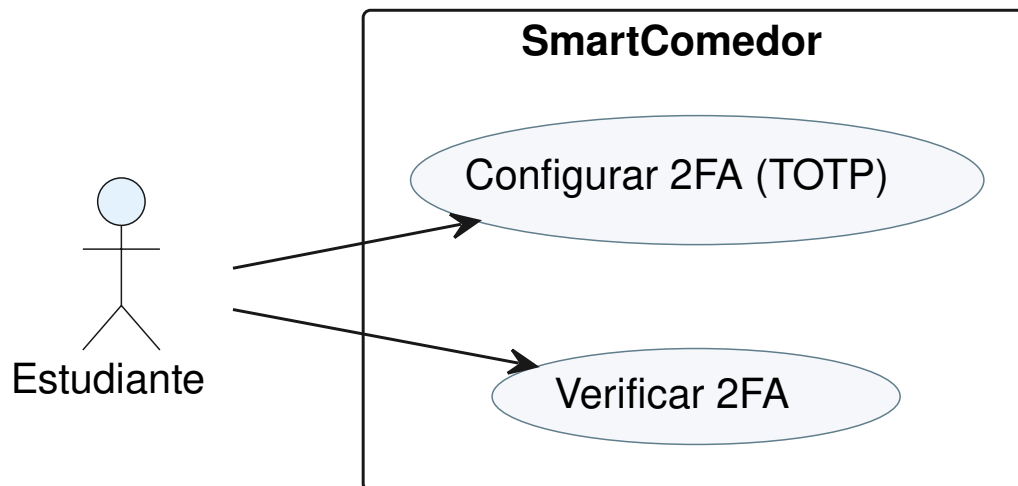
**Figura 12**

Diagrama de caso de uso — CU-04

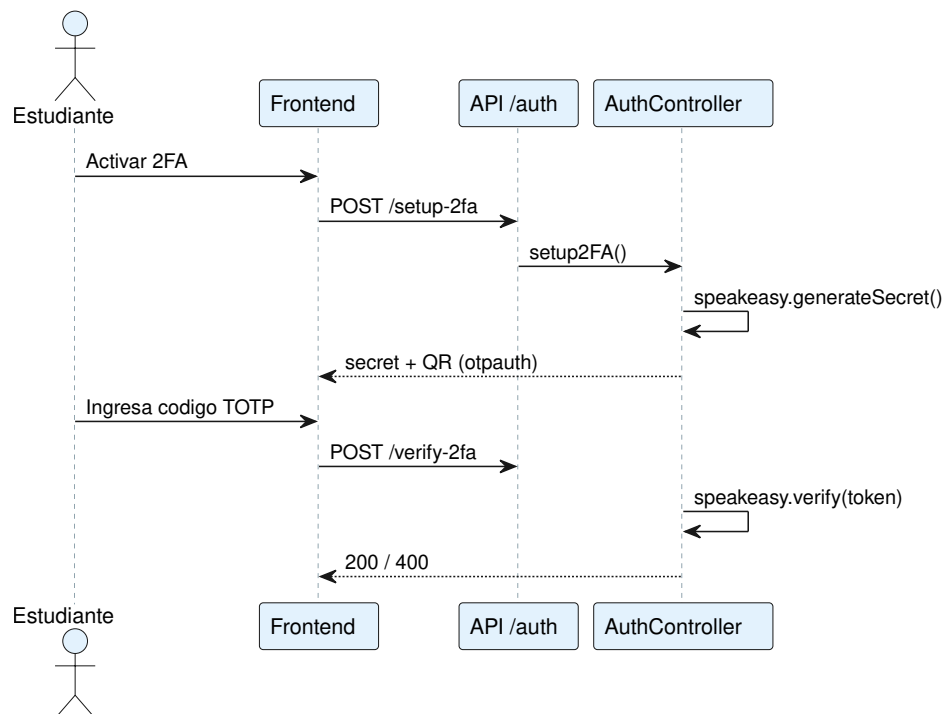
**Figura 13**

Diagrama de secuencia — CU-04

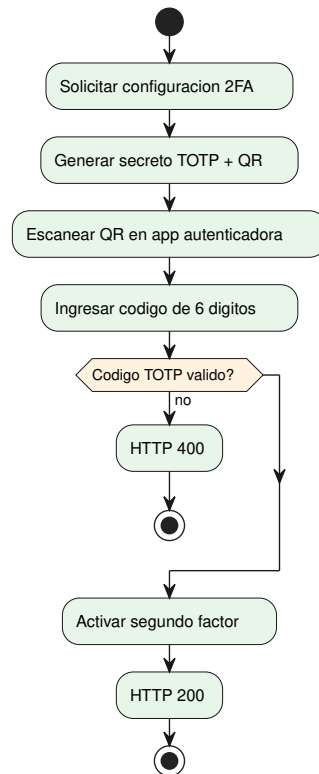
**Figura 14**

Diagrama de actividades — CU-04

CU-05 — Gestionar estudiantes (CRUD)

Endpoint(s): GET/POST/PUT/DELETE /api/students

Tabla 19*Ficha del caso de uso CU-05 — Gestionar estudiantes (CRUD)*

Campo	Descripcion
Objetivo del contexto	Permitir al administrador listar, consultar, crear, actualizar y deshabilitar estudiantes del sistema.
Precondiciones	El administrador debe estar autenticado con rol admin.
Condicion de exito	La operacion solicitada se refleja en la base de datos.
Condicion de fracaso	Datos invalidos o estudiante inexistente.
Actores principales	Administrador
Actores secundarios	Base de datos

Tabla 20*Flujo principal de CU-05*

Paso	Accion
1	El administrador abre la gestion de estudiantes.
2	El sistema lista los estudiantes con paginacion y filtros.
3	El administrador selecciona crear, editar o deshabilitar.
4	El sistema valida los datos y ejecuta la operacion.
5	El sistema responde con el recurso afectado.

Tabla 21*Extensiones (flujos alternativos) de CU-05*

Paso	Accion
4.1	Datos invalidos → HTTP 400.
4.2	Estudiante inexistente → HTTP 404.

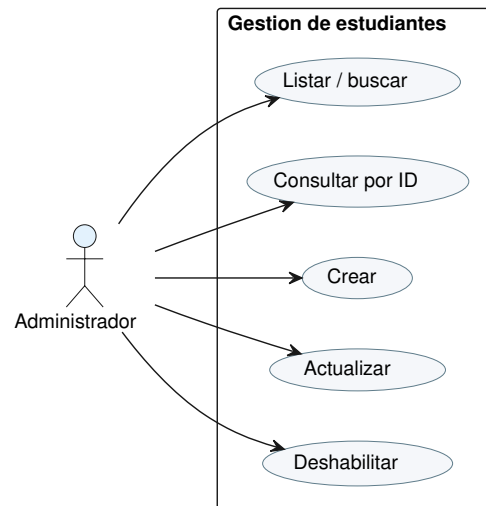
**Figura 15**

Diagrama de caso de uso — CU-05

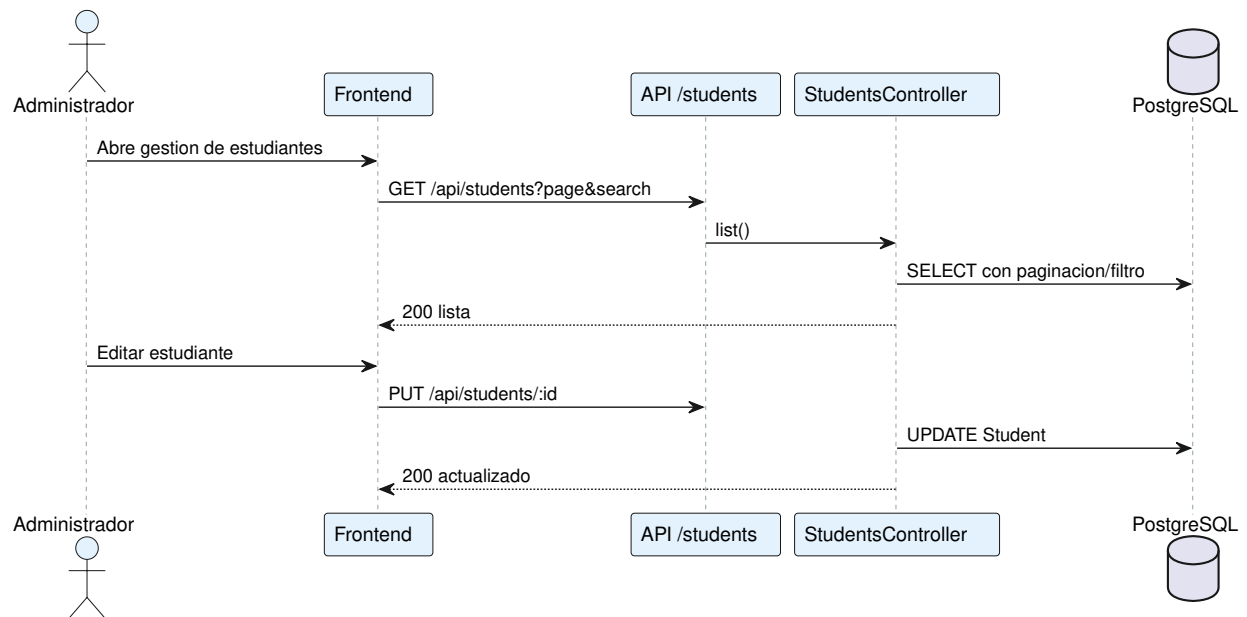
**Figura 16**

Diagrama de secuencia — CU-05

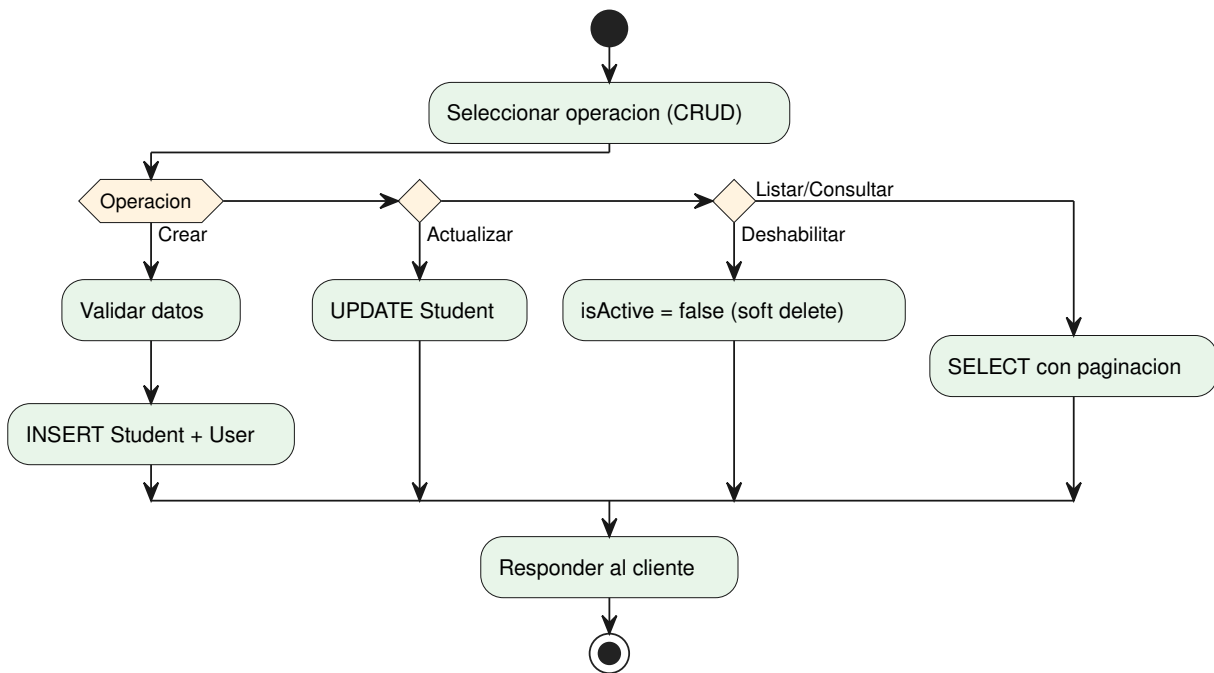
**Figura 17**

Diagrama de actividades — CU-05

CU-06 — Importar estudiantes desde Excel

Endpoint(s): POST /api/students/import

Tabla 22*Ficha del caso de uso CU-06 — Importar estudiantes desde Excel*

Campo	Descripcion
Objetivo del contexto	Permitir al administrador cargar masivamente estudiantes a partir de un archivo Excel.
Precondiciones	El administrador debe estar autenticado; el archivo debe tener el formato esperado.
Condicion de exito	Se crean los estudiantes de las filas validas y se reporta el resultado.
Condicion de fracaso	El archivo es invalido o no contiene filas procesables.
Actores principales	Administrador
Actores secundarios	Libreria de parseo xlsx

Tabla 23*Flujo principal de CU-06*

Paso	Accion
1	El administrador sube un archivo .xlsx.
2	El sistema parsea las filas del archivo.
3	Por cada fila valida, el sistema crea el estudiante y su usuario.
4	El sistema acumula errores de las filas invalidas.
5	El sistema responde con un resumen (creados/errores).

Tabla 24*Extensiones (flujos alternativos) de CU-06*

Paso	Accion
2.1	Archivo invalido o vacio → HTTP 400.
3.1	Fila con datos invalidos → se registra el error y continua.

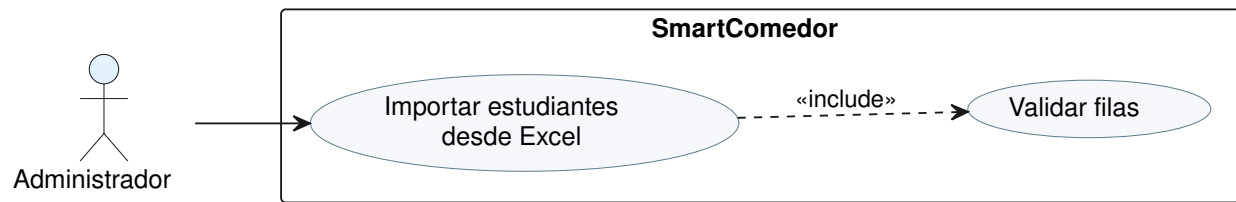
**Figura 18**

Diagrama de caso de uso — CU-06

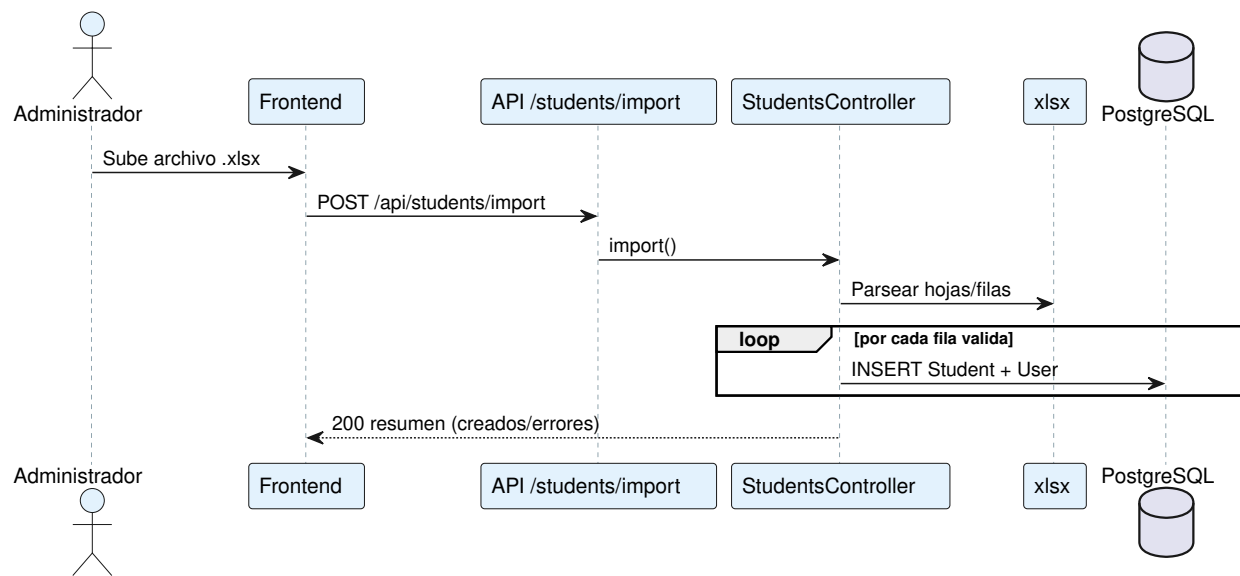
**Figura 19**

Diagrama de secuencia — CU-06

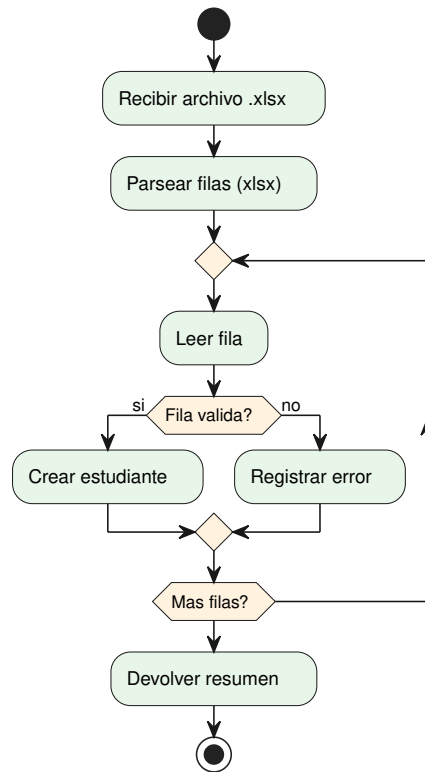
**Figura 20**

Diagrama de actividades — CU-06

CU-07 — Validar SISBEN

Endpoint(s): POST /api/students/:id/validate-sisben

Tabla 25*Ficha del caso de uso CU-07 — Validar SISBEN*

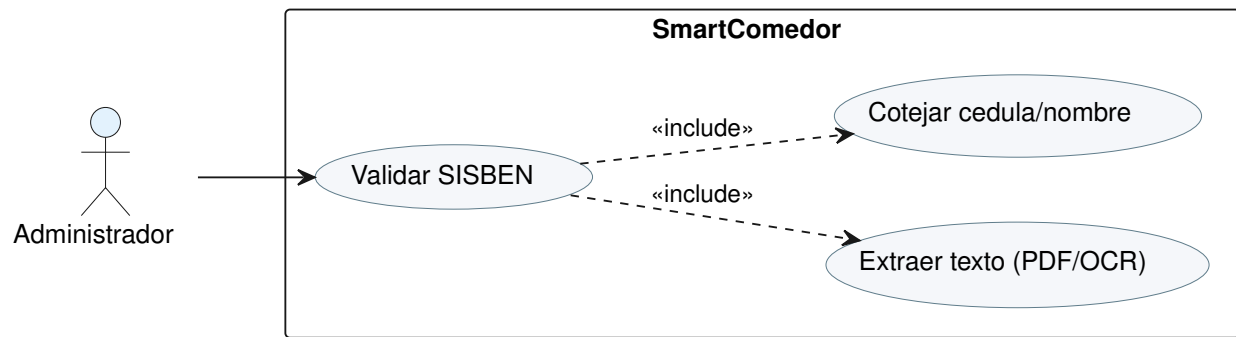
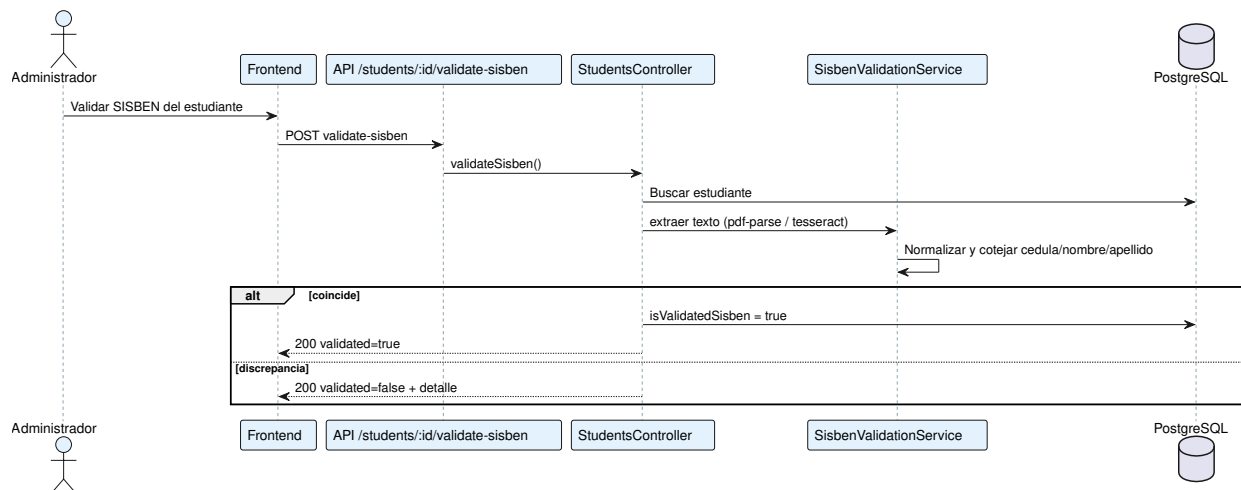
Campo	Descripcion
Objetivo del contexto	Permitir al administrador validar la elegibilidad del estudiante cotejando el documento SISBEN con la cedula registrada.
Precondiciones	El estudiante debe existir y tener documentos adjuntos.
Condicion de exito	El estudiante queda con SISBEN validado.
Condicion de fracaso	Existen discrepancias entre los documentos.
Actores principales	Administrador
Actores secundarios	Servicio de validacion SISBEN (PDF/OCR)

Tabla 26*Flujo principal de CU-07*

Paso	Accion
1	El administrador solicita validar el SISBEN del estudiante.
2	El sistema extrae el texto del documento (pdf-parse u OCR).
3	El sistema normaliza y coteja cedula, nombre y apellido.
4	Si todo coincide, marca isValidatedSisben = true y responde 200.

Tabla 27*Extensiones (flujos alternativos) de CU-07*

Paso	Accion
1.1	Estudiante no encontrado → HTTP 404.
3.1	Inconsistencia → validated=false con el detalle de cada discrepancia.

**Figura 21***Diagrama de caso de uso — CU-07***Figura 22***Diagrama de secuencia — CU-07*

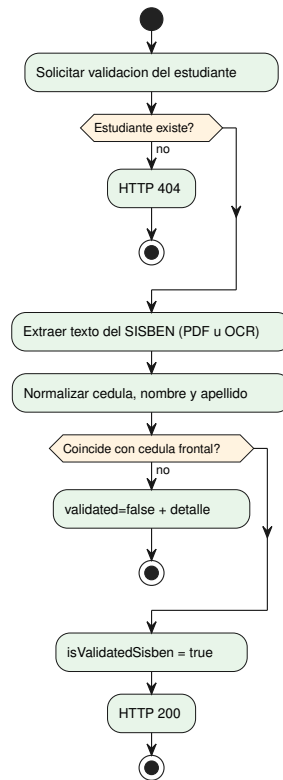
**Figura 23**

Diagrama de actividades — CU-07

CU-08 — Gestionar supervisores

Endpoint(s): GET/POST/PUT/PATCH/DELETE /api/supervisors

Tabla 28*Ficha del caso de uso CU-08 — Gestionar supervisores*

Campo	Descripcion
Objetivo del contexto	Permitir al administrador crear, actualizar, cambiar de estado y deshabilitar supervisores, y consultar sus bitacoras.
Precondiciones	El administrador debe estar autenticado con rol admin.
Condicion de exito	El supervisor se crea o actualiza segun la operacion.
Condicion de fracaso	Datos invalidos o supervisor inexistente.
Actores principales	Administrador
Actores secundarios	Base de datos

Tabla 29*Flujo principal de CU-08*

Paso	Accion
1	El administrador selecciona la operacion sobre el supervisor.
2	Para crear, el sistema genera una contrasena temporal cifrada.
3	El sistema persiste el usuario con rol supervisor.
4	Para cambiar estado, actualiza isActive/isActiveAuthorized.
5	El sistema responde con el resultado.

Tabla 30*Extensiones (flujos alternativos) de CU-08*

Paso	Accion
3.1	Correo duplicado → HTTP 400.
4.1	Supervisor inexistente → HTTP 404.

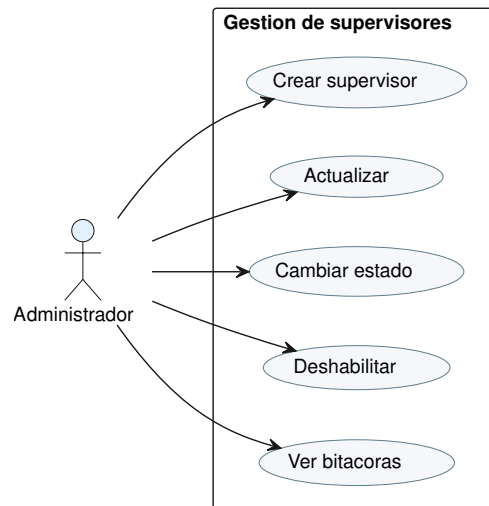
**Figura 24**

Diagrama de caso de uso — CU-08

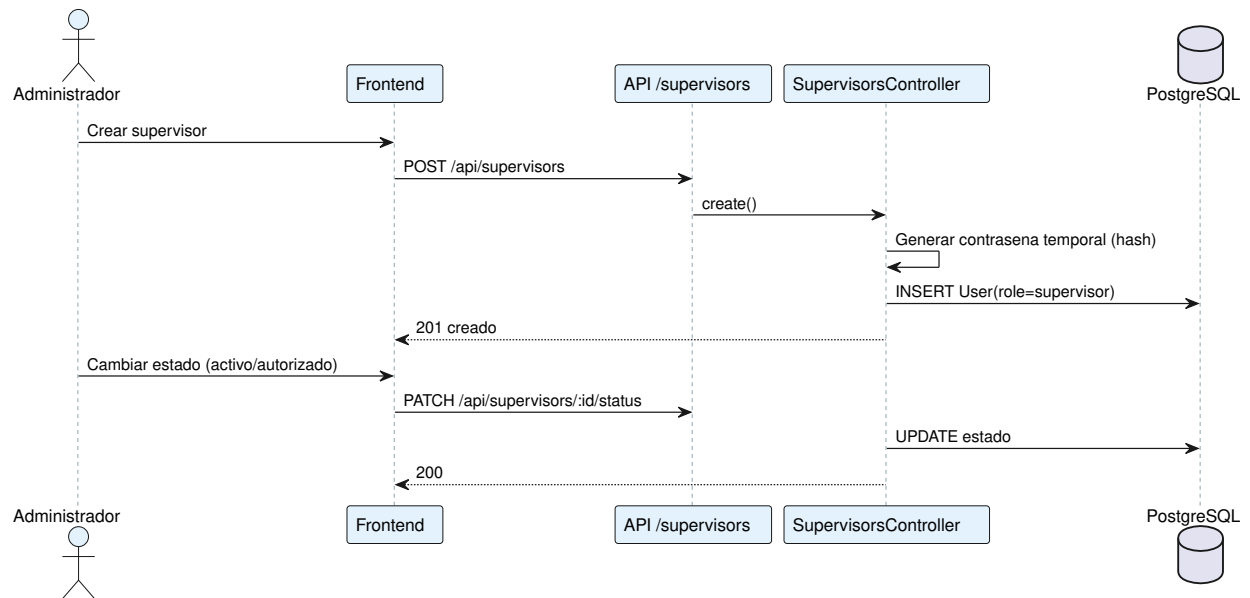
**Figura 25**

Diagrama de secuencia — CU-08

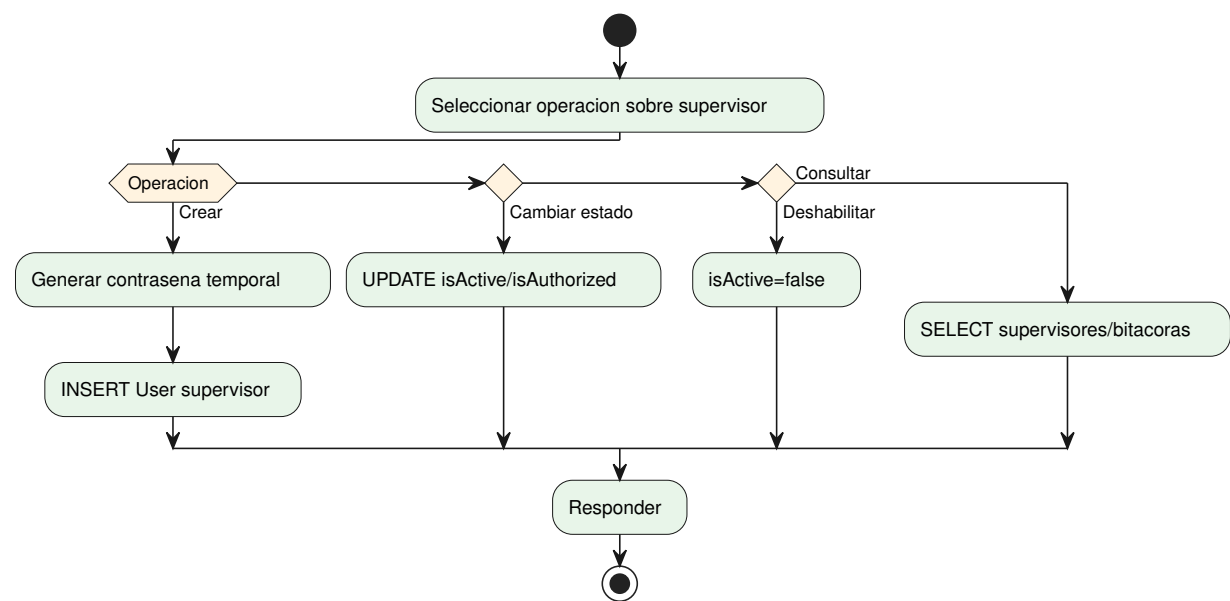


Figura 26
Diagrama de actividades — CU-08

CU-09 — Invitar supervisor / unirse por invitacion

Endpoint(s): POST /api/supervisors/invite, POST /api/supervisors/join

Tabla 31

Ficha del caso de uso CU-09 — Invitar supervisor / unirse por invitacion

Campo	Descripcion
Objetivo del contexto	Permitir al administrador invitar supervisores por correo y a estos crear su cuenta mediante un enlace temporal.
Precondiciones	El administrador debe estar autenticado; la invitacion debe estar vigente.
Condicion de exito	El supervisor crea su cuenta a partir de la invitacion.
Condicion de fracaso	El token de invitacion es invalido o expirado.
Actores principales	Administrador, Supervisor
Actores secundarios	Servicio de correo

Tabla 32

Flujo principal de CU-09

Paso	Accion
1	El administrador genera una invitacion indicando el correo.
2	El sistema crea un token con vigencia de 48 horas.
3	El sistema envia el enlace de invitacion por correo.
4	El supervisor abre el enlace y completa sus datos.
5	El sistema valida el token y crea la cuenta de supervisor.

Tabla 33

Extensiones (flujos alternativos) de CU-09

Paso	Accion
5.1	Token vencido → HTTP 400.
5.2	Correo duplicado → HTTP 400.

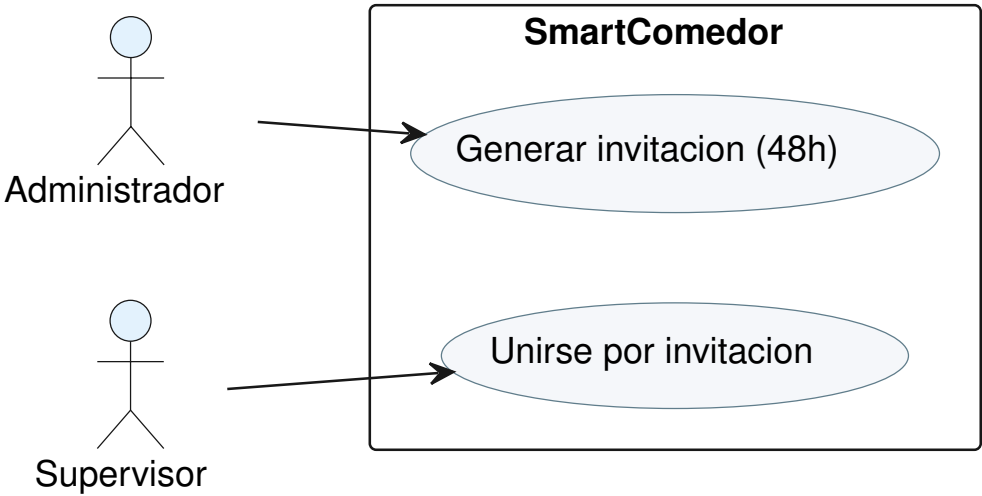
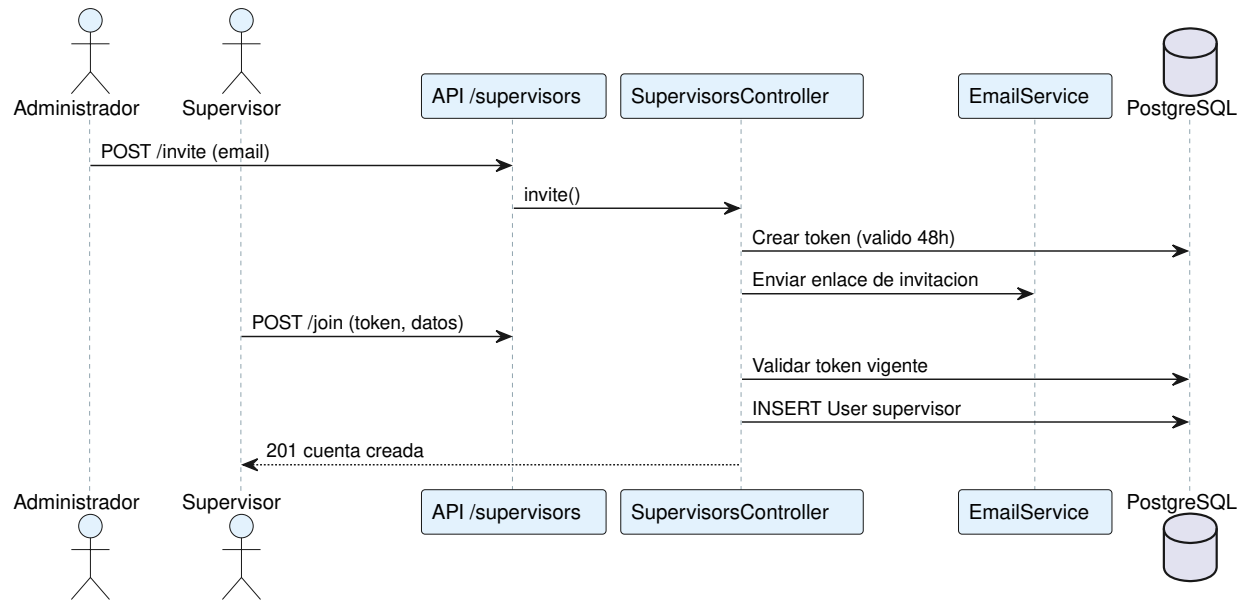
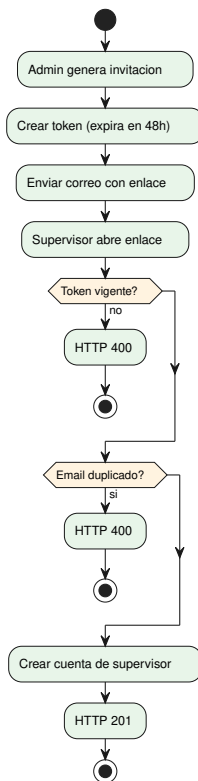


Figura 27

Diagrama de caso de uso — CU-09

**Figura 28***Diagrama de secuencia — CU-09***Figura 29***Diagrama de actividades — CU-09*

CU-10 — Asignar estudiantes a supervisor**Endpoint(s):** POST/DELETE /api/supervisors/:id/assignments**Tabla 34***Ficha del caso de uso CU-10 — Asignar estudiantes a supervisor*

Campo	Descripcion
Objetivo del contexto	Permitir al administrador asociar estudiantes a un supervisor para delimitar su ambito de supervision.
Precondiciones	El supervisor y el estudiante deben existir.
Condicion de exito	Se crea la relacion supervisor-estudiante.
Condicion de fracaso	La asignacion ya existe.
Actores principales	Administrador
Actores secundarios	Base de datos

Tabla 35*Flujo principal de CU-10*

Paso	Accion
1	El administrador selecciona un supervisor y un estudiante.
2	El sistema verifica que la asignacion no exista.
3	El sistema crea el registro de asignacion (N:M).
4	El sistema responde 201.

Tabla 36*Extensiones (flujos alternativos) de CU-10*

Paso	Accion
2.1	Asignacion duplicada → HTTP 400.

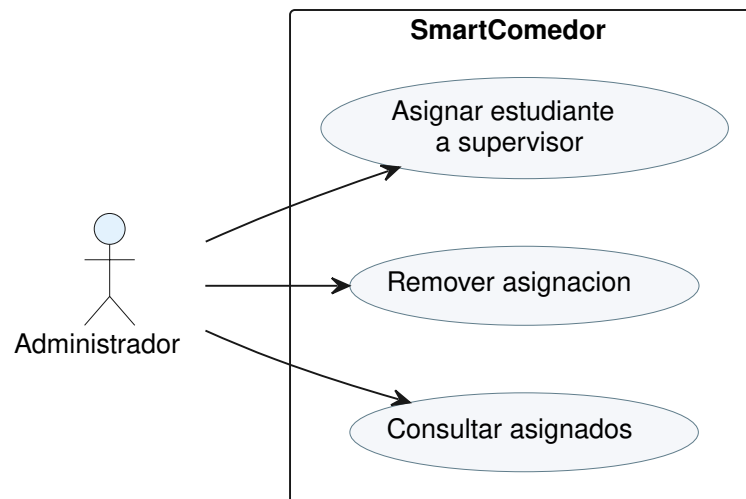
**Figura 30**

Diagrama de caso de uso — CU-10

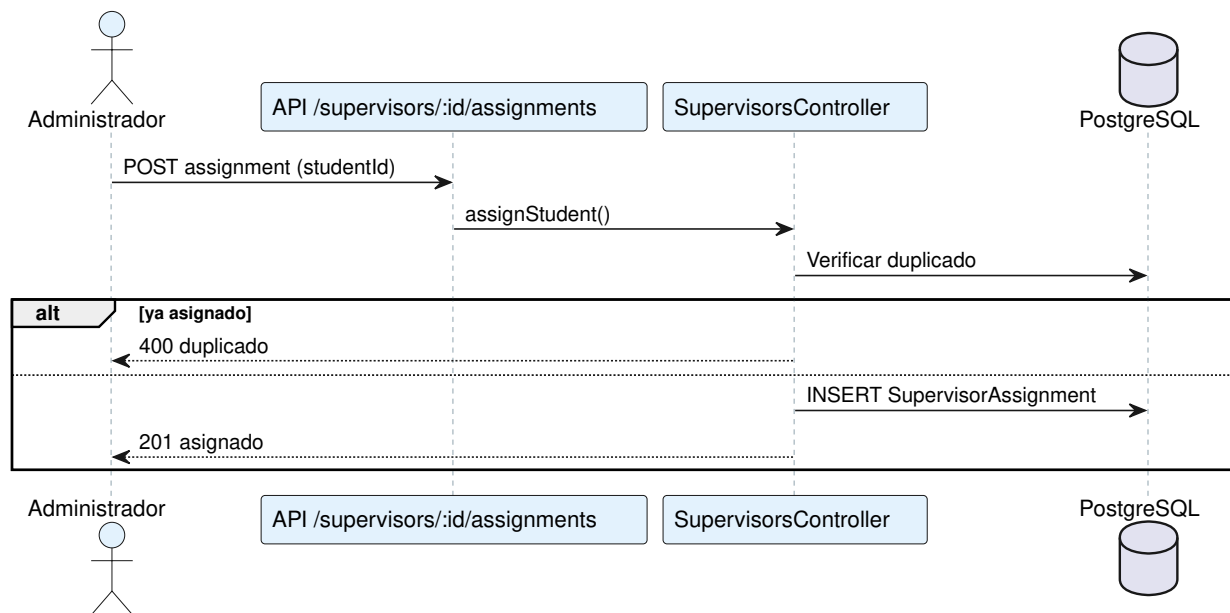
**Figura 31**

Diagrama de secuencia — CU-10

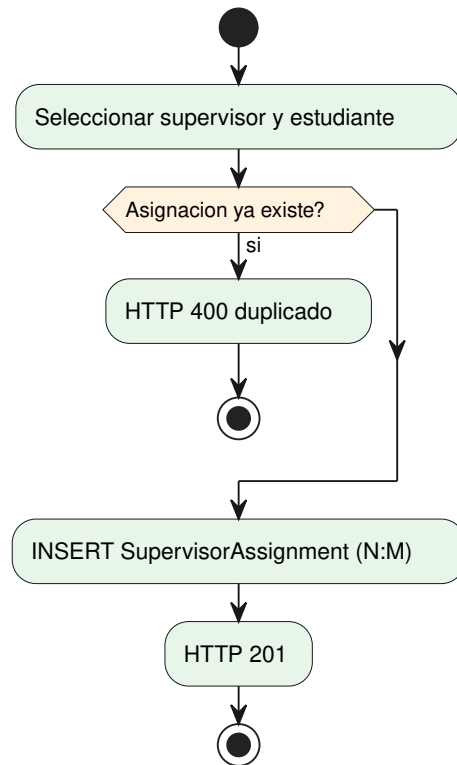
**Figura 32**

Diagrama de actividades — CU-10

CU-11 — Subir pago y comprobantes

Endpoint(s): POST /api/payments, POST /api/payments/upload, POST /api/payments/calculate

Tabla 37*Ficha del caso de uso CU-11 — Subir pago y comprobantes*

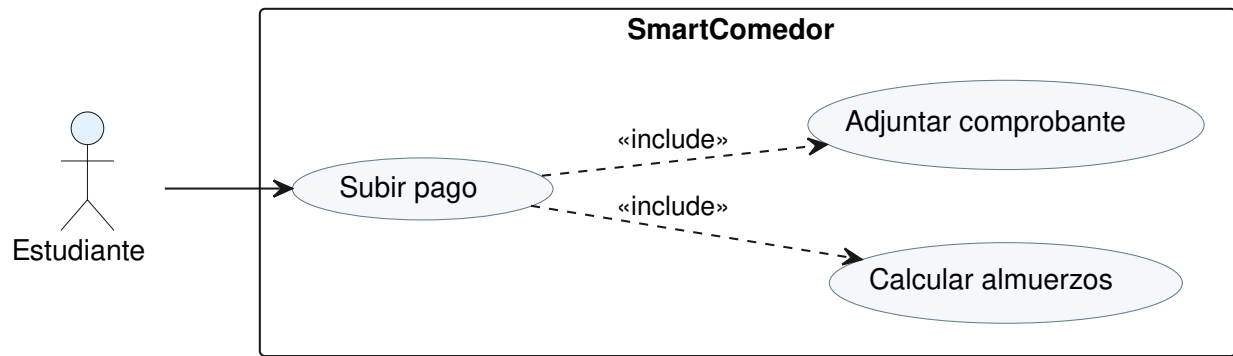
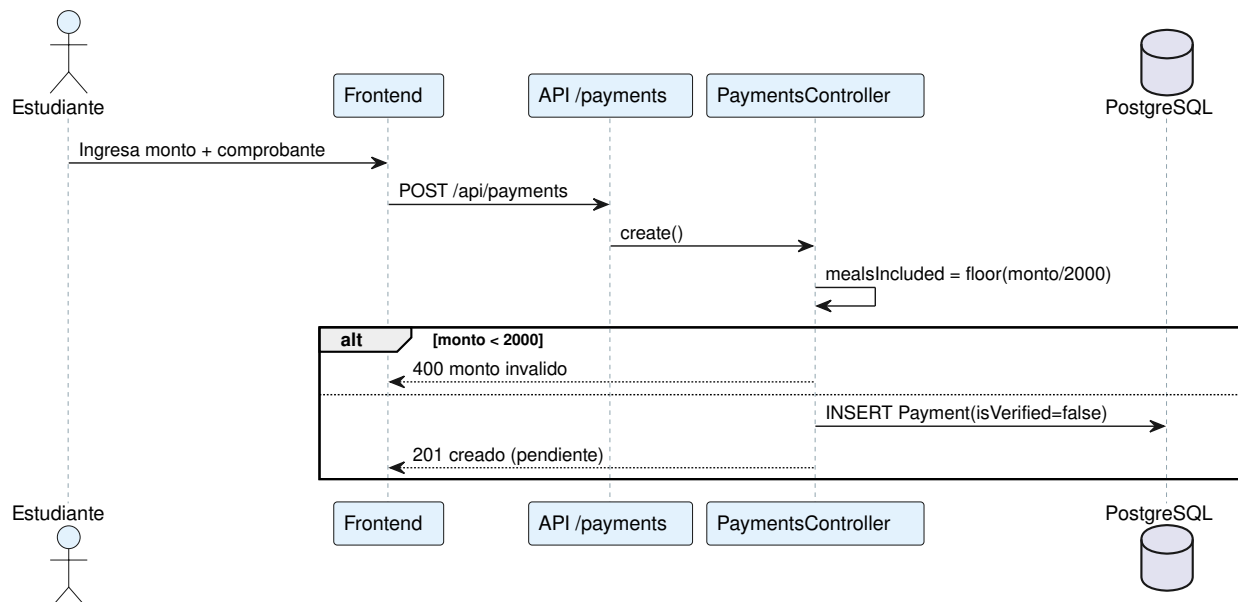
Campo	Descripcion
Objetivo del contexto	Permitir al estudiante registrar un pago/recarga indicando el monto y adjuntando el comprobante, calculando los almuerzos incluidos.
Precondiciones	El estudiante debe estar autenticado.
Condicion de exito	Se crea un pago pendiente de verificacion con los almuerzos calculados.
Condicion de fracaso	El monto es invalido (menor al precio de un almuerzo).
Actores principales	Estudiante
Actores secundarios	Almacenamiento de archivos

Tabla 38*Flujo principal de CU-11*

Paso	Accion
1	El estudiante ingresa el monto y adjunta el comprobante.
2	El sistema calcula almuerzos = piso(monto/2000).
3	El sistema crea el pago con isVerified=false.
4	El sistema responde 201.

Tabla 39*Extensiones (flujos alternativos) de CU-11*

Paso	Accion
2.1	Monto menor a 2000 → HTTP 400.

**Figura 33***Diagrama de caso de uso — CU-11***Figura 34***Diagrama de secuencia — CU-11*

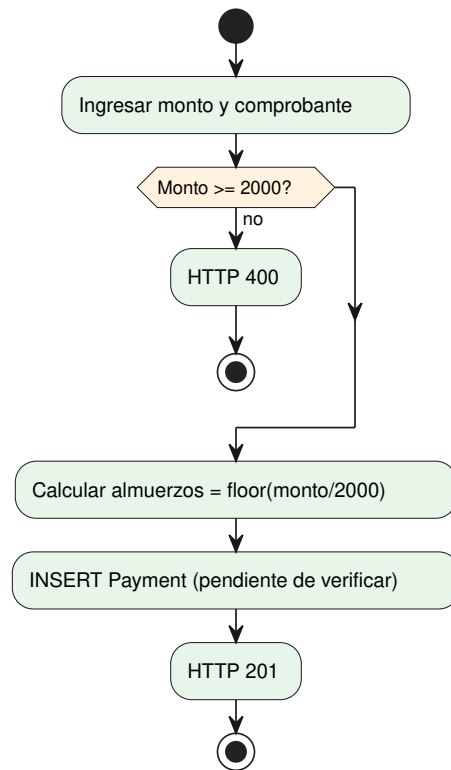
**Figura 35**

Diagrama de actividades — CU-11

CU-12 — Verificar pago

Endpoint(s): PATCH /api/payments/:id/verify

Tabla 40*Ficha del caso de uso CU-12 — Verificar pago*

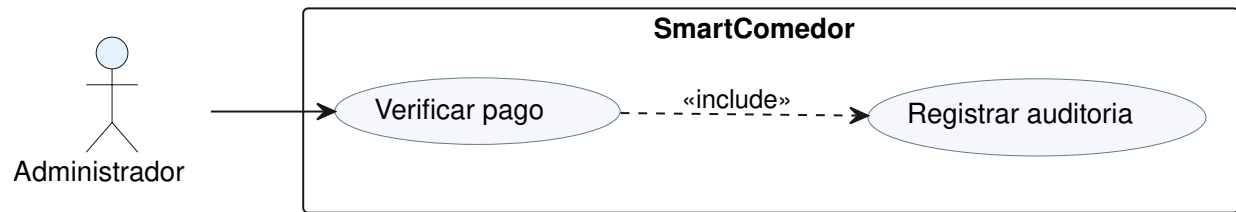
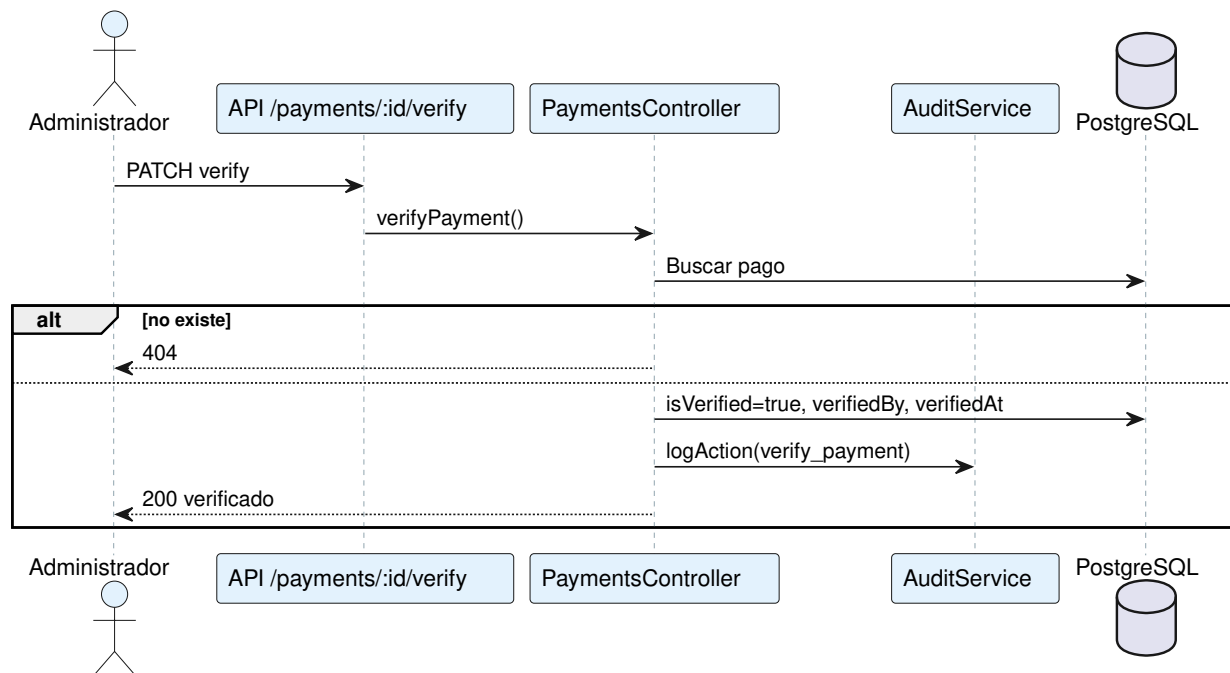
Campo	Descripcion
Objetivo del contexto	Permitir al administrador verificar un pago para habilitar los almuerzos del estudiante, dejando trazabilidad.
Precondiciones	El pago debe existir y estar pendiente.
Condicion de exito	El pago queda verificado y se registra en auditoria.
Condicion de fracaso	El pago no existe.
Actores principales	Administrador
Actores secundarios	Servicio de auditoria

Tabla 41*Flujo principal de CU-12*

Paso	Accion
1	El administrador selecciona el pago a verificar.
2	El sistema verifica que el pago exista.
3	El sistema marca isVerified=true (verifiedBy, verifiedAt).
4	El sistema registra el evento en auditoria.
5	El sistema responde 200.

Tabla 42*Extensiones (flujos alternativos) de CU-12*

Paso	Accion
2.1	Pago inexistente → HTTP 404.

**Figura 36***Diagrama de caso de uso — CU-12***Figura 37***Diagrama de secuencia — CU-12*

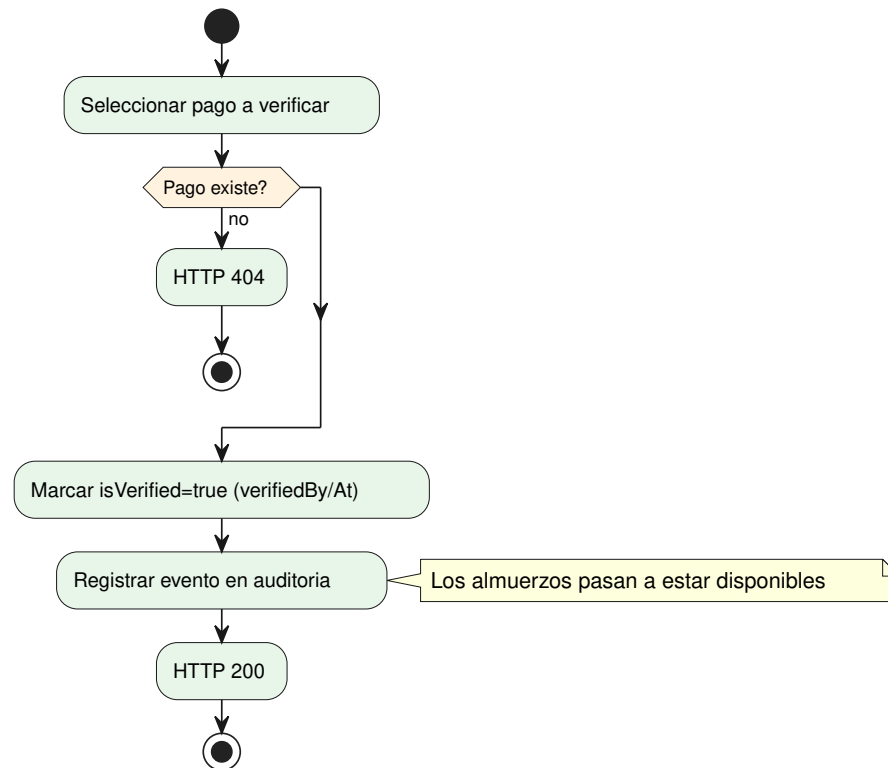
**Figura 38**

Diagrama de actividades — CU-12

CU-13 — Calcular almuerzos por monto

Endpoint(s): POST /api/payments/calculate

Tabla 43*Ficha del caso de uso CU-13 — Calcular almuerzos por monto*

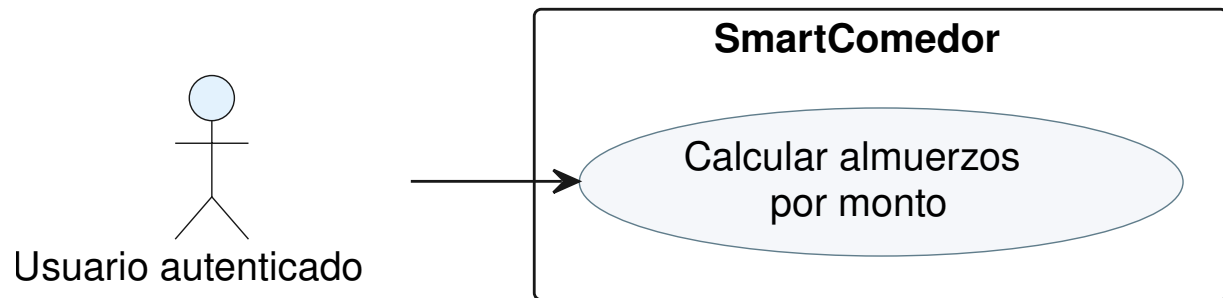
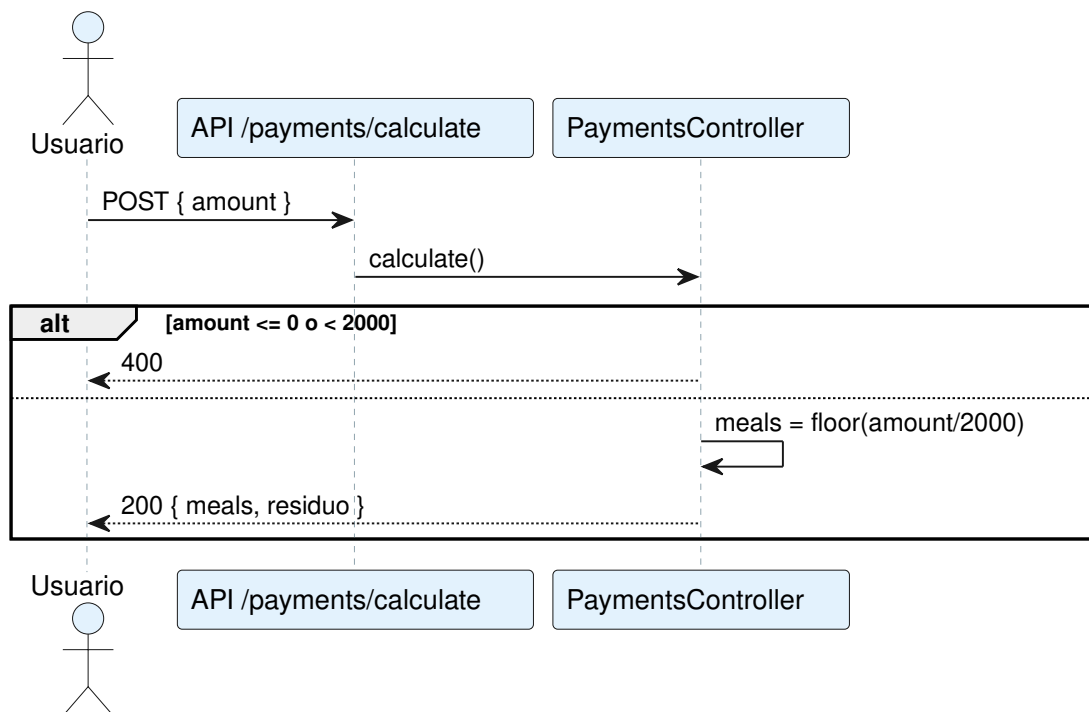
Campo	Descripcion
Objetivo del contexto	Permitir a un usuario autenticado conocer cuantos almuerzos corresponden a un monto dado, antes de registrar el pago.
Precondiciones	El usuario debe estar autenticado.
Condicion de exito	Se devuelve el numero de almuerzos y el residuo.
Condicion de fracaso	El monto es invalido.
Actores principales	Usuario autenticado
Actores secundarios	-

Tabla 44*Flujo principal de CU-13*

Paso	Accion
1	El usuario envia un monto.
2	El sistema valida que el monto sea mayor o igual a 2000.
3	El sistema calcula almuerzos= $\text{piso}(\text{monto}/2000)$ y el residuo.
4	El sistema responde 200 con el resultado.

Tabla 45*Extensiones (flujos alternativos) de CU-13*

Paso	Accion
2.1	Monto menor a 2000 o no numerico $\rightarrow$ HTTP 400.

**Figura 39***Diagrama de caso de uso — CU-13***Figura 40***Diagrama de secuencia — CU-13*

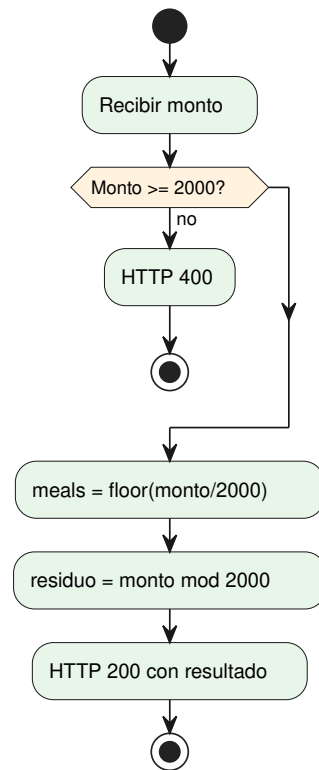
**Figura 41**

Diagrama de actividades — CU-13

CU-14 — Registrar almuerzo

Endpoint(s): POST /api/meals

Tabla 46*Ficha del caso de uso CU-14 — Registrar almuerzo*

Campo	Descripcion
Objetivo del contexto	Permitir al supervisor registrar el consumo de almuerzo de un estudiante validando todas las reglas de elegibilidad del dia.
Precondiciones	El supervisor debe estar autenticado; el estudiante debe estar activo y tener almuerzos.
Condicion de exito	Se registra la asistencia, se descuenta un almuerzo y se deja bitacora.
Condicion de fracaso	El estudiante no cumple alguna regla (inactivo, dia no permitido, ya comio, sin saldo).
Actores principales	Supervisor
Actores secundarios	Servicios de bitacora y auditoria

Tabla 47*Flujo principal de CU-14*

Paso	Accion
1	El supervisor busca al estudiante por QR, cedula o UID.
2	El sistema valida que exista y este activo.
3	El sistema valida el dia autorizado y que no haya comido hoy.
4	El sistema valida que tenga almuerzos disponibles.
5	El sistema registra la asistencia e incrementa mealsUsed.
6	El sistema registra la bitacora y la auditoria, y responde 201.

Tabla 48

Extensiones (flujos alternativos) de CU-14

Paso	Accion
2.1	Estudiante inexistente → 404; inactivo → 400.
3.1	Dia no autorizado o ya comio hoy → 400.
4.1	Sin almuerzos disponibles → 400.

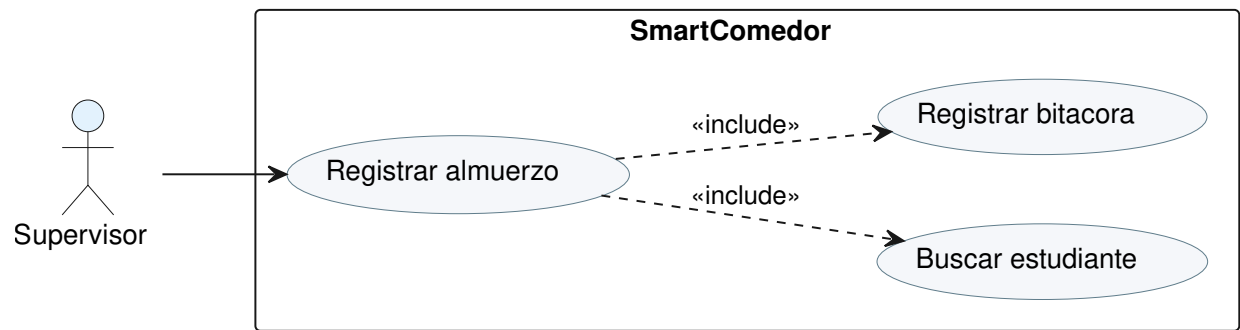
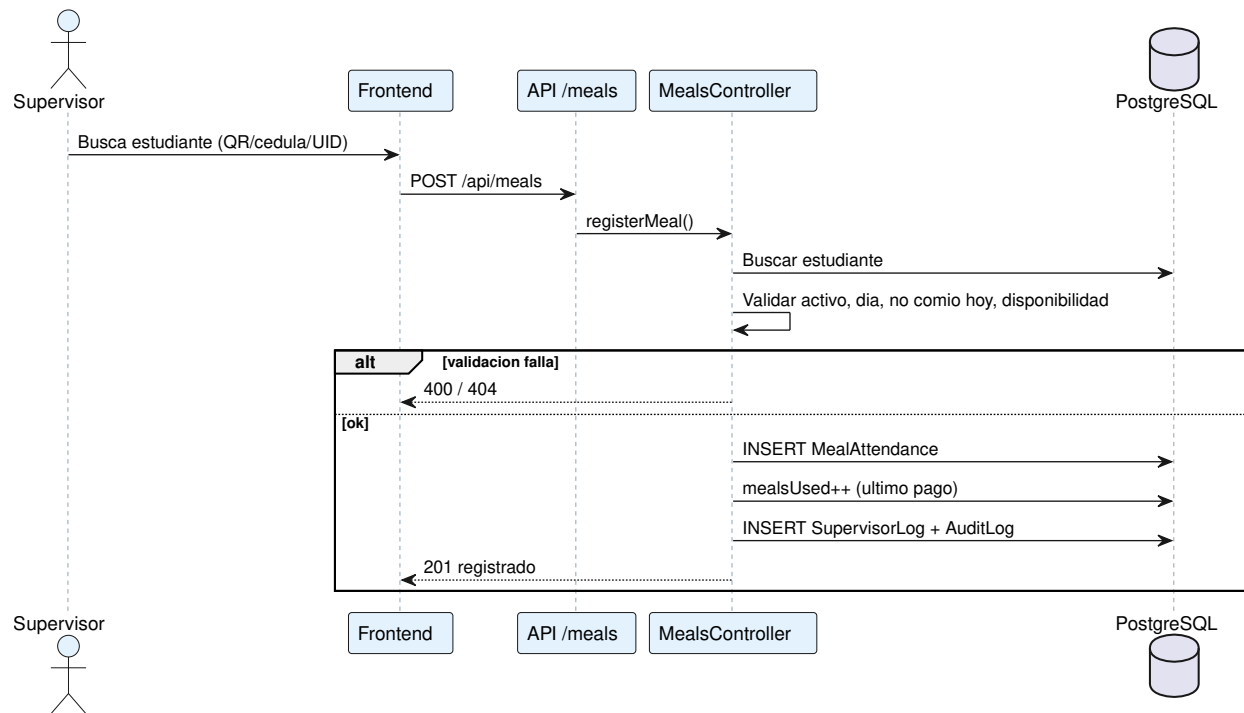


Figura 42

Diagrama de caso de uso — CU-14

**Figura 43***Diagrama de secuencia — CU-14*

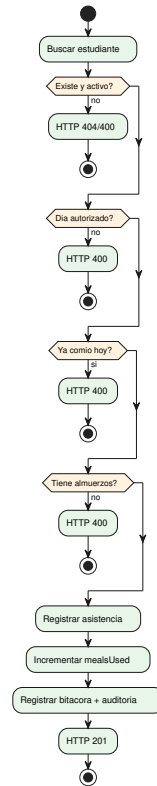
**Figura 44***Diagrama de actividades — CU-14***CU-15 — Consultar historial y asistencia del día****Endpoint(s):** GET /api/meals/history, GET /api/meals/today

Tabla 49*Ficha del caso de uso CU-15 — Consultar historial y asistencia del día*

Campo	Descripcion
Objetivo del contexto	Permitir a los usuarios autorizados consultar el historial de almuerzos y la asistencia del día.
Precondiciones	El usuario debe estar autenticado.
Condicion de exito	Se devuelven los registros segun los filtros.
Condicion de fracaso	Parametros de filtro invalidos.
Actores principales	Usuario autenticado
Actores secundarios	Base de datos

Tabla 50*Flujo principal de CU-15*

Paso	Accion
1	El usuario elige consultar historial o asistencia del día.
2	El usuario aplica filtros (estudiante, rango de fechas).
3	El sistema consulta las asistencias correspondientes.
4	El sistema responde 200 con los resultados.

Tabla 51*Extensiones (flujos alternativos) de CU-15*

Paso	Accion
2.1	Filtros invalidos → HTTP 400.

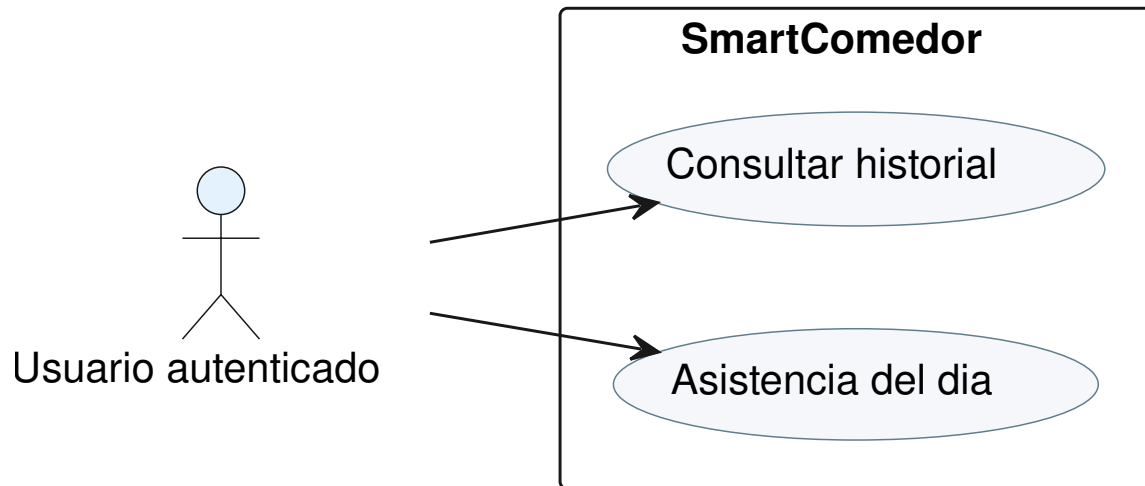
**Figura 45**

Diagrama de caso de uso — CU-15

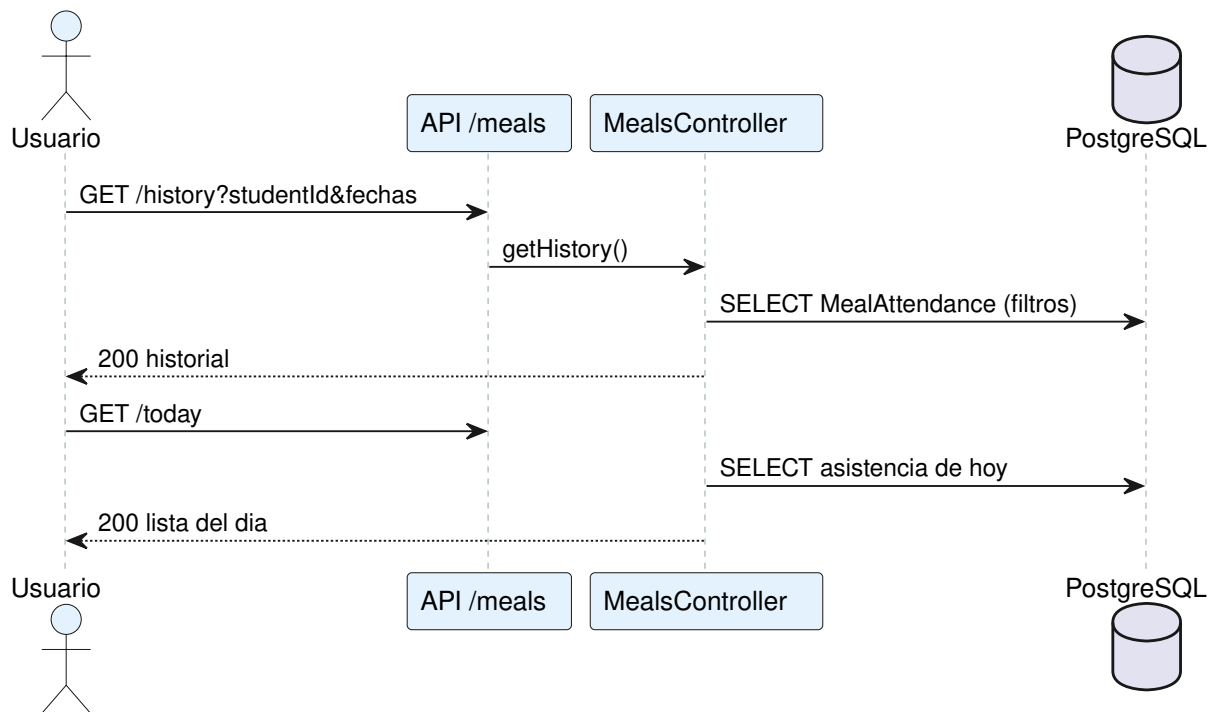
**Figura 46**

Diagrama de secuencia — CU-15

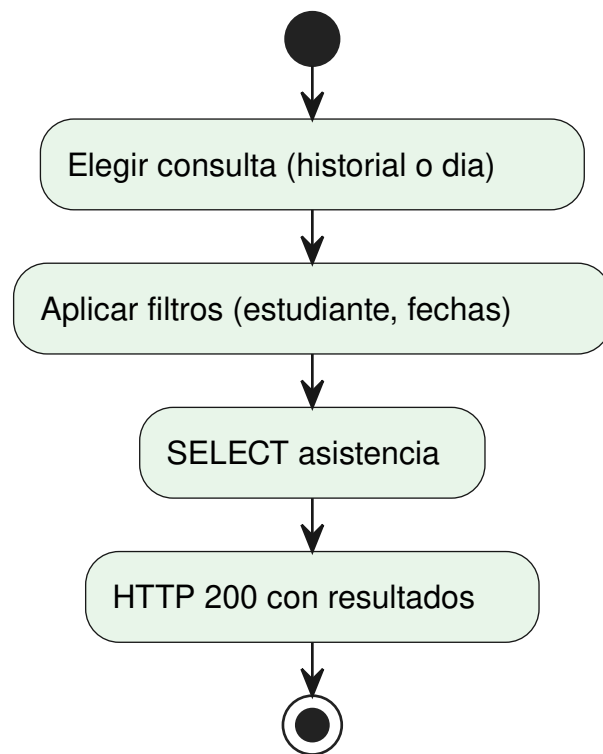
**Figura 47**

Diagrama de actividades — CU-15

CU-16 — Consultar analitica del tablero

Endpoint(s): GET /api/meals/analytics

Tabla 52*Ficha del caso de uso CU-16 — Consultar analitica del tablero*

Campo	Descripcion
Objetivo del contexto	Proveer indicadores agregados (KPIs) del servicio de comedor para la toma de decisiones.
Precondiciones	El usuario debe tener rol admin o auditor.
Condicion de exito	Se devuelven las metricas agregadas.
Condicion de fracaso	Sin autorizacion.
Actores principales	Administrador, Auditor externo
Actores secundarios	Base de datos

Tabla 53*Flujo principal de CU-16*

Paso	Accion
1	El usuario solicita la analitica del tablero.
2	El sistema ejecuta las agregaciones (conteos y totales).
3	El sistema construye la respuesta con los KPIs.
4	El sistema responde 200.

Tabla 54*Extensiones (flujos alternativos) de CU-16*

Paso	Accion
1.1	Rol no autorizado → HTTP 403.

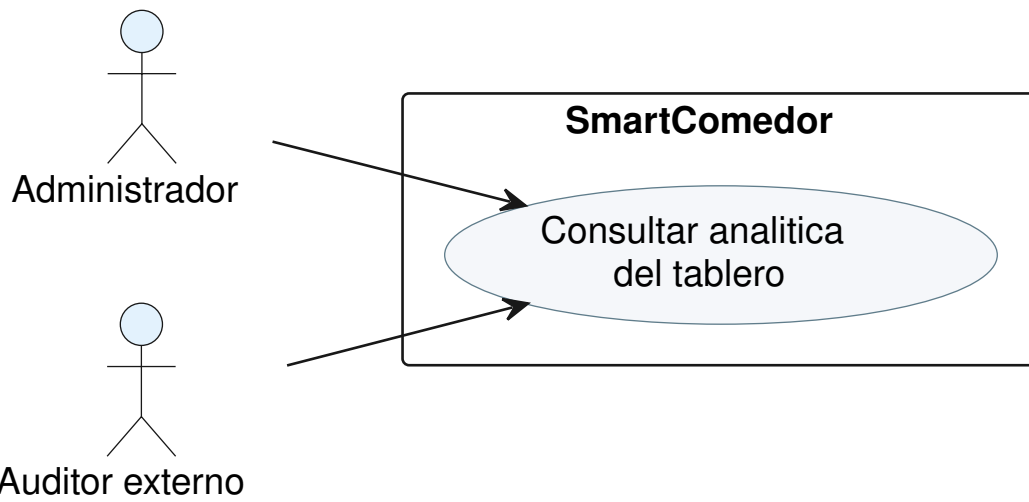
**Figura 48**

Diagrama de caso de uso — CU-16

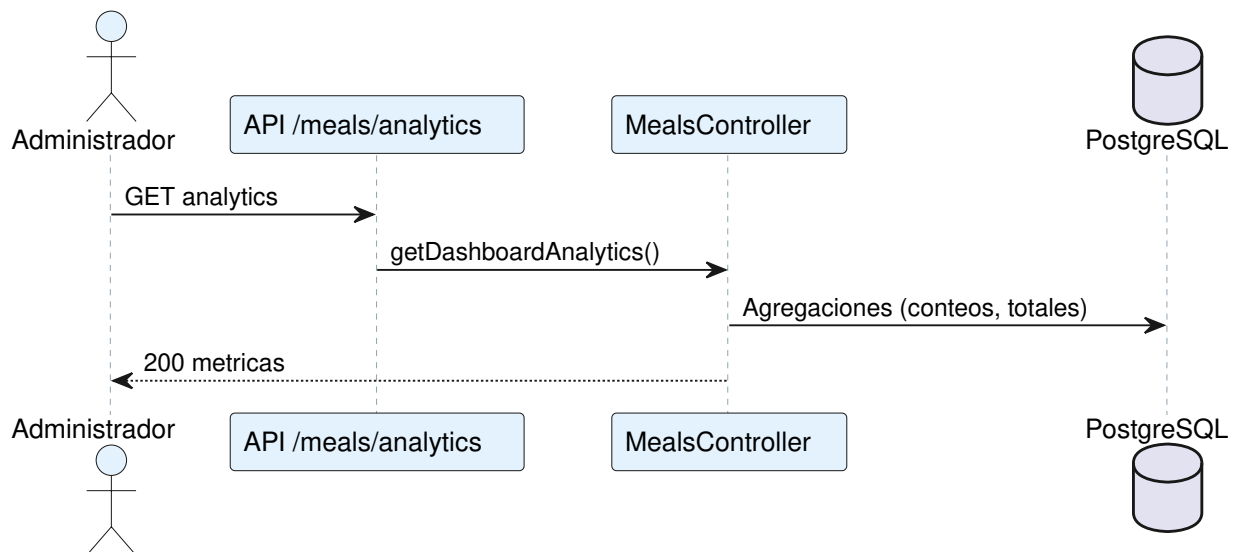
**Figura 49**

Diagrama de secuencia — CU-16

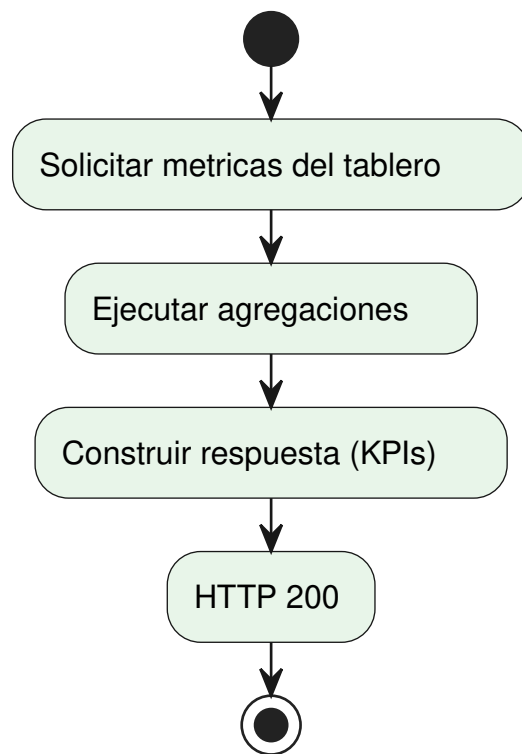
**Figura 50**

Diagrama de actividades — CU-16

CU-17 — Gestionar ciclos y cierre automatico

Endpoint(s): GET/POST/PUT /api/cycles + tarea programada

Tabla 55*Ficha del caso de uso CU-17 — Gestionar ciclos y cierre automatico*

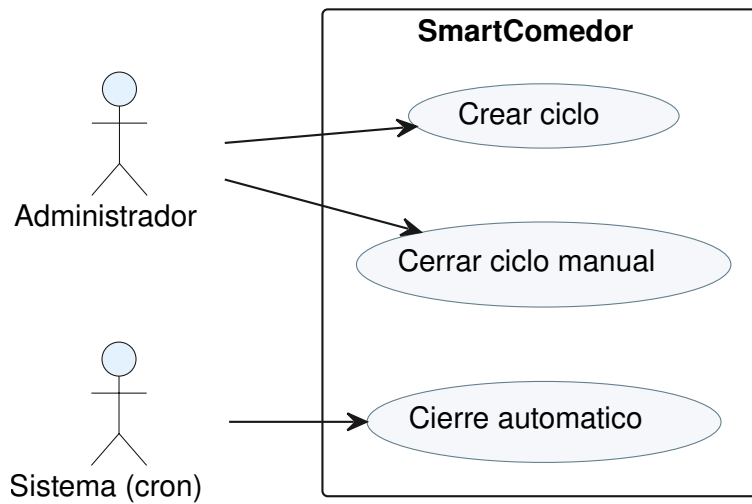
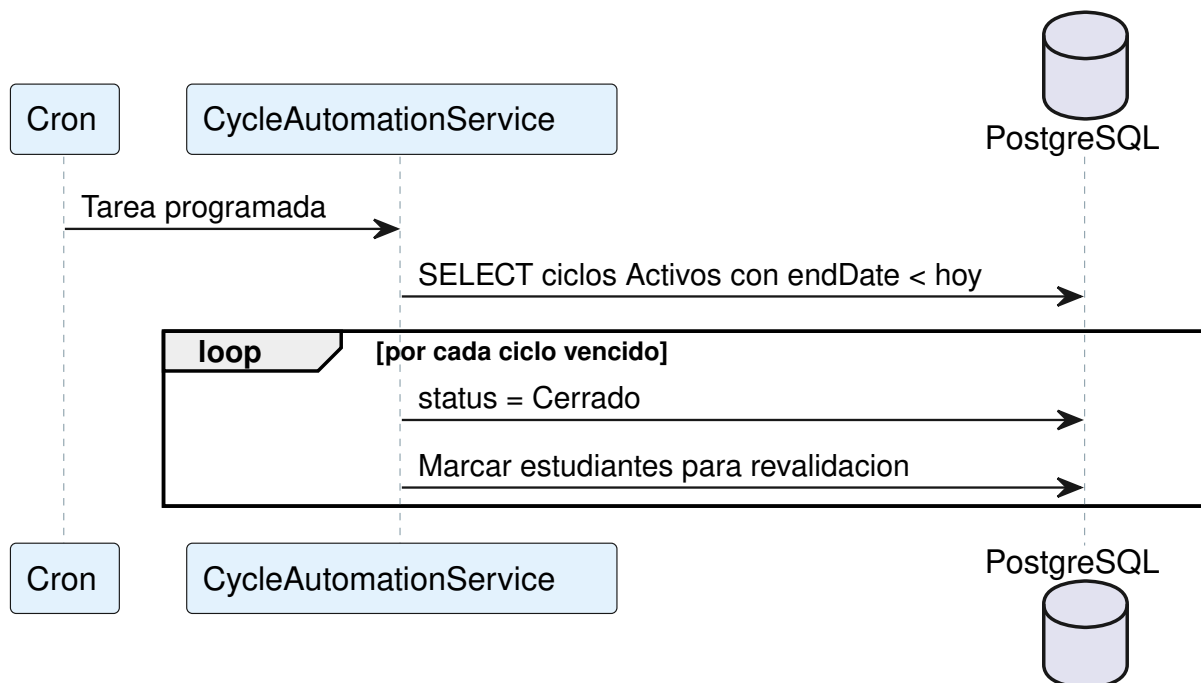
Campo	Descripcion
Objetivo del contexto	Administrar los periodos del servicio y cerrar automaticamente los ciclos vencidos solicitando revalidacion documental.
Precondiciones	Debe existir al menos un ciclo configurado.
Condicion de exito	Los ciclos vencidos se cierran y los estudiantes quedan marcados para revalidacion.
Condicion de fracaso	No hay ciclos vencidos por procesar.
Actores principales	Administrador, Sistema (cron)
Actores secundarios	Servicio de automatizacion de ciclos

Tabla 56*Flujo principal de CU-17*

Paso	Accion
1	El administrador crea o cierra ciclos manualmente.
2	Una tarea programada se ejecuta periodicamente.
3	El sistema busca ciclos activos cuya fecha de fin ya paso.
4	El sistema cierra cada ciclo y marca a los estudiantes para revalidacion.

Tabla 57*Extensiones (flujos alternativos) de CU-17*

Paso	Accion
3.1	Si no hay ciclos vencidos, la tarea finaliza sin cambios.

**Figura 51***Diagrama de caso de uso — CU-17***Figura 52***Diagrama de secuencia — CU-17*

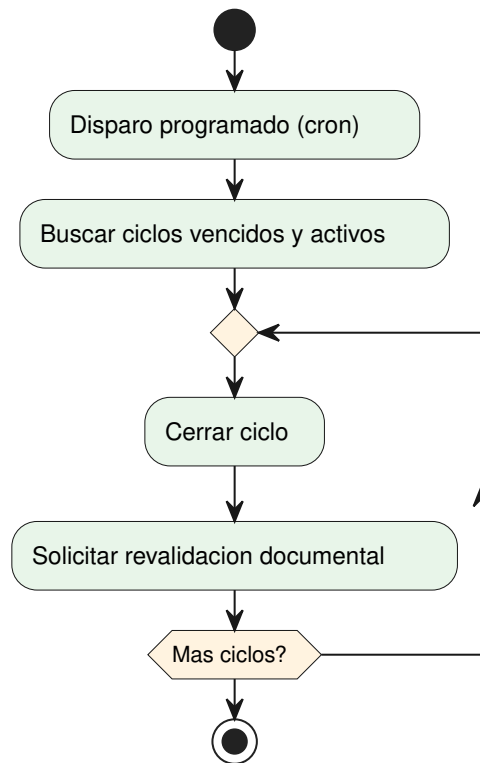
**Figura 53**

Diagrama de actividades — CU-17

CU-18 — Calificar servicio

Endpoint(s): POST /api/ratings

Tabla 58*Ficha del caso de uso CU-18 — Calificar servicio*

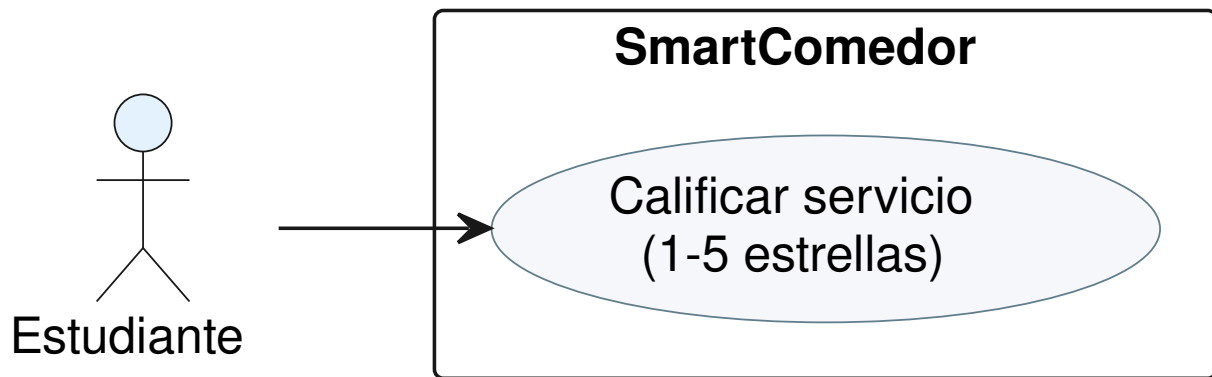
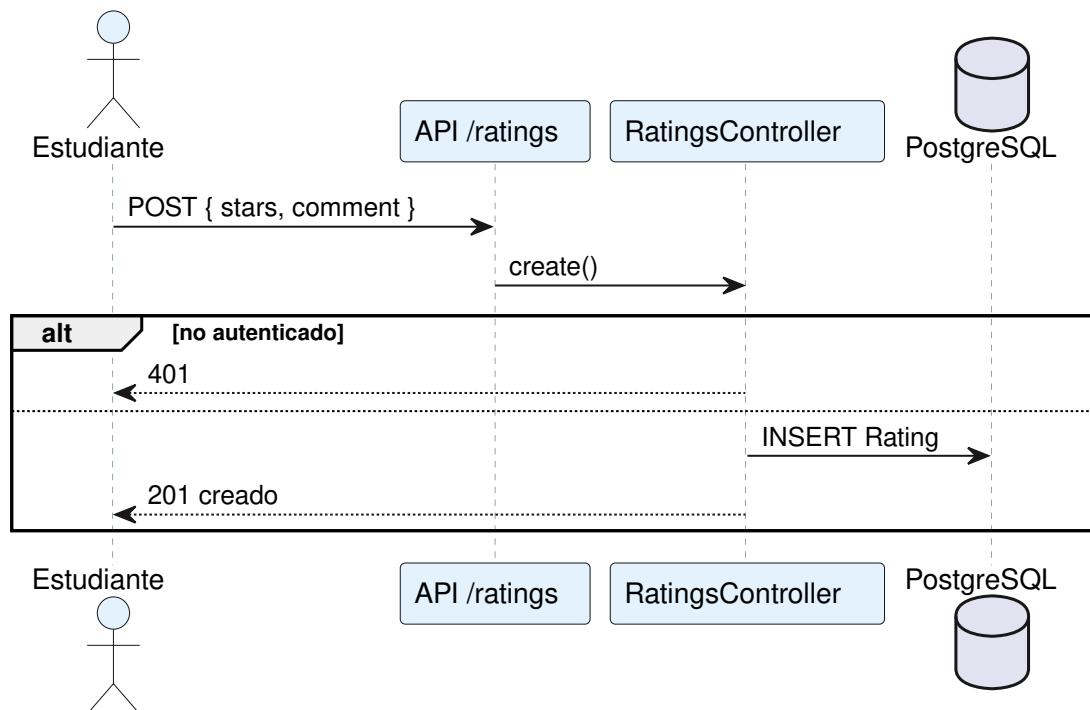
Campo	Descripcion
Objetivo del contexto	Permitir al estudiante calificar el servicio del comedor con una puntuacion y un comentario opcional.
Precondiciones	El estudiante debe estar autenticado.
Condicion de exito	Se registra la calificacion y se actualiza el promedio.
Condicion de fracaso	Usuario no autenticado.
Actores principales	Estudiante
Actores secundarios	Base de datos

Tabla 59*Flujo principal de CU-18*

Paso	Accion
1	El estudiante selecciona una puntuacion de 1 a 5 estrellas.
2	El estudiante agrega un comentario opcional.
3	El sistema registra la calificacion.
4	El sistema responde 201.

Tabla 60*Extensiones (flujos alternativos) de CU-18*

Paso	Accion
3.1	Usuario no autenticado → HTTP 401.

**Figura 54***Diagrama de caso de uso — CU-18***Figura 55***Diagrama de secuencia — CU-18*

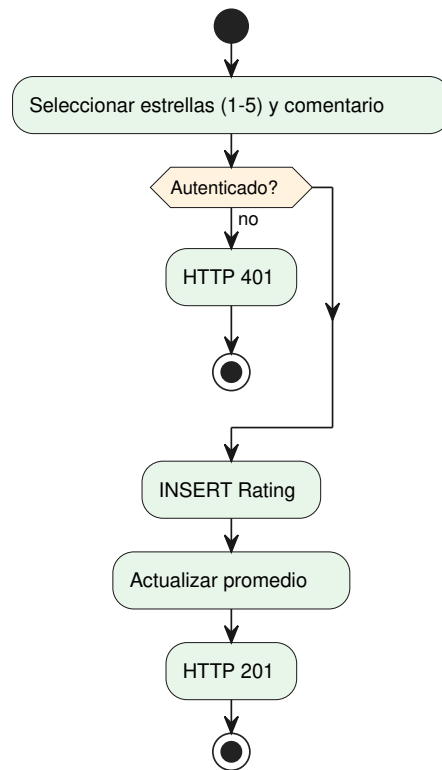
**Figura 56**

Diagrama de actividades — CU-18

CU-19 — Enviar y responder quejas

Endpoint(s): POST /api/complaints, POST /api/complaints/:id/respond

Tabla 61*Ficha del caso de uso CU-19 — Enviar y responder quejas*

Campo	Descripcion
Objetivo del contexto	Permitir al estudiante enviar quejas, sugerencias o comentarios (anonimos o no) y al administrador responderlas.
Precondiciones	El estudiante debe estar autenticado para enviar; el administrador para responder.
Condicion de exito	Se registra la queja y, en su caso, la respuesta del administrador.
Condicion de fracaso	Datos invalidos.
Actores principales	Estudiante, Administrador
Actores secundarios	Base de datos

Tabla 62*Flujo principal de CU-19*

Paso	Accion
1	El estudiante crea una queja/sugerencia/comentario.
2	Si es anonima, no se asocia al estudiante.
3	El sistema registra la queja.
4	El administrador responde y la marca como resuelta.

Tabla 63*Extensiones (flujos alternativos) de CU-19*

Paso	Accion
1.1	Contenido vacio → HTTP 400.

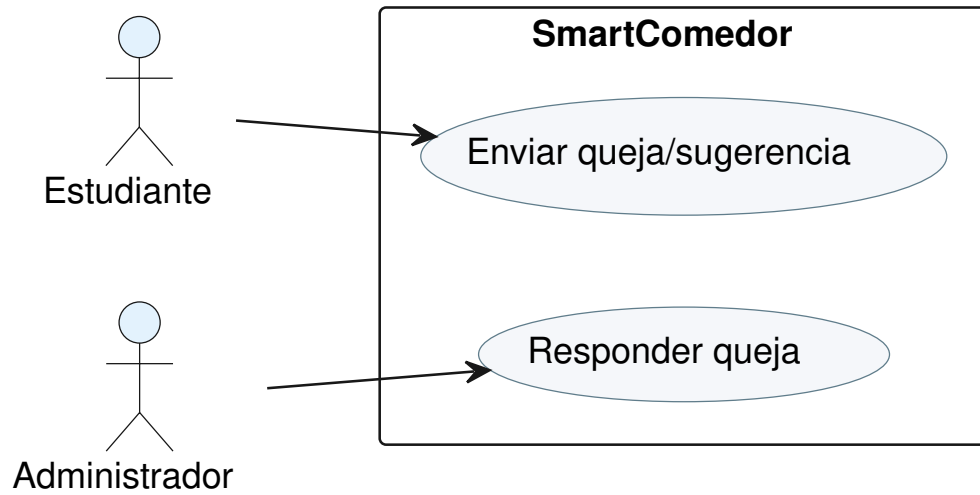


Figura 57

Diagrama de caso de uso — CU-19

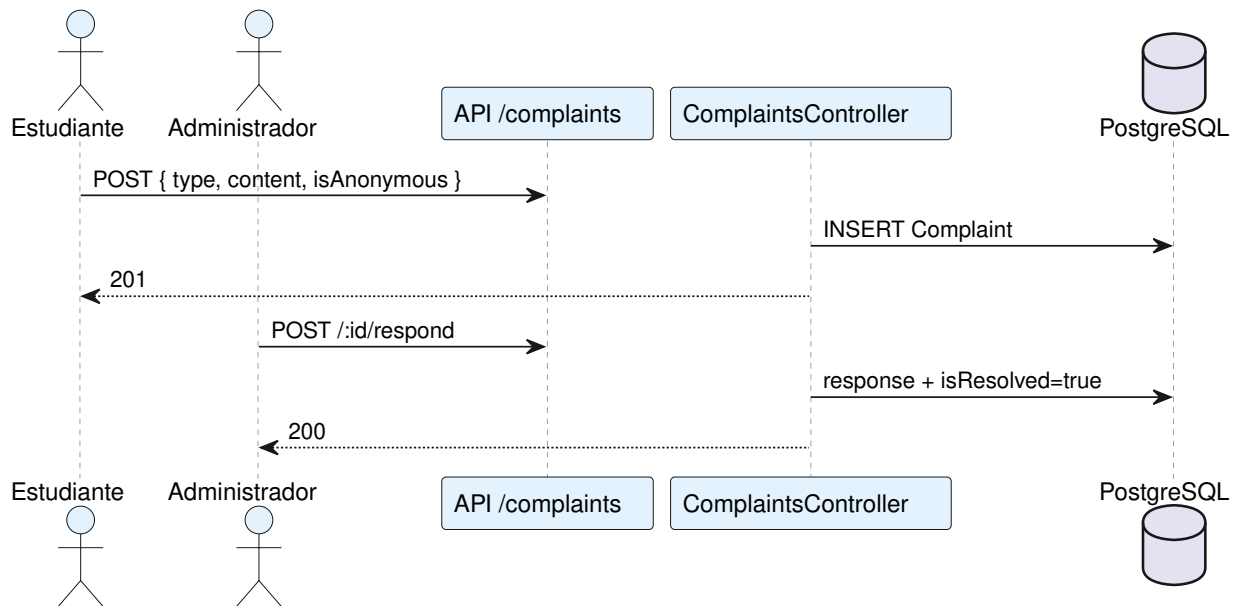


Figura 58

Diagrama de secuencia — CU-19

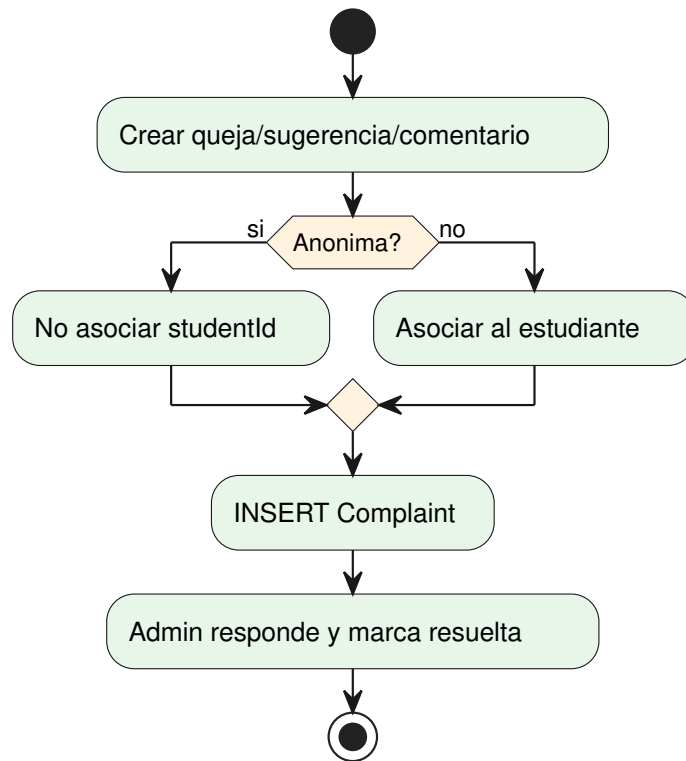
**Figura 59**

Diagrama de actividades — CU-19

CU-20 — Gestionar noticias

Endpoint(s): GET/POST/PUT/DELETE /api/news

Tabla 64*Ficha del caso de uso CU-20 — Gestionar noticias*

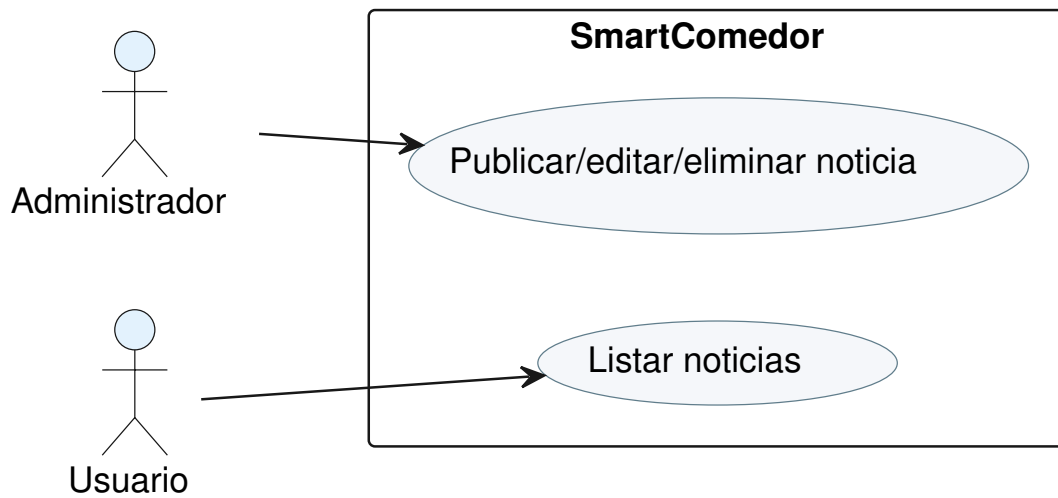
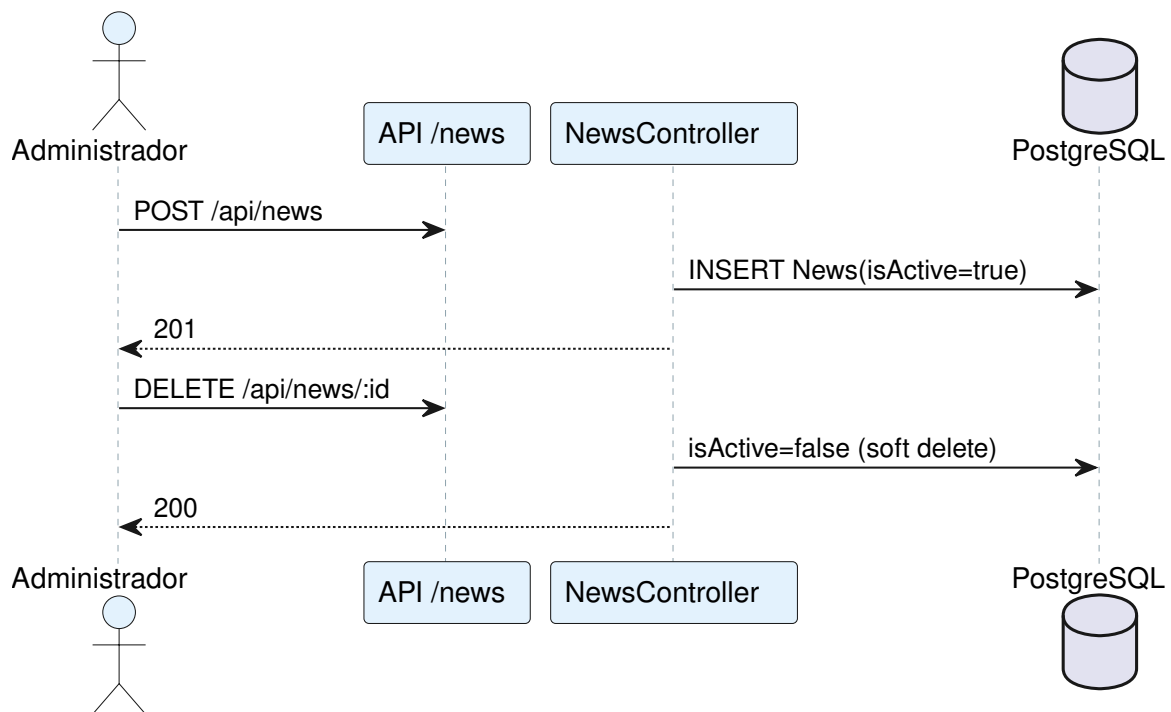
Campo	Descripcion
Objetivo del contexto	Permitir al administrador publicar, editar y eliminar noticias visibles para los usuarios.
Precondiciones	El administrador debe estar autenticado con rol admin.
Condicion de exito	La noticia se crea, actualiza o desactiva.
Condicion de fracaso	Datos invalidos o noticia inexistente.
Actores principales	Administrador, Usuario
Actores secundarios	Base de datos

Tabla 65*Flujo principal de CU-20*

Paso	Accion
1	El administrador selecciona la operacion sobre la noticia.
2	El sistema valida los datos.
3	El sistema crea, actualiza o desactiva (soft delete) la noticia.
4	Los usuarios consultan las noticias activas.

Tabla 66*Extensiones (flujos alternativos) de CU-20*

Paso	Accion
2.1	Datos invalidos → HTTP 400.
3.1	Noticia inexistente → HTTP 404.

**Figura 60***Diagrama de caso de uso — CU-20***Figura 61***Diagrama de secuencia — CU-20*

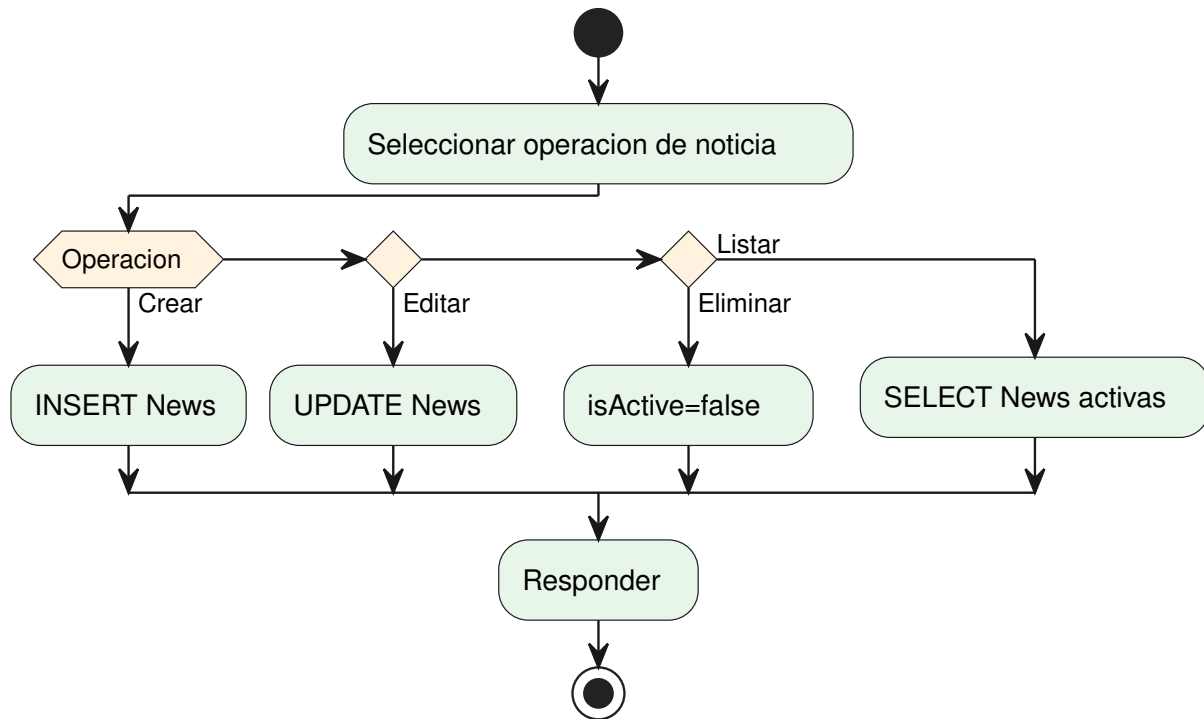
**Figura 62**

Diagrama de actividades — CU-20

CU-21 — Consultar auditoria

Endpoint(s): GET /api/audit

Tabla 67*Ficha del caso de uso CU-21 — Consultar auditoria*

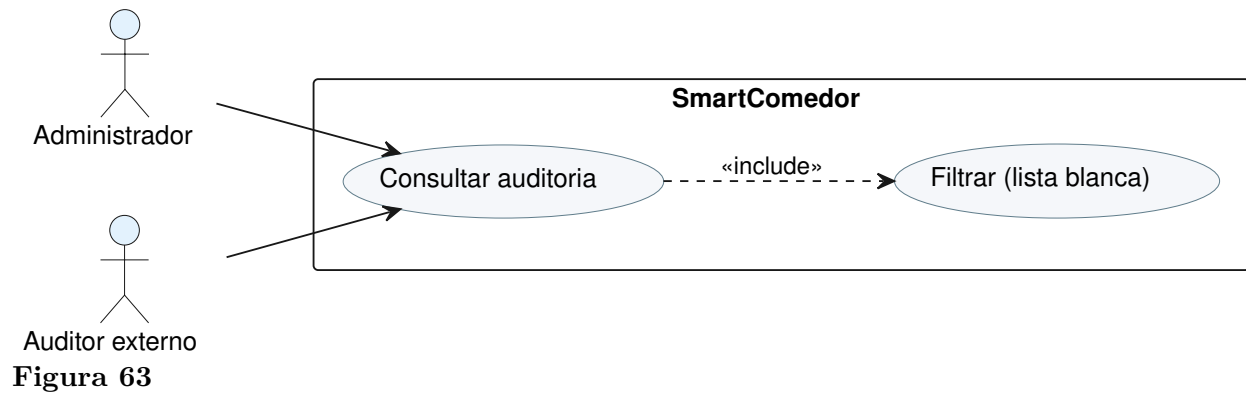
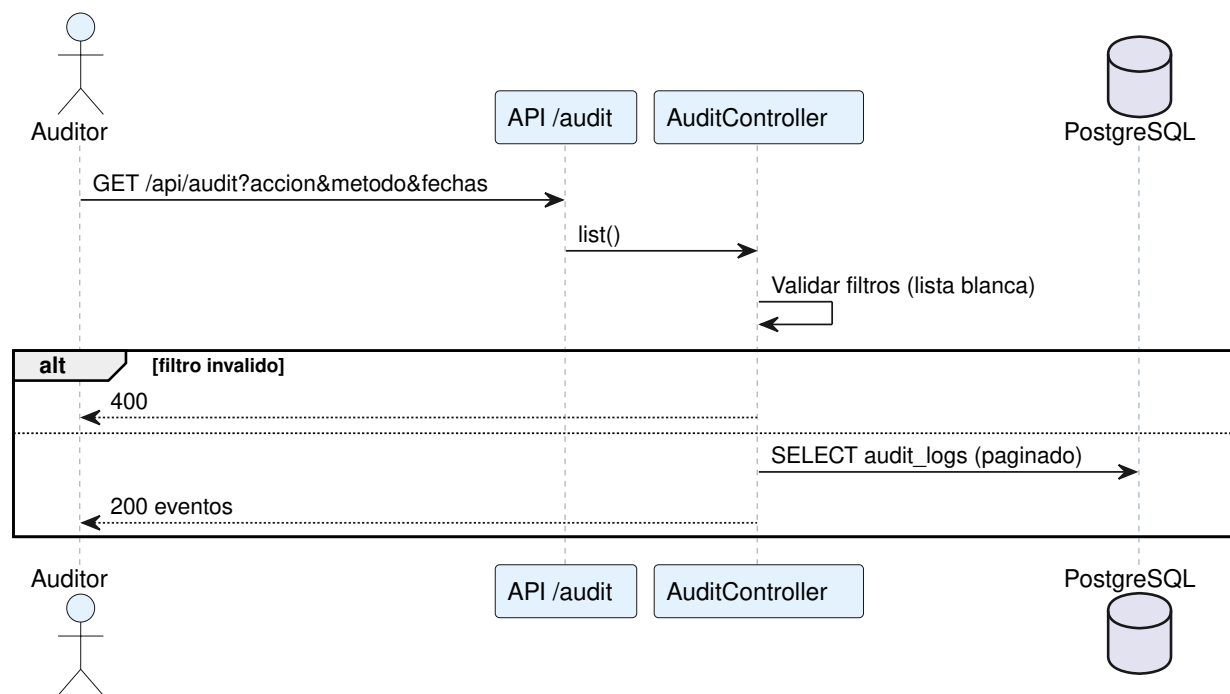
Campo	Descripcion
Objetivo del contexto	Permitir a los perfiles autorizados consultar la bitacora de auditoria del sistema con filtros controlados.
Precondiciones	El usuario debe tener rol admin o auditor.
Condicion de exito	Se devuelven los eventos de auditoria paginados.
Condicion de fracaso	Filtros fuera de la lista blanca.
Actores principales	Administrador, Auditor externo
Actores secundarios	Base de datos

Tabla 68*Flujo principal de CU-21*

Paso	Accion
1	El usuario ingresa filtros (accion, metodo, fechas).
2	El sistema valida los filtros contra una lista blanca.
3	El sistema consulta audit_logs con paginacion.
4	El sistema responde 200 con los eventos.

Tabla 69*Extensiones (flujos alternativos) de CU-21*

Paso	Accion
2.1	Filtro no permitido → HTTP 400.

*Diagrama de caso de uso — CU-21***Figura 64***Diagrama de secuencia — CU-21*

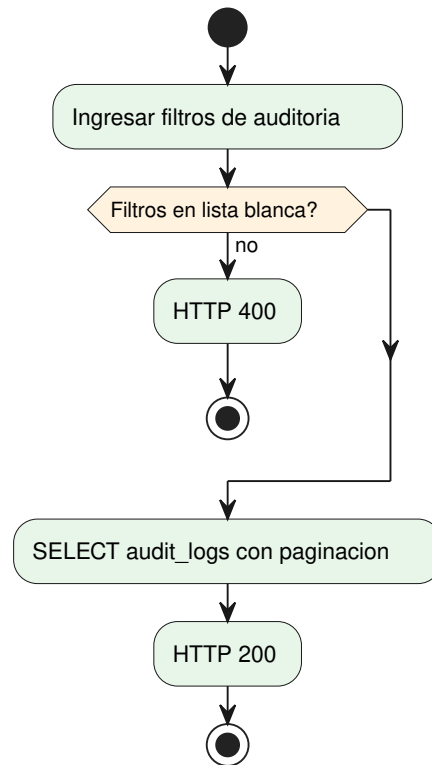
**Figura 65**

Diagrama de actividades — CU-21

Modelo de diseño del sistema

Diagrama de clases detallado

El modelo de dominio se implementa con Sequelize. La Figura 66 muestra el diagrama de clases. Todas las entidades incluyen `id` (UUID, PK), `uid` (identificador legible único con prefijo por tipo, generado en el hook `beforeValidate`), `createdAt` y `updatedAt`.

Diagrama de clases del dominio.

La Tabla 70 resume las entidades principales y sus atributos representativos.

Tabla 70*Entidades principales y atributos representativos*

Entidad (tabla)	Atributos representativos
User (users)	email (único), password (hash), name, lastName, telefono, role, isActive, isAuthorized, diasAsignados, resetPasswordToken/Expires; método <code>validatePassword</code> .
Student (students)	userId (FK), cedula, carrera, semestre, categoriaSisben, archivoSisben, cedulaFrontalPath, horarioPdfPath, direccion, barrio, telefono, trabaja, etnia, desplazado, diasComedor, qrCode/qrCodeSecret, isValidatedSisben, sisbenValidationDetails, currentCycle, cycleRevalidationDueAt.
Payment (payments)	studentId (FK), amount, mealsIncluded, mealsUsed, comprobantePath, universityReceiptPath, bankReceiptPath, isVerified, verifiedBy, verifiedAt.
MealAttendance (meal_attendances)	studentId (FK), supervisorId (FK), date, hora.
SupervisorAssignments (supervisor_assignments)	supervisorId (FK), studentId (FK) — relación N:M.
SupervisorLog (supervisor_logs)	supervisorId (FK), studentId (FK), action, hora.
Rating (ratings)	studentId (FK), stars (1–5), comment.
Complaint (complaints)	studentId (FK, opcional), type (queja/sugerencia/comentario), content, isAnonymous, response, respondedBy, isResolved.
News (news)	title, content, imageUrl, isActive.
Cycle (cycles)	name, startDate, endDate, status (Activo/Cerrado).
AuditLog (audit_logs)	userId, userEmail, action, details, ipAddress.

Diagramas de secuencias

Se documentan las secuencias de los flujos críticos: inicio de sesión (Figura 67), registro de estudiante (Figura 68), pago (Figura 69), registro de almuerzo (Figura 70) y validación SISBÉN (Figura 71).

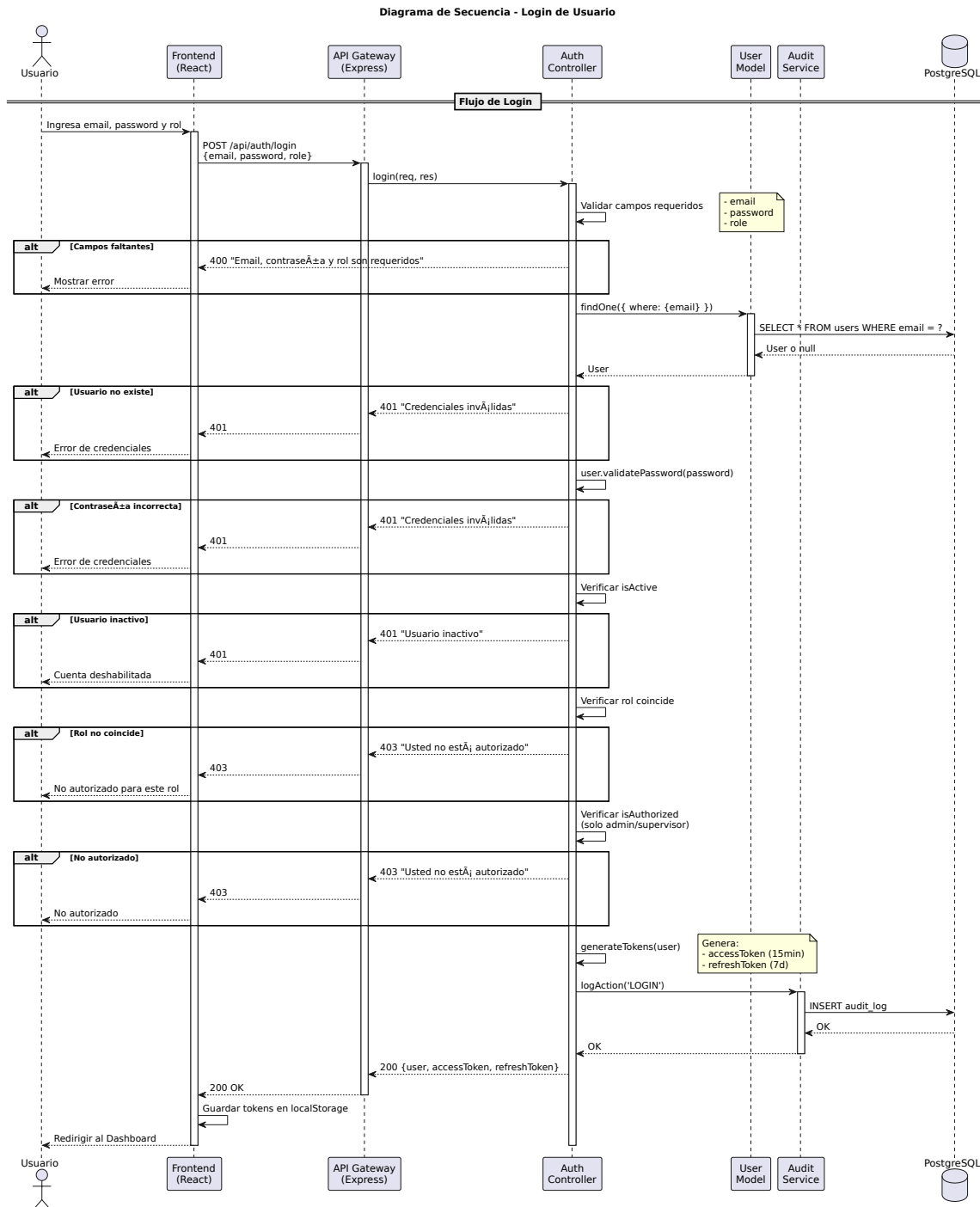
**Figura 67**

Diagrama de secuencia — Inicio de sesión.

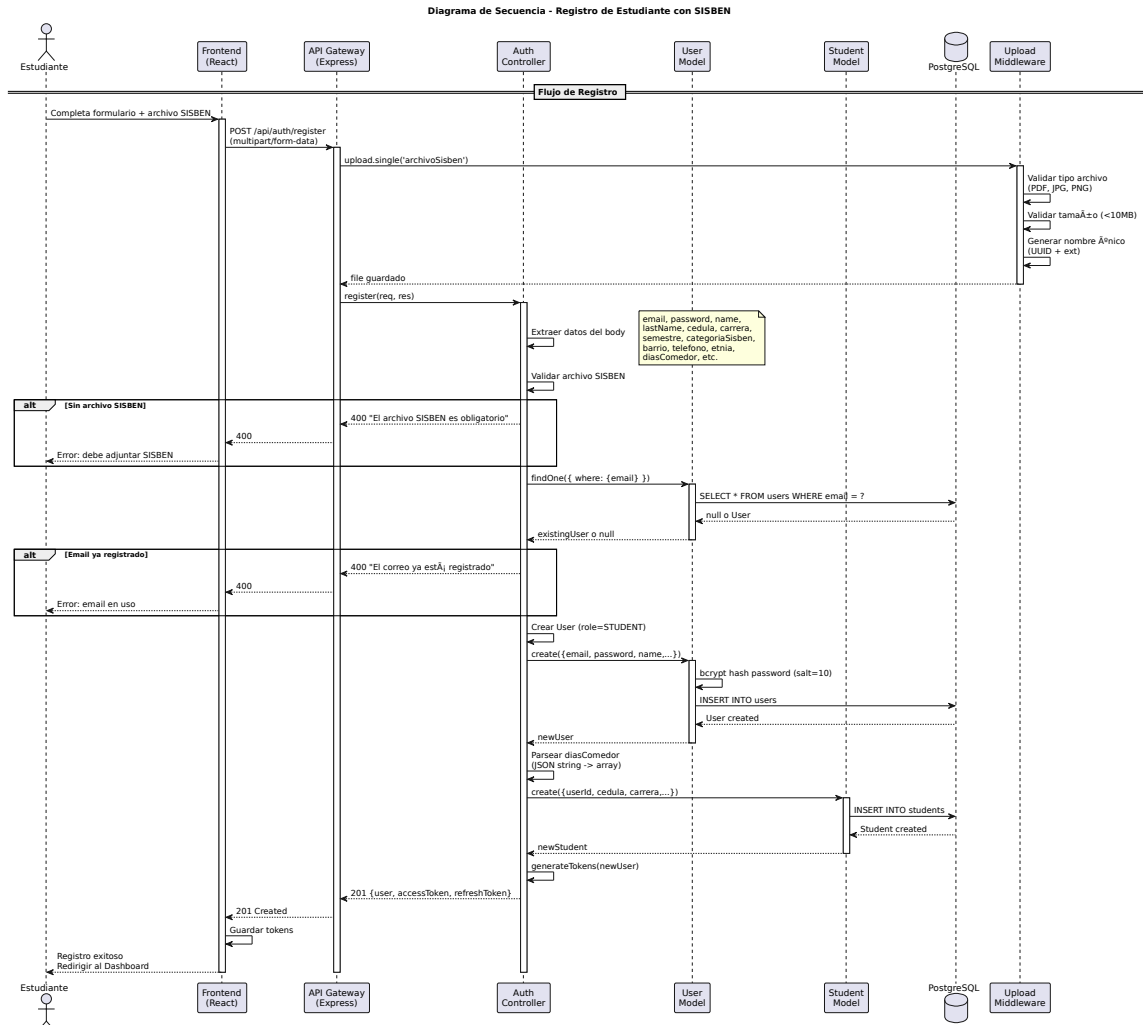


Figura 68

Diagrama de secuencia — Registro de estudiante.

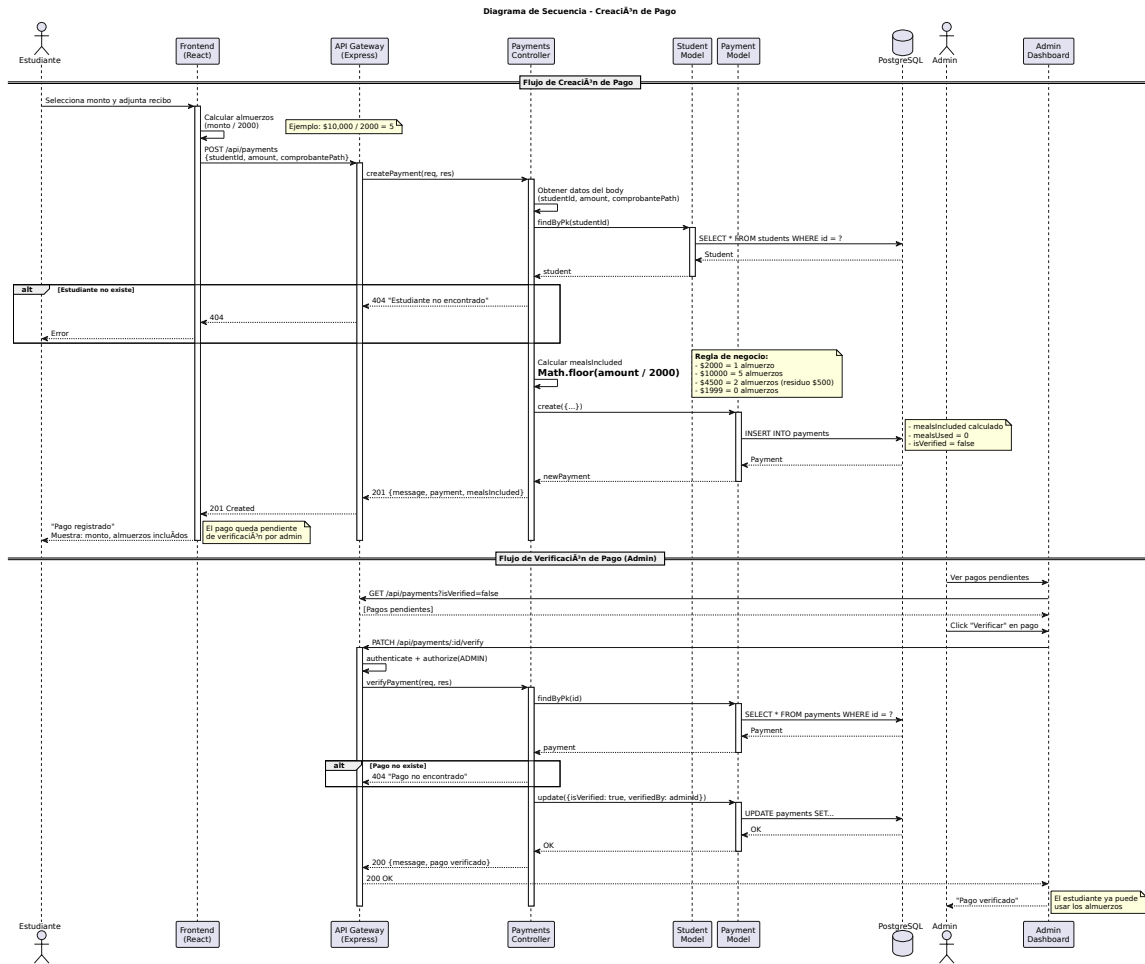


Figura 69

Diagrama de secuencia — Subida y verificación de pago.

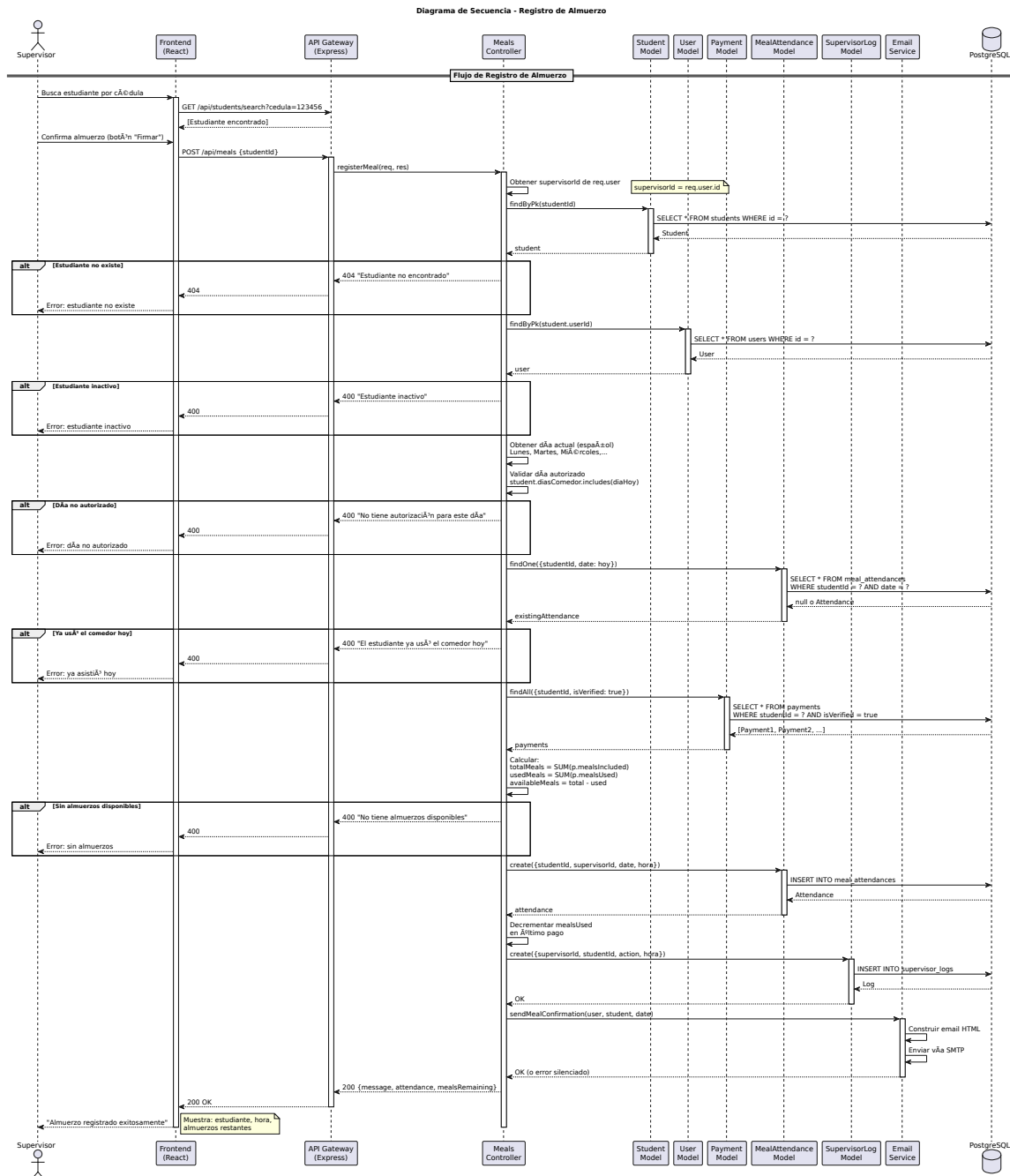


Figura 70

Diagrama de secuencia — Registro de almuerzo.

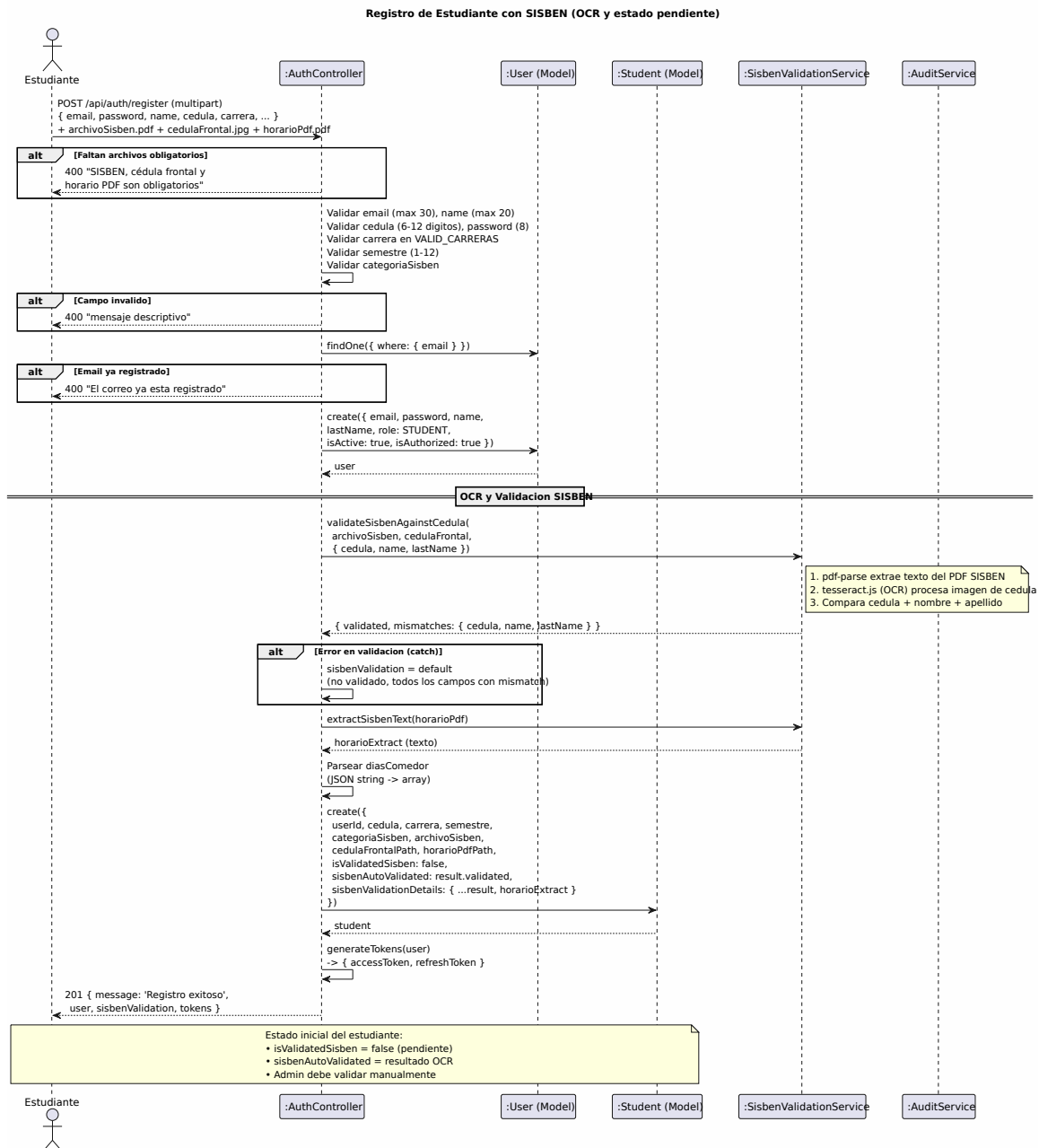
**Figura 71**

Diagrama de secuencia — Validación SISBÉN.

Diagrama entidad-relación

La Figura 72 presenta el modelo entidad-relación. La Tabla 71 resume las cardinalidades.

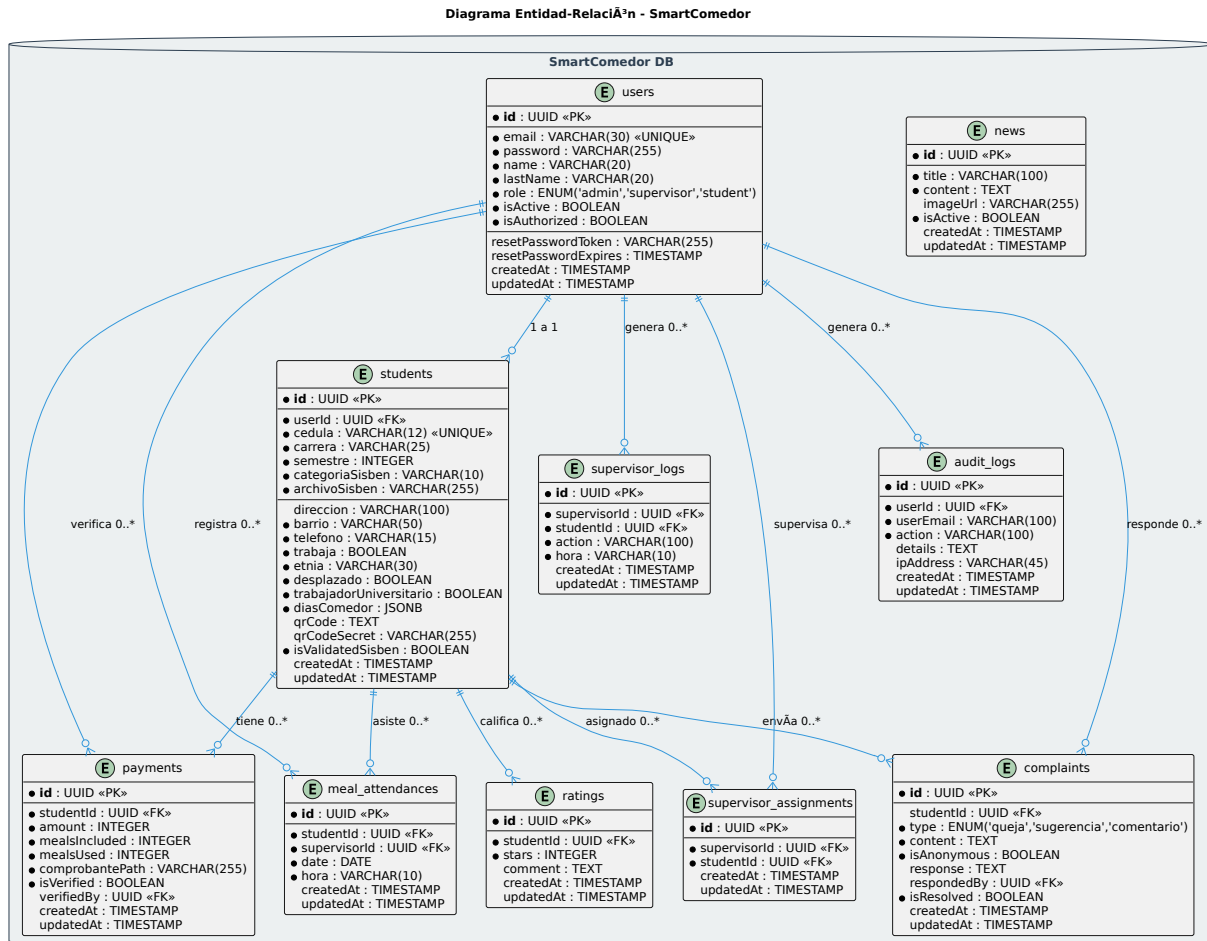
**Figura 72***Diagrama entidad-relación.*

Tabla 71

Cardinalidades del modelo relacional

Relación	Cardinalidad
USERS — STUDENTS	1 : 1
STUDENTS — PAYMENTS	1 : N
STUDENTS — MEAL_ATTENDANCES	1 : N
USERS (supervisor) — MEAL_ATTENDANCES	1 : N
STUDENTS — RATINGS	1 : N
STUDENTS — COMPLAINTS	1 : N (0..1 del lado estudiante, anónimas)
STUDENTS — SUPERVISOR_LOGS	1 : N
USERS (supervisor) — STUDENTS	N : M (vía SUPERVISOR_ASSIGNMENTS)
USERS — AUDIT_LOGS	1 : N

Diagrama de componentes

La Figura 73 muestra la organización en componentes; la Tabla 72 describe sus responsabilidades.

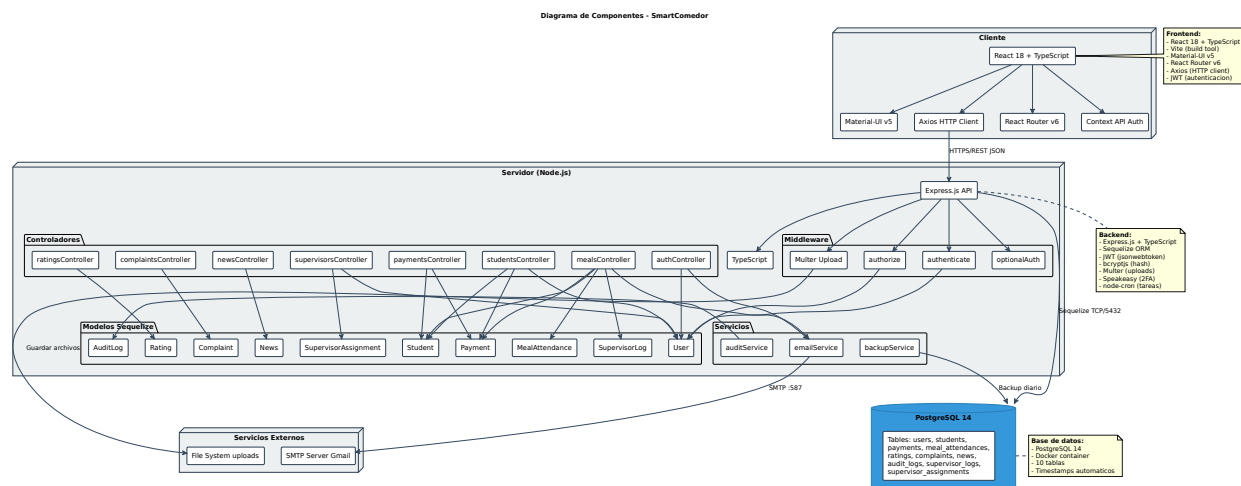


Figura 73

Diagrama de componentes.

Tabla 72*Responsabilidades de los componentes*

Componente	Responsabilidad
Frontend SPA	Interfaz por rol, manejo de sesión (tokens), consumo de la API (axios), rutas protegidas.
API Express	Exposición de endpoints REST bajo <code>/api</code> , validación y orquestación de servicios.
Middleware Auth	<code>authenticate</code> (verifica JWT), <code>authorize</code> (rol), <code>optionalAuth</code> .
Middleware Upload	Recepción de archivos con <code>multer</code> .
Middleware EventAudit	Registro automático de eventos HTTP en la auditoría.
Controllers	Lógica de negocio por entidad.
EmailService	Envío de correos (recuperación de contraseña, invitaciones).
SisbenValidationService	Extracción de texto (PDF/OCR) y cotejo de elegibilidad.
AuditService	Persistencia de la bitácora de auditoría.
BackupService	Respaldo periódico de la base de datos.
CycleAutomationService	Cierre automático de ciclos y revalidación (cron).
Models (Sequelize)	Mapeo objeto-relacional y asociaciones.
PostgreSQL	Persistencia de datos.

Producto del software

Despliegue

El producto se despliega con Docker Compose. El Código 1 muestra los comandos principales.

```
# Variables de entorno
cp .env.example server/.env # completar JWT_SECRET, JWT_REFRESH_SECRET, SMTP_USER, SMTP_PASS

# Desarrollo (hot-reload): Frontend :5174, Backend :3002, PostgreSQL :5433
docker compose --profile dev up --build
```

```
# Produccion (builds + nginx): Frontend :80, Backend :3002
docker compose --profile prod up --build
```

Listing 1: Despliegue del producto

Estructura del producto

```
SmartDinningRoom/
  client/      # Frontend React + Vite (SPA por rol)
  server/      # Backend Express + Sequelize (API REST /api)
  diagrams/    # Diagramas PlantUML + SVG
  docs/        # Documentacion, diagramas e informes
  tests/       # Pruebas de sistema (seguridad, rendimiento, portabilidad)
  docker-compose.yml
  .env.example
```

Listing 2: Estructura del proyecto

Características destacadas

Seguridad por diseño (JWT con refresh, control de acceso por rol, 2FA TOTP, cifrado de contraseñas, validación de filtros por lista blanca y cabeceras de seguridad); antifraude (validación automática del SISBÉN por OCR/PDF e identificación con QR); trazabilidad total (`audit_logs` y `supervisor_logs`); automatización (importación masiva, cálculo de almuerzos, cierre de ciclos y backups programados); y operación 24/7 (healthchecks, perfiles dev/prod y backups con retención de 30 días).

SEGUNDA PARTE. Pruebas del Software

Introducción

Esta parte describe el proceso de verificación y validación (V&V) del sistema SmartComedor (IEEE, 2008). Las pruebas garantizan que el software cumple los requisitos funcionales y no funcionales y que las reglas de negocio críticas (elegibilidad SISBÉN, cálculo de

almuerzos, control de asistencia, control de acceso y auditoría) se comportan de forma correcta, segura y predecible. La estrategia combina varios niveles: pruebas unitarias (caja negra y caja blanca), pruebas de integración (incremental y basada en hilos), pruebas de sistema (seguridad, rendimiento, usabilidad y portabilidad) y pruebas de aceptación (Myers y col., 2011; Pressman & Maxim, 2014). Como resumen, la suite automatizada ejecuta **196 pruebas con 100 % de éxito** (10 suites) y una cobertura global aproximada del **80,4 %** de sentencias medida con Jest (Meta Open Source, 2024).

Planificación de las pruebas

Objetivos de las pruebas

Tabla 73

Objetivos de las pruebas

ID	Objetivo
OBJ-01	Verificar que cada requisito funcional (RF-01 . . . RF-45) está implementado y se comporta según lo especificado.
OBJ-02	Verificar las reglas de negocio críticas (cálculo de almuerzos, monto mínimo, un almuerzo por día, día autorizado, disponibilidad y validación SISBÉN).
OBJ-03	Verificar el control de acceso (JWT, autorización por rol y manejo de cuentas inactivas/no autorizadas).
OBJ-04	Detectar defectos lo antes posible mediante pruebas automatizadas.
OBJ-05	Validar atributos de calidad (seguridad, rendimiento, usabilidad, portabilidad).
OBJ-06	Asegurar una cobertura de código adecuada y trazable.
OBJ-07	Validar, con criterios de aceptación, la satisfacción de las necesidades del usuario.

Alcance de las pruebas

Dentro del alcance: controladores del backend; middleware de seguridad; servicios (validación SISBÉN, auditoría); flujos de integración entre módulos; atributos de calidad (cabeceras de seguridad, control de acceso, carga/estrés, portabilidad cross-browser y usabilidad/accesibilidad básica). **Fuera del alcance (o cubierto indirectamente):** pruebas

exhaustivas de componentes React; pruebas de proveedores externos (SMTP, infraestructura de BrowserStack); y *penetration testing* avanzado más allá del escaneo automatizado (OWASP ZAP) y JMeter (OWASP Foundation, 2021).

Estrategias de pruebas

La Tabla 74 resume el mapeo de niveles, técnicas y herramientas.

Tabla 74

Estrategias de pruebas por nivel

Nivel	Técnica	Herramienta / ubicación
Unitario	Caja negra (equivalencia, valores límite) + caja blanca (camino)	Jest + ts-jest (<code>server/src/tests/unit/</code>)
Integración	Incremental (bottom-up)	Jest (<code>integration/incremental.test.ts</code>)
Integración	Basada en hilos (por actor)	Jest (<code>integration/threads.test.ts</code>)
Sistema — Seguridad	Cabeceras, control de acceso, escaneo	Playwright + OWASP ZAP + JMeter
Sistema — Rendimiento	Carga y estrés	Apache JMeter (<code>tests/security/jmeter/</code>)
Sistema — Portabilidad	Cross-browser	Playwright + BrowserStack
Sistema — Usabilidad	Inspección heurística + accesibilidad	Playwright + revisión MUI
Aceptación	Escenarios de usuario (Gherkin)	Manual / Playwright

Las técnicas de diseño empleadas son: **particiones de equivalencia** (clases válidas e inválidas), **análisis de valores límite** (fronteras de rangos), **cobertura de caminos** (grafo de flujo) y **pruebas basadas en estado** (Beizer, 1990; Myers y col., 2011).

Ambiente de pruebas

La Tabla 75 describe el ambiente. Las pruebas unitarias y de integración emplean una base de datos simulada en memoria (mocks), lo que permite ejecutarlas sin PostgreSQL real, de forma determinista y veloz.

Tabla 75

Ambiente de pruebas

Elemento	Versión / detalle
Node.js	20+ (probado con v22)
Framework de pruebas	Jest 29 + ts-jest 29
Lenguaje	TypeScript 5
BD en unitarias/integración	Mock en memoria (sin PostgreSQL real)
Pruebas de sistema	Docker (JMeter), Playwright 1.44, BrowserStack, OWASP ZAP

```
cd server
npm test           # toda la suite
npm run test:unit  # solo unitarias
npm run test:integration # solo integracion
npm run test:coverage # con reporte de cobertura
```

Listing 3: Scripts de ejecución (backend)

Pruebas unitarias

Análisis de las pruebas

Las pruebas unitarias aíslan la unidad bajo prueba (controlador, middleware o servicio) mediante mocks de los modelos Sequelize, de `jsonwebtoken`, de `bcrypt`, de `nodemailer` y del sistema de archivos. Se priorizaron las unidades de mayor riesgo: `authController` (seguridad de acceso, tokens, recuperación de contraseña, 2FA), `middleware/auth` (control de acceso), `paymentsController` (cálculo monetario), `mealsController` (reglas de asistencia), `studentsController` (CRUD, importación, SISBÉN) y los servicios de validación. La Tabla 76

presenta la distribución de casos por suite.

Tabla 76

Distribución de casos por suite

Suite (archivo)	Casos <code>it()</code>
<code>unit/authController.test.ts</code>	59
<code>unit/studentsController.test.ts</code>	28
<code>unit/paymentsController.test.ts</code>	21
<code>unit/supervisorsController.test.ts</code>	19
<code>unit/ratingsComplaintsNews.test.ts</code>	17
<code>unit/middleware.test.ts</code>	15
<code>unit/mealsController.test.ts</code>	13
<code>unit/sisbenValidationService.test.ts</code>	6
<code>integration/incremental.test.ts</code>	16
<code>integration/threads.test.ts</code>	8
Total	202 declarados (196 ejecutados/aprobados)

Catálogo detallado de casos de prueba

A continuacion se presenta el catalogo completo de los casos de prueba automatizados, extraido directamente del codigo fuente de las suites (`server/src/tests/`). Cada caso corresponde a un bloque `it()` ejecutable; la columna *Contexto* indica el bloque `describe()` que lo agrupa.

El catalogo agrupa **196** casos declarados en las diez suites.

AuthController (`unit/authController.test.ts`)

La suite `unit/authController.test.ts` contiene 59 casos de prueba (Tabla 77).

Tabla 77*Casos de prueba — AuthController*

ID	Contexto	Caso de prueba
TC-AUTH-001	authController >login >Caja Negra - Clases de Equivalencia	TC-LOGIN-001: Login exitoso como administrador
TC-AUTH-002	authController >login >Caja Negra - Clases de Equivalencia	TC-LOGIN-002: Login exitoso como estudiante
TC-AUTH-003	authController >login >Caja Negra - Clases de Equivalencia	TC-LOGIN-003: Login exitoso como supervisor autorizado
TC-AUTH-004	authController >login >Caja Negra - Clases de Equivalencia	TC-LOGIN-004: Error 400 - Campos requeridos faltantes (sin email)
TC-AUTH-005	authController >login >Caja Negra - Clases de Equivalencia	TC-LOGIN-005: Error 400 - Campos requeridos faltantes (sin password)
TC-AUTH-006	authController >login >Caja Negra - Clases de Equivalencia	TC-LOGIN-006: Error 400 - Campos requeridos faltantes (sin role)
TC-AUTH-007	authController >login >Caja Negra - Clases de Equivalencia	TC-LOGIN-007: Error 401 - Usuario no existe
TC-AUTH-008	authController >login >Caja Negra - Clases de Equivalencia	TC-LOGIN-008: Error 401 - Contraseña incorrecta

ID	Contexto	Caso de prueba
TC-AUTH-009	authController >login >Caja Negra - Clases de Equivalencia	TC-LOGIN-009: Error 401 - Usuario inactivo
TC-AUTH-010	authController >login >Caja Negra - Clases de Equivalencia	TC-LOGIN-010: Error 403 - Rol no coincide (student intenta como admin)
TC-AUTH-011	authController >login >Caja Negra - Clases de Equivalencia	TC-LOGIN-011: Error 403 - Admin no autorizado (isAuthorized=false)
TC-AUTH-012	authController >login >Caja Negra - Clases de Equivalencia	TC-LOGIN-012: Error 403 - Supervisor no autorizado (isAuthorized=false)
TC-AUTH-013	authController >login >Caja Negra - Clases de Equivalencia	TC-LOGIN-013: Error 500 - Excepción del servidor
TC-AUTH-014	authController >login >Camino Básico - Cobertura de Caminos (Grafo de Flujo)	Camino 1: Campos faltantes ->400
TC-AUTH-015	authController >login >Camino Básico - Cobertura de Caminos (Grafo de Flujo)	Camino 2: Campos OK ->Usuario no existe ->401
TC-AUTH-016	authController >login >Camino Básico - Cobertura de Caminos (Grafo de Flujo)	Camino 3: Campos OK ->Usuario existe ->Contraseña incorrecta ->401
TC-AUTH-017	authController >login >Camino Básico - Cobertura de Caminos (Grafo de Flujo)	Camino 4: Campos OK ->Usuario ->Pass OK ->Inactivo ->401

ID	Contexto	Caso de prueba
TC-AUTH-018	authController >login >Camino Básico - Cobertura de Caminos (Grafo de Flujo)	Camino 5: Campos OK ->Usuario ->Pass OK ->Activo ->Rol incorrecto ->403
TC-AUTH-019	authController >login >Camino Básico - Cobertura de Caminos (Grafo de Flujo)	Camino 6: Camino completo hasta no autorizado ->403
TC-AUTH-020	authController >login >Camino Básico - Cobertura de Caminos (Grafo de Flujo)	Camino 7: Camino completo exitoso ->200
TC-AUTH-021	authController >register >Valores Límite - Campos con restricciones	VL-EMAIL-001: Email de exactamente 30 caracteres - válido
TC-AUTH-022	authController >register >Valores Límite - Campos con restricciones	VL-CEDULA-001: Cédula de 6 dígitos - válido (mínimo)
TC-AUTH-023	authController >register >Valores Límite - Campos con restricciones	VL-CEDULA-002: Cédula de 12 dígitos - válido (máximo)
TC-AUTH-024	authController >register >Valores Límite - Campos con restricciones	VL-CEDULA-003: Cédula de 5 dígitos - inválido (debajo del mínimo)
TC-AUTH-025	authController >register >Valores Límite - Campos con restricciones	VL-NOMBRE-001: Nombre de 20 caracteres - válido
TC-AUTH-026	authController >register >Valores Límite - Campos con restricciones	VL-NOMBRE-002: Nombre de 21 caracteres - inválido

ID	Contexto	Caso de prueba
TC-AUTH-027	authController >register >Caja Negra - Clases de Equivalencia	TC-REG-001: Registro exitoso con todos los campos válidos
TC-AUTH-028	authController >register >Caja Negra - Clases de Equivalencia	TC-REG-002: Error 400 - Archivo SIS-BEN faltante
TC-AUTH-029	authController >register >Caja Negra - Clases de Equivalencia	TC-REG-003: Error 400 - Email ya registrado
TC-AUTH-030	authController >register >Caja Negra - Clases de Equivalencia	TC-REG-004: Parseo correcto de dias-Comedor desde string JSON
TC-AUTH-031	authController >refreshToken	TC-REFRESH-001: Refresh exitoso con token válido
TC-AUTH-032	authController >refreshToken	TC-REFRESH-002: Error 400 - Token faltante
TC-AUTH-033	authController >refreshToken	TC-REFRESH-003: Error 401 - Token JWT inválido
TC-AUTH-034	authController >refreshToken	TC-REFRESH-004: Error 401 - Usuario no encontrado
TC-AUTH-035	authController >refreshToken	TC-REFRESH-005: Error 401 - Usuario inactivo
TC-AUTH-036	authController >getProfile	TC-PROFILE-001: Obtener perfil exitoso
TC-AUTH-037	authController >getProfile	TC-PROFILE-002: Error 404 - Usuario no encontrado

ID	Contexto	Caso de prueba
TC-AUTH-038	authController >updateProfile	TC-UPDATE-001: Actualización de perfil exitosa
TC-AUTH-039	authController >updateProfile	TC-UPDATE-002: Error 404 - Usuario no encontrado
TC-AUTH-040	authController >forgotPassword	TC-FORGOT-001: Email enviado cuando usuario existe
TC-AUTH-041	authController >forgotPassword	TC-FORGOT-002: Error 400 - Email faltante
TC-AUTH-042	authController >forgotPassword	TC-FORGOT-003: Mensaje genérico cuando usuario NO existe (seguridad)
TC-AUTH-043	authController >resetPassword	TC-RESET-001: Contraseña restablecida exitosamente
TC-AUTH-044	authController >resetPassword	TC-RESET-002: Error 400 - Token faltante
TC-AUTH-045	authController >resetPassword	TC-RESET-003: Error 400 - Contraseña nueva faltante
TC-AUTH-046	authController >resetPassword	TC-RESET-004: Error 400 - Contraseña NO tiene exactamente 8 caracteres (7)
TC-AUTH-047	authController >resetPassword	TC-RESET-005: Error 400 - Contraseña NO tiene exactamente 8 caracteres (9)
TC-AUTH-048	authController >resetPassword	TC-RESET-006: Error 400 - Token inválido o expirado
TC-AUTH-049	authController >changePassword	TC-CHANGE-001: Contraseña cambiada exitosamente

ID	Contexto	Caso de prueba
TC-AUTH-050	authController >changePass-word	TC-CHANGE-002: Error 404 - Usuario no encontrado
TC-AUTH-051	authController >changePass-word	TC-CHANGE-003: Error 400 - Contraseña actual incorrecta
TC-AUTH-052	authController >setupTwo-Factor	TC-2FA-SETUP-001: Configuración 2FA exitosa para estudiante
TC-AUTH-053	authController >setupTwo-Factor	TC-2FA-SETUP-002: Error 404 - Usuario no encontrado
TC-AUTH-054	authController >setupTwo-Factor	TC-2FA-SETUP-003: Error 400 - Solo estudiantes (admin no puede)
TC-AUTH-055	authController >verifyTwo-Factor	TC-2FA-VERIFY-001: Verificación 2FA exitosa
TC-AUTH-056	authController >verifyTwo-Factor	TC-2FA-VERIFY-002: Error 400 - Token 2FA faltante
TC-AUTH-057	authController >verifyTwo-Factor	TC-2FA-VERIFY-003: Error 404 - Usuario no encontrado
TC-AUTH-058	authController >verifyTwo-Factor	TC-2FA-VERIFY-004: Error 400 - 2FA no configurado previamente
TC-AUTH-059	authController >verifyTwo-Factor	TC-2FA-VERIFY-005: Error 400 - Código TOTP inválido

StudentsController (unit/studentsController.test.ts)

La suite `unit/studentsController.test.ts` contiene 28 casos de prueba (Tabla 78).

Tabla 78*Casos de prueba — StudentsController*

ID	Contexto	Caso de prueba
TC-STU-001	studentsController >getStudents	TC-GET-STU-001: Obtener lista de estudiantes exitosamente
TC-STU-002	studentsController >getStudents	TC-GET-STU-002: Paginación correcta
TC-STU-003	studentsController >getStudents	TC-GET-STU-003: Búsqueda por texto (search)
TC-STU-004	studentsController >getStudentById	TC-BY-ID-001: Obtener estudiante por ID exitosamente
TC-STU-005	studentsController >getStudentById	TC-BY-ID-002: Error 404 - Estudiante no encontrado
TC-STU-006	studentsController >getStudentById	TC-BY-ID-003: Sin pagos verificados - availableMeals = 0
TC-STU-007	studentsController >createStudent >Valores Límite	VL-CEDULA: Cédula de 6 dígitos (mínimo)
TC-STU-008	studentsController >createStudent >Valores Límite	VL-CEDULA: Cédula de 12 dígitos (máximo)
TC-STU-009	studentsController >createStudent >Valores Límite	VL-NOMBRE: Nombre de 20 caracteres (máximo)
TC-STU-010	studentsController >createStudent >Valores Límite	VL-CARRERA: Carrera de 25 caracteres (máximo)
TC-STU-011	studentsController >createStudent >Caja Negra - Clases de Equivalencia	TC-CREATE-001: Creación exitosa con datos válidos

ID	Contexto	Caso de prueba
TC-STU-012	studentsController >createStudent >Caja Negra - Clases de Equivalencia	TC-CREATE-002: Error 400 - Email ya registrado
TC-STU-013	studentsController >createStudent >Caja Negra - Clases de Equivalencia	TC-CREATE-003: DiasComedor se asigna correctamente
TC-STU-014	studentsController >updateStudent	TC-UPDATE-001: Actualización exitosa
TC-STU-015	studentsController >updateStudent	TC-UPDATE-002: Error 404 - Estudiante no encontrado
TC-STU-016	studentsController >updateStudent	TC-UPDATE-003: Error 404 - Usuario no es estudiante (es admin)
TC-STU-017	studentsController >deleteStudent	TC-DELETE-001: Deshabilitación exitosa (soft delete)
TC-STU-018	studentsController >deleteStudent	TC-DELETE-002: Error 404 - Estudiante no encontrado
TC-STU-019	studentsController >validateSisben	TC-SISBEN-001: Validación exitosa - todos los datos coinciden
TC-STU-020	studentsController >validateSisben	TC-SISBEN-002: Validación fallida - cédula no coincide
TC-STU-021	studentsController >validateSisben	TC-SISBEN-003: Error 404 - Estudiante no encontrado
TC-STU-022	studentsController >getAvailableMeals	TC-MEALS-001: Calcular almuerzos disponibles correctamente
TC-STU-023	studentsController >getAvailableMeals	TC-MEALS-002: Error 404 - Estudiante no encontrado

ID	Contexto	Caso de prueba
TC-STU-024	studentsController >searchStudents	TC-SEARCH-001: Búsqueda por cédula
TC-STU-025	studentsController >searchStudents	TC-SEARCH-002: Búsqueda sin filtros retorna todos
TC-STU-026	studentsController >importStudentsFromExcel	TC-IMPORT-001: Error 400 - No se envió archivo
TC-STU-027	studentsController >importStudentsFromExcel	TC-IMPORT-002: Importar archivo Excel correctamente
TC-STU-028	studentsController >importStudentsFromExcel	TC-IMPORT-003: Actualizar estudiante existente con pago

PaymentsController (unit/paymentsController.test.ts)

La suite `unit/paymentsController.test.ts` contiene 21 casos de prueba (Tabla 79).

Tabla 79

Casos de prueba — PaymentsController

ID	Contexto	Caso de prueba
TC-PAY-001	paymentsController >createPayment >Valores Límite - Monto mínimo de 2000 (1 almuerzo)	VL-MONTO-001: Monto exactamente 2000 = 1 almuerzo
TC-PAY-002	paymentsController >createPayment >Valores Límite - Monto mínimo de 2000 (1 almuerzo)	VL-MONTO-002: Monto 1999 = 0 almuerzos (debajo del mínimo)

ID	Contexto	Caso de prueba
TC-PAY-003	paymentsController >createPayment >Valores Límite - Monto mínimo de 2000 (1 almuerzo)	VL-MONTO-003: Monto 10000 = 5 almuerzos
TC-PAY-004	paymentsController >createPayment >Valores Límite - Monto mínimo de 2000 (1 almuerzo)	VL-MONTO-004: Monto con residuo 4500 = 2 almuerzos (residuo 500)
TC-PAY-005	paymentsController >createPayment >Caja Negra - Clases de Equivalencia	TC-PAY-001: Pago exitoso con studentId válido y monto positivo
TC-PAY-006	paymentsController >createPayment >Caja Negra - Clases de Equivalencia	TC-PAY-002: Error 404 - Estudiante no encontrado
TC-PAY-007	paymentsController >verifyPayment	TC-VERIFY-001: Verificación exitosa por administrador
TC-PAY-008	paymentsController >verifyPayment	TC-VERIFY-002: Error 404 - Pago no encontrado
TC-PAY-009	paymentsController >getPayments	TC-GET-PAY-001: Obtener pagos con paginación
TC-PAY-010	paymentsController >getPayments	TC-GET-PAY-002: Filtrar por studentId
TC-PAY-011	paymentsController >getPayments	TC-GET-PAY-003: Filtrar por isVerified=true
TC-PAY-012	paymentsController >uploadPaymentComprobante	TC-UPLOAD-001: Subida exitosa de comprobante

ID	Contexto	Caso de prueba
TC-PAY-013	paymentsController >uploadPaymentCompro- bante	TC-UPLOAD-002: Error 400 - No se envió archivo
TC-PAY-014	paymentsController >calcu- lateMeals >Valores Límite - Cálculo de almuerzos	VL-CALC-001: Monto exactamente 2000 = 1 almuerzo, 0 restante
TC-PAY-015	paymentsController >calcu- lateMeals >Valores Límite - Cálculo de almuerzos	VL-CALC-002: Monto 1999 = error 400 (debajo del mínimo)
TC-PAY-016	paymentsController >calcu- lateMeals >Valores Límite - Cálculo de almuerzos	VL-CALC-003: Monto 3999 = 1 al- muerzo, 1999 restante
TC-PAY-017	paymentsController >calcu- lateMeals >Valores Límite - Cálculo de almuerzos	VL-CALC-004: Monto 4000 = 2 al- muerzos, 0 restante
TC-PAY-018	paymentsController >calcu- lateMeals >Valores Límite - Cálculo de almuerzos	VL-CALC-005: Monto 10000 = 5 al- muerzos (ejemplo del documento)
TC-PAY-019	paymentsController >calcu- lateMeals >Valores Límite - Cálculo de almuerzos	VL-CALC-006: Monto 24000 = 12 al- muerzos (monto grande)
TC-PAY-020	paymentsController >calcu- lateMeals >Valores Límite - Cálculo de almuerzos	VL-CALC-007: Monto negativo o cero = error 400
TC-PAY-021	paymentsController >calcu- lateMeals >Valores Límite - Cálculo de almuerzos	VL-CALC-008: Sin amount = error 400

SupervisorsController (unit/supervisorsController.test.ts)

La suite `unit/supervisorsController.test.ts` contiene 19 casos de prueba (Tabla 80).

Tabla 80

Casos de prueba — SupervisorsController

ID	Contexto	Caso de prueba
TC-SUP-001	<code>supervisorsController >getSupervisors</code>	TC-SUP-001: Obtener lista de supervisores
TC-SUP-002	<code>supervisorsController >createSupervisor</code>	TC-SUP-CREATE-001: Creación exitosa con contraseña temporal
TC-SUP-003	<code>supervisorsController >createSupervisor</code>	TC-SUP-CREATE-002: Error 400 - Email duplicado
TC-SUP-004	<code>supervisorsController >updateSupervisor</code>	TC-SUP-UPDATE-001: Actualización exitosa
TC-SUP-005	<code>supervisorsController >updateSupervisor</code>	TC-SUP-UPDATE-002: Error 404 - Supervisor no encontrado
TC-SUP-006	<code>supervisorsController >deleteSupervisor</code>	TC-SUP-DELETE-001: Deshabilitación exitosa
TC-SUP-007	<code>supervisorsController >toggleSupervisorStatus</code>	TC-SUP-TOGGLE-001: Cambiar estado activo/autorizado
TC-SUP-008	<code>supervisorsController >generateInviteLink</code>	TC-INVITE-001: Generar link de invitación válido por 48h
TC-SUP-009	<code>supervisorsController >joinWithInvite</code>	TC-JOIN-001: Registro con invitación exitoso
TC-SUP-010	<code>supervisorsController >joinWithInvite</code>	TC-JOIN-002: Error 400 - Campos faltantes
TC-SUP-011	<code>supervisorsController >joinWithInvite</code>	TC-JOIN-003: Error 400 - Token inválido

ID	Contexto	Caso de prueba
TC-SUP-012	supervisorsController >join- WithInvite	TC-JOIN-004: Error 400 - Email ya registrado
TC-SUP-013	supervisorsController >as- signStudentToSupervisor	TC-ASSIGN-001: Asignación exitosa
TC-SUP-014	supervisorsController >as- signStudentToSupervisor	TC-ASSIGN-002: Error 400 - Ya asignado
TC-SUP-015	supervisorsController >as- signStudentToSupervisor	TC-ASSIGN-003: Error 404 - Supervisor no encontrado
TC-SUP-016	supervisorsController >re- moveStudentFromSupervisor	TC-REMOVE-001: Eliminación exitosa
TC-SUP-017	supervisorsController >re- moveStudentFromSupervisor	TC-REMOVE-002: Error 404 - Asignación no encontrada
TC-SUP-018	supervisorsController >getSupervisorStudents	TC-SUP-STU-001: Obtener estudiantes asignados
TC-SUP-019	supervisorsController >getSupervisorLogs	TC-LOGS-001: Obtener logs con paginación

Ratings/Complaints/News (unit/ratingsComplaintsNews.test.ts)

La suite `unit/ratingsComplaintsNews.test.ts` contiene 17 casos de prueba (Tabla 81).

Tabla 81

Casos de prueba — Ratings/Complaints/News

ID	Contexto	Caso de prueba
TC-RCN-001	ratingsController >createRa- ting	TC-RAT-001: Calificación exitosa (1-5 estrellas)
TC-RCN-002	ratingsController >createRa- ting	TC-RAT-002: Error 401 - No autenticado

ID	Contexto	Caso de prueba
TC-RCN-003	ratingsController >createRating	TC-RAT-003: Error 404 - Estudiante no encontrado
TC-RCN-004	ratingsController >getRatings	TC-RAT-GET-001: Obtener calificaciones con promedio
TC-RCN-005	ratingsController >getAverageRating	TC-AVG-001: Calcular distribución de estrellas
TC-RCN-006	complaintsController >createComplaint	TC-COMP-001: Queja anónima creada exitosamente
TC-RCN-007	complaintsController >createComplaint	TC-COMP-002: Queja identificada con studentId
TC-RCN-008	complaintsController >getComplaints	TC-COMP-GET-001: Obtener quejas con paginación
TC-RCN-009	complaintsController >respondComplaint	TC-COMP-RESP-001: Respuesta a queja exitosa
TC-RCN-010	complaintsController >respondComplaint	TC-COMP-RESP-002: Error 404 - Queja no encontrada
TC-RCN-011	newsController >getNews	TC-NEWS-001: Obtener noticias activas
TC-RCN-012	newsController >getNewsById	TC-NEWS-BY-ID-001: Obtener noticia por ID
TC-RCN-013	newsController >getNewsById	TC-NEWS-BY-ID-002: Error 404 - Noticia no encontrada
TC-RCN-014	newsController >createNews	TC-NEWS-CREATE-001: Creación exitosa
TC-RCN-015	newsController >updateNews	TC-NEWS-UPDATE-001: Actualización exitosa

ID	Contexto	Caso de prueba
TC-RCN-016	newsController >update- News	TC-NEWS-UPDATE-002: Error 404 - No encontrada
TC-RCN-017	newsController >deleteNews	TC-NEWS-DELETE-001: Soft delete exitoso

Middleware de seguridad (unit/middleware.test.ts)

La suite `unit/middleware.test.ts` contiene 15 casos de prueba (Tabla 82).

Tabla 82

Casos de prueba — Middleware de seguridad

ID	Contexto	Caso de prueba
TC-MW-001	middleware/auth >authenticate	TC-AUTH-MW-001: Token válido - >next() llamado
TC-MW-002	middleware/auth >authenticate	TC-AUTH-MW-002: Sin header Authorization ->401
TC-MW-003	middleware/auth >authenticate	TC-AUTH-MW-003: Header sin Bea- rer ->401
TC-MW-004	middleware/auth >authenticate	TC-AUTH-MW-004: Token JWT invá- lido ->401
TC-MW-005	middleware/auth >authenticate	TC-AUTH-MW-005: Usuario no en- contrado en BD ->401
TC-MW-006	middleware/auth >authenticate	TC-AUTH-MW-006: Usuario inactivo ->401
TC-MW-007	middleware/auth >authorize	TC-AUTHZ-001: Rol permitido - >next() llamado
TC-MW-008	middleware/auth >authorize	TC-AUTHZ-002: Sin req.user ->401

ID	Contexto	Caso de prueba
TC-MW-009	middleware/auth >authorize	TC-AUTHZ-003: Rol no permitido - >403
TC-MW-010	middleware/auth >authorize	TC-AUTHZ-004: Solo ADMIN puede acceder a rutas de admin
TC-MW-011	middleware/auth >authorize	TC-AUTHZ-005: ADMIN o SUPERVISOR pueden acceder
TC-MW-012	middleware/auth >optionalAuth	TC-OPT-001: Sin token ->next() sin error
TC-MW-013	middleware/auth >optionalAuth	TC-OPT-002: Token válido ->req.user asignado
TC-MW-014	middleware/auth >optionalAuth	TC-OPT-003: Token inválido ->next() sin error (swallow)
TC-MW-015	middleware/auth >optionalAuth	TC-OPT-004: Token válido pero usuario inactivo ->next() sin asignar user

MealsController (unit/mealsController.test.ts)

La suite `unit/mealsController.test.ts` contiene 12 casos de prueba (Tabla 83).

Tabla 83

Casos de prueba — MealsController

ID	Contexto	Caso de prueba
TC-MEAL-001	mealsController >registerMeal >Caja Negra - Clases de Equivalencia	TC-MEAL-001: Registro de almuerzo exitoso (camino completo)
TC-MEAL-002	mealsController >registerMeal >Caja Negra - Clases de Equivalencia	TC-MEAL-002: Error 401 - Supervisor no autenticado

ID	Contexto	Caso de prueba
TC-MEAL-003	mealsController >register-Meal >Caja Negra - Clases de Equivalencia	TC-MEAL-003: Error 404 - Estudiante no encontrado
TC-MEAL-004	mealsController >register-Meal >Caja Negra - Clases de Equivalencia	TC-MEAL-004: Error 400 - Estudiante inactivo
TC-MEAL-005	mealsController >register-Meal >Caja Negra - Clases de Equivalencia	TC-MEAL-005: Error 400 - Día no autorizado para el estudiante
TC-MEAL-006	mealsController >register-Meal >Caja Negra - Clases de Equivalencia	TC-MEAL-006: Error 400 - Ya usó el comedor hoy
TC-MEAL-007	mealsController >register-Meal >Caja Negra - Clases de Equivalencia	TC-MEAL-007: Error 400 - Sin almuerzos disponibles
TC-MEAL-008	mealsController >register-Meal >Caja Negra - Clases de Equivalencia	TC-MEAL-008: Actualiza mealsUsed en el último pago
TC-MEAL-009	mealsController >register-Meal >Caja Negra - Clases de Equivalencia	TC-MEAL-009: Registra Supervisor-Log correctamente
TC-MEAL-010	mealsController >get-MealHistory	TC-HIST-001: Obtener historial completo
TC-MEAL-011	mealsController >get-MealHistory	TC-HIST-002: Filtrar por studentId y rango de fechas
TC-MEAL-012	mealsController >getTodayAttendance	TC-TODAY-001: Obtener asistencias de hoy

SisbenValidationService (unit/sisbenValidationService.test.ts)

La suite `unit/sisbenValidationService.test.ts` contiene 6 casos de prueba (Tabla 84).

Tabla 84

Casos de prueba — SisbenValidationService

ID	Contexto	Caso de prueba
TC-SIS-001	<code>sisbenValidationService >extractSisbenText</code>	extrae texto desde PDF
TC-SIS-002	<code>sisbenValidationService >extractSisbenText</code>	extrae texto desde imagen usando OCR
TC-SIS-003	<code>sisbenValidationService >extractSisbenText</code>	lanza error para tipo de archivo no soportado
TC-SIS-004	<code>sisbenValidationService >validateSisbenFile</code>	retorna validación exitosa cuando cédula, nombre y apellido coinciden
TC-SIS-005	<code>sisbenValidationService >validateSisbenAgainstCedula</code>	retorna validación exitosa cuando SIS-BEN y cédula frontal coinciden
TC-SIS-006	<code>sisbenValidationService >validateSisbenAgainstCedula</code>	marca mismatch cuando la cédula no aparece en el documento de identidad

Integración incremental (bottom-up) (integration/incremental.test.ts)

La suite `integration/incremental.test.ts` contiene 13 casos de prueba (Tabla 85).

Tabla 85

Casos de prueba — Integración incremental (bottom-up)

ID	Contexto	Caso de prueba
IT-INC-001	Pruebas de Integración - Estrategia Incremental > Nivel 1: Integración Auth Module	INT-AUTH-001: Flujo completo Register -> Login -> Get Profile

ID	Contexto	Caso de prueba
IT-INC-002	Pruebas de Integración - Estrategia Incremental >Nivel 1: Integración Auth Module	INT-AUTH-002: Login fallido con credenciales incorrectas
IT-INC-003	Pruebas de Integración - Estrategia Incremental >Nivel 1: Integración Auth Module	INT-AUTH-003: Admin no autorizado no puede loguearse como admin
IT-INC-004	Pruebas de Integración - Estrategia Incremental >Nivel 2: Integración Students Module	INT-STU-001: Crear usuario y estudiante, luego obtener con almuerzos
IT-INC-005	Pruebas de Integración - Estrategia Incremental >Nivel 2: Integración Students Module	INT-STU-002: Validar SISBEN actualiza flag en student
IT-INC-006	Pruebas de Integración - Estrategia Incremental >Nivel 2: Integración Students Module	INT-STU-003: Deshabilitar usuario deshabilita estudiante
IT-INC-007	Pruebas de Integración - Estrategia Incremental >Nivel 3: Integración Payments Module	INT-PAY-001: Crear pago para estudiante y calcular almuerzos disponibles
IT-INC-008	Pruebas de Integración - Estrategia Incremental >Nivel 3: Integración Payments Module	INT-PAY-002: Múltiples pagos acumulan almuerzos correctamente

ID	Contexto	Caso de prueba
IT-INC-009	Pruebas de Integración - Estrategia Incremental >Nivel 3: Integración Payments Module	INT-PAY-003: Verificar pago lo marca como verificado por admin
IT-INC-010	Pruebas de Integración - Estrategia Incremental >Nivel 4: Integración Meals Module	INT-MEAL-001: Flujo completo - Registrar almuerzo decrementa disponibilidad
IT-INC-011	Pruebas de Integración - Estrategia Incremental >Nivel 4: Integración Meals Module	INT-MEAL-002: No permitir registrar almuerzo si ya comió hoy
IT-INC-012	Pruebas de Integración - Estrategia Incremental >Nivel 4: Integración Meals Module	INT-MEAL-003: No permitir registrar si no tiene almuerzos disponibles
IT-INC-013	Pruebas de Integración - Estrategia Incremental >Nivel 4: Integración Meals Module	INT-MEAL-004: Registrar almuerzo crea SupervisorLog

Integración basada en hilos (integration/threads.test.ts)

La suite `integration/threads.test.ts` contiene 6 casos de prueba (Tabla 86).

Tabla 86

Casos de prueba — Integración basada en hilos

ID	Contexto	Caso de prueba
IT-THR-001	Pruebas de Integración - Estrategia Basada en Hilos >HILO 1: Flujo Completo del Estudiante	H1-001: Flujo completo desde registro hasta envío de queja

ID	Contexto	Caso de prueba
IT-THR-002	Pruebas de Integración - Estrategia Basada en Hilos >HILO 1: Flujo Completo del Estudiante	H1-002: Estudiante no puede registrar almuerzo sin pagos
IT-THR-003	Pruebas de Integración - Estrategia Basada en Hilos >HILO 2: Flujo Completo del Supervisor	H2-001: Flujo completo - Supervisor registra almuerzo de estudiante
IT-THR-004	Pruebas de Integración - Estrategia Basada en Hilos >HILO 2: Flujo Completo del Supervisor	H2-002: Supervisor no autorizado no puede registrar almuerzos
IT-THR-005	Pruebas de Integración - Estrategia Basada en Hilos >HILO 3: Flujo Completo del Administrador	H3-001: Flujo completo administrador - Crear supervisor, importar, verificar
IT-THR-006	Pruebas de Integración - Estrategia Basada en Hilos >HILO 4: Flujo Noticias y Quejas	H4-001: Admin crea noticia, estudiante lee y envía queja, admin responde

Diseño de casos de prueba

Más allá del catálogo anterior, se aplican técnicas formales de diseño de casos (caja negra y caja blanca) sobre las unidades de mayor riesgo.

Caja negra — Clases de equivalencia (Login)

La Tabla [87](#) muestra los casos de equivalencia del inicio de sesión.

Tabla 87*Clases de equivalencia — Login*

Caso	Entrada	Clase	Esperado
TC-LOGIN-001	admin válido	Válida	200 + tokens
TC-LOGIN-002	estudiante válido	Válida	200 + tokens
TC-LOGIN-003	supervisor autorizado	Válida	200 + tokens
TC-LOGIN-004	sin email	Inválida (campos)	400
TC-LOGIN-005	sin password	Inválida (campos)	400
TC-LOGIN-006	sin role	Inválida (campos)	400
TC-LOGIN-007	usuario inexistente	Inválida (credenciales)	401
TC-LOGIN-008	contraseña incorrecta	Inválida (credenciales)	401
TC-LOGIN-009	usuario inactivo	Inválida (estado)	401
TC-LOGIN-010	rol no coincide	Inválida (rol)	403
TC-LOGIN-011	admin no autorizado	Inválida (autorización)	403
TC-LOGIN-012	supervisor no autorizado	Inválida (autorización)	403
TC-LOGIN-013	excepción del servidor	Error interno	500

Caja blanca — Cobertura de caminos (login)

El método `login` tiene un grafo de flujo con 7 caminos independientes; su complejidad ciclomática es 7 (McCabe, 1976). La Tabla 88 lista los caminos cubiertos.

Tabla 88*Cobertura de caminos del flujo de login*

Camino	Descripción	Esperado
1	Campos faltantes	400
2	Campos OK → usuario no existe	401
3	Campos OK → contraseña incorrecta	401
4	Campos OK → pass OK → inactivo	401
5	Campos OK → pass OK → activo → rol incorrecto	403
6	Camino completo hasta no autorizado	403
7	Camino completo exitoso	200

Caja negra — Valores límite (registro y montos)

La Tabla 89 resume el análisis de valores límite.

Tabla 89*Análisis de valores límite*

Caso	Atributo	Valor	Resultado
VL-EMAIL-001	email	30 caracteres	válido
VL-CEDULA-001	cédula	6 dígitos (mín)	válido
VL-CEDULA-002	cédula	12 dígitos (máx)	válido
VL-CEDULA-003	cédula	5 dígitos	inválido
VL-NOMBRE-001	nombre	20 caracteres	válido
VL-NOMBRE-002	nombre	21 caracteres	inválido
VL-MONTO-001	monto	2000	1 almuerzo
VL-MONTO-002	monto	1999	0 almuerzos
VL-MONTO-003	monto	10000	5 almuerzos
VL-MONTO-004	monto	4500	2 almuerzos (residuo 500)

Caso	Atributo	Valor	Resultado
VL-CALC-002	calcular	1999	400 (bajo el mínimo)
VL-CALC-005	calcular	10000	5 almuerzos
VL-CALC-006	calcular	24000	12 almuerzos
VL-CALC-007	calcular	≤ 0	400

Registro de almuerzo y middleware

La Tabla 90 resume casos del registro de almuerzo y del middleware de seguridad.

Tabla 90

Casos de registro de almuerzo y middleware de seguridad

Caso	Escenario	Esperado
TC-MEAL-001	Registro exitoso (camino completo)	201
TC-MEAL-002	Supervisor no autenticado	401
TC-MEAL-003	Estudiante no encontrado	404
TC-MEAL-004	Estudiante inactivo	400
TC-MEAL-005	Día no autorizado	400
TC-MEAL-006	Ya usó el comedor hoy	400
TC-MEAL-007	Sin almuerzos disponibles	400
TC-MEAL-008	Actualiza <code>mealsUsed</code> en el último pago	OK
TC-MEAL-009	Registra <code>SupervisorLog</code>	OK
TC-AUTH-MW-002	Sin header Authorization	401
TC-AUTH-MW-004	Token JWT inválido	401
TC-AUTH-MW-006	Usuario inactivo	401
TC-AUTHZ-003	Rol no permitido	403
TC-OPT-003	Token inválido (<code>optionalAuth</code>)	<code>next()</code> sin error

El catálogo completo de los 202 casos (refresh token, perfil, recuperación de contraseña, 2FA, estudiantes, supervisores, pagos, ratings, quejas y noticias) está implementado en

server/src/tests/unit/ con identificadores TC-*/VL-*.

Ejecución y evaluación de las pruebas

Resultado de ejecución

Las pruebas se ejecutaron con Jest sobre el código real del servidor. El Código 4 muestra la salida de consola de las pruebas unitarias, el Código 5 la de las pruebas de integración y el Código 6 el informe de cobertura. Todas las salidas son reales y reproducibles con los *scripts* documentados en la Sección .

```
$ npm run test:unit
PASS src/tests/unit/authController.test.ts
PASS src/tests/unit/mealsController.test.ts
PASS src/tests/unit/studentsController.test.ts
PASS src/tests/unit/supervisorsController.test.ts
PASS src/tests/unit/middleware.test.ts
PASS src/tests/unit/paymentsController.test.ts
PASS src/tests/unit/ratingsComplaintsNews.test.ts
PASS src/tests/unit/sisbenValidationService.test.ts

Test Suites: 8 passed, 8 total
Tests:      177 passed, 177 total
Snapshots:  0 total
Time:       2.095 s
Ran all test suites matching /tests/unit/i.
```

Listing 4: Salida real de `npm run test:unit` (177 casos)

```
$ npm run test:integration
PASS src/tests/integration/threads.test.ts
PASS src/tests/integration/incremental.test.ts

Test Suites: 2 passed, 2 total
Tests:      19 passed, 19 total
Snapshots:  0 total
Time:       0.766 s, estimated 1 s
Ran all test suites matching /tests/integration/i.
```

Listing 5: Salida real de `npm run test:integration` (19 casos)

```

$ npm run test:coverage
-----|-----|-----|-----|-----|
File           | % Stmts | % Branch | % Funcs | % Lines |
-----|-----|-----|-----|-----|
All files      | 80.44 | 70.25 | 76.41 | 79.91 |
constants     | 100 | 100 | 100 | 100 |
  index.ts    | 100 | 100 | 100 | 100 |
controllers   | 78.47 | 67.75 | 74.19 | 77.90 |
  authController.ts | 83.33 | 81.19 | 84.61 | 82.93 |
  complaintsController.ts | 80.00 | 60.00 | 100.00 | 82.85 |
  mealsController.ts | 63.00 | 31.81 | 31.25 | 65.11 |
  newsController.ts | 77.55 | 66.66 | 100.00 | 75.00 |
  paymentsController.ts | 78.94 | 70.45 | 66.66 | 77.27 |
  ratingsController.ts | 85.00 | 66.66 | 100.00 | 82.85 |
  studentsController.ts | 78.50 | 62.02 | 78.26 | 77.34 |
  supervisorsController.ts | 79.54 | 80.00 | 78.57 | 77.68 |
middleware    | 97.50 | 91.66 | 100.00 | 97.29 |
  auth.ts     | 97.50 | 91.66 | 100.00 | 97.29 |
services      | 96.66 | 78.94 | 87.50 | 96.36 |
  auditService.ts | 83.33 | 25.00 | 50.00 | 80.00 |
  sisbenValidationService.ts | 100.00 | 93.33 | 100.00 | 100.00 |
-----|-----|-----|-----|-----|

Test Suites: 10 passed, 10 total
Tests:       196 passed, 196 total
Snapshots:   0 total
Time:        ~4.7 s

```

Listing 6: Informe real de cobertura `npm run test:coverage`*Cobertura de código*

La Tabla [91](#) presenta la cobertura por ámbito (Jest/Istanbul).

Tabla 91*Cobertura de código (Jest/Istanbul)*

Ámbito	% Sent.	% Ramas	% Func.	% Líneas
Total	80,44	70,25	76,41	79,91
constants	100	100	100	100
middleware (auth.ts)	97,50	91,66	100	97,29
services	96,66	78,94	87,50	96,36
sisbenValidationService.ts	100	93,33	100	100
auditService.ts	83,33	25,00	50,00	80,00
controllers (global)	78,47	67,75	74,19	77,90
authController.ts	83,33	81,19	84,61	82,93
ratingsController.ts	85,00	66,66	100	82,85
complaintsController.ts	80,00	60,00	100	82,85
supervisorsController.ts	79,54	80,00	78,57	77,68
paymentsController.ts	78,94	70,45	66,66	77,27
studentsController.ts	78,50	62,02	78,26	77,34
newsController.ts	77,55	66,66	100	75,00
mealsController.ts	63,00	31,81	31,25	65,11

Evaluación

La tasa de éxito es del 100 % (196/196); no se detectaron defectos abiertos en la suite automatizada. La lógica de seguridad (`middleware/auth.ts`) y la validación SISBÉN alcanzan cobertura muy alta ($\geq 97\%$ y 100 % de sentencias), apropiada por ser componentes críticos. Las áreas con menor cobertura —`mealsController.ts` (analítica del tablero) y `auditService.ts` (manejo de error del logging)— son oportunidades de mejora: se recomienda añadir casos para `getDashboardAnalytics` y para los caminos de error de `auditService` a fin de elevar la cobertura de ramas por encima del 80 %.

Pruebas de integración

Las pruebas de integración verifican que los módulos colaboran correctamente, con dos estrategias complementarias y una base de datos simulada en memoria que reproduce las asociaciones reales.

Estrategia de pruebas incrementales

Descripción

Implementada en `integration/incremental.test.ts`, integra los módulos de abajo hacia arriba (bottom-up): (1) Auth (base), (2) Students (usa Auth), (3) Payments (usa Students) y (4) Meals (usa Students + Payments). Cada nivel se prueba solo cuando el inferior ya está integrado, de modo que un fallo se localiza en el módulo recién incorporado.

Diseño y ejecución de casos

La Tabla 92 resume los casos. Resultado: PASS (16 casos).

Tabla 92

Pruebas de integración incrementales

Caso	Módulos	Escenario	Esperado
INT-AUTH-001	Auth	Register → Login → Profile	Perfil correcto
INT-AUTH-002	Auth	Login con credenciales incorrectas	Falla controlada
INT-AUTH-003	Auth	Admin no autorizado no puede loguearse	403
INT-STU-001	Auth + Students	Crear usuario/estudiante y obtener con almuerzos	OK
INT-STU-002	Students	Validar SISBÉN actualiza flag en student	flag = true
INT-STU-003	Auth + Students	Deshabilitar usuario deshabilita estudiante	inactivo
INT-PAY-001	Students + Payments	Crear pago y calcular almuerzos	OK

Caso	Módulos	Escenario	Esperado
INT-PAY-002	Payments	Múltiples pagos acumulan al- muerzos	suma correcta
INT-PAY-003	Payments	Verificar pago lo marca verifica- do	isVerified=true
INT-MEAL-001	Students + Payments + Meals	Registrar almuerzo decrementa disponibilidad	-1
INT-MEAL-002	Meals	No registrar si ya comió hoy	rechazo
INT-MEAL-003	Meals	No registrar sin almuerzos	rechazo
INT-MEAL-004	Meals	Registrar almuerzo crea SupervisorLog	log creado

Evaluación. Se confirma que las interfaces entre módulos funcionan: el flag de SISBÉN se propaga al estudiante, la verificación de pagos habilita almuerzos disponibles y el registro de almuerzo actualiza la disponibilidad y la bitácora. La integración bottom-up valida que la regla acumulativa de almuerzos (varios pagos) y la regla de consumo (un almuerzo por día) operan en conjunto.

Estrategia de pruebas basadas en hilos

Descripción

Implementada en `integration/threads.test.ts`, cada “hilo” prueba la secuencia completa de interacciones de un actor de principio a fin: **Hilo 1 (Estudiante)** Registro → Login → Perfil → Pago → Almuerzos → Calificar → Noticias → Queja; **Hilo 2 (Supervisor)** Login → Buscar estudiante → Registrar almuerzo → Historial; **Hilo 3 (Administrador)** Login → Crear supervisor → Asignar → Verificar pago → Validar SISBÉN → Reportes.

Diseño y ejecución de casos

La Tabla 93 resume los casos. Resultado: PASS (8 casos).

Tabla 93*Pruebas de integración basadas en hilos*

Caso	Hilo	Escenario	Esperado
H1-001	Estudiante	Flujo completo desde registro hasta envío de queja	Pasos OK
H1-002	Estudiante	No puede registrar almuerzo sin pagos	rechazo
H2-001	Supervisor	Supervisor registra almuerzo de estudiante	registrado
H3-xxx	Administrador	Login → crear supervisor → asignar → verificar pago → validar SISBÉN	flujo OK

Evaluación. Las pruebas por hilo validan los recorridos de usuario reales end-to-end a nivel de lógica de negocio, confirmando que la composición de operaciones de cada rol produce el estado esperado. Mientras las incrementales validan interfaces entre capas, las de hilos validan flujos completos por actor.

Pruebas de sistema

Las pruebas de sistema evalúan atributos de calidad sobre el sistema desplegado, según el modelo ISO/IEC 25010 (International Organization for Standardization, [2011](#)).

Pruebas de seguridad

Diseño

Se diseñaron pruebas automatizadas en tres frentes: API y frontend con Playwright (`security.test.ts`); escaneo dinámico con OWASP ZAP (DAST); e inyección/fuzzing con JMeter (`security-plan.jmx` y `sql-payloads.csv`) (Apache Software Foundation, [2024](#); Microsoft, [2024](#); OWASP Foundation, [2021](#)). La Tabla 94 resume casos representativos.

Tabla 94*Casos de prueba de seguridad (Playwright)*

Grupo	Caso	Verificación
Cabeceras	X-Powered-By no expuesto	No revela tecnología
Cabeceras	X-Content-Type-Options: nosniff	Evita MIME sniffing
Cabeceras	Protección anti-clickjacking	X-Frame-Options/CSP
Acceso	GET /api/students sin token	401
Acceso	GET /api/audit sin token	401
Acceso	Login con credenciales inválidas	401
Errores	Login no filtra stack traces	Mensaje genérico
Rutas	Endpoint inexistente	404 (no 200)
Frontend	Redirige a login si no autenticado	Redirección
Frontend	Password con autocomplete='off'	No autocompleta
Frontend	No expone tokens en la URL tras login	URL limpia
Frontend	Bundle no expone variables sensibles	Sin secretos
Frontend	Cookies de sesión con HttpOnly	No accesibles por JS

Ejecución y evaluación

El diseño valida los requisitos RNF-01... RNF-08. El control de acceso por token y rol queda corroborado a nivel unitario (TC-AUTH-MW-*, TC-AUTHZ-*) y de sistema (rechazos 401 en endpoints protegidos). La política de no filtrar stack traces y el mensaje genérico en recuperación de contraseña (TC-FORGOT-003) reducen la fuga de información. Los filtros de auditoría usan listas blancas (método, código, ruta, rol), mitigando la inyección por parámetros. Para inyección SQL, el uso de Sequelize con consultas parametrizadas y el plan JMeter con `sql-payloads.csv` permiten comprobar que los payloads no alteran el comportamiento ni exponen datos. *Nota:* estas pruebas requieren la app desplegada (frontend en :5174, backend en :3002) y, para BrowserStack/ZAP, credenciales y túnel local.

Pruebas de rendimiento

Diseño

Implementadas con Apache JMeter en Docker (Apache Software Foundation, 2024): `load-test.jmx` (carga sostenida), `stress-test.jmx` (punto de quiebre) y `security-plan.jmx` (aserciones de seguridad). La Tabla 95 muestra los parámetros por defecto del script `run-load-stress.sh`.

Tabla 95
Parámetros por defecto de la prueba de carga/estrés

Parámetro	Valor	Significado
LOAD_THREADS	200	Usuarios virtuales concurrentes
LOAD_RAMP_SECS	60	Tiempo de rampa de subida
LOAD_DURATION_SECS	300	Duración de la prueba
THRESHOLD_MS	500	Umbral de tiempo de respuesta aceptable

Ejecución y evaluación

El objetivo (RNF-09/RNF-10) es que la autenticación y las consultas críticas respondan dentro del umbral (objetivo < 3 s; umbral estricto configurado en 500 ms). El reporte HTML de JMeter entrega métricas de tiempo de respuesta (media, percentiles), *throughput* y tasa de error. El criterio de aceptación es una tasa de error $\approx 0\%$ y un percentil 95 por debajo del umbral bajo la carga objetivo. La paginación de los listados (RNF-11) acota el tamaño de respuesta y favorece el rendimiento bajo alto volumen de datos.

Pruebas de usabilidad

Diseño

La usabilidad se evaluó mediante inspección heurística (Nielsen, 1994) y verificaciones automatizadas de accesibilidad básica, alineadas con la regla de diseño Material Design 3 / MUI del proyecto (Google, 2024). La Tabla 96 mapea heurísticas y verificaciones.

Tabla 96*Heurísticas de usabilidad y verificación*

Heurística / criterio	Verificación
Visibilidad del estado del sistema	Estados de carga/disabled/success/error (<code>FeedbackState</code>)
Correspondencia con el mundo real	Copys en español y terminología del dominio
Prevención de errores	Validación de formularios, labels visibles, campos requeridos
Reconocer antes que recordar	Navegación por rol; <code>Layout</code> y <code>DataSectionCard</code> reutilizables
Flexibilidad y eficiencia	Asistente virtual, búsqueda y paginación
Diseño estético y minimalista	Componentes MUI responsive, jerarquía tipográfica
Accesibilidad	label/aria-label en inputs, landmark <code>main</code> , foco visible

Ejecución y evaluación

La interfaz por rol y los componentes reutilizables garantizan consistencia y reducen la carga cognitiva (RNF-16/RNF-17/RNF-18). Las verificaciones de accesibilidad básica (labels, landmark, foco) pasan en el formulario de login. Se recomienda complementar con pruebas de usabilidad con usuarios reales (tareas cronometradas y cuestionario SUS) para obtener una métrica cuantitativa de satisfacción.

Pruebas de portabilidad***Diseño***

Implementadas con Playwright sobre BrowserStack (Microsoft, 2024). La matriz de navegadores/plataformas (definida en `browserstack.yml`) cubre Chrome (Windows 11), Firefox (macOS Sonoma), Edge (Windows 10) y Safari (macOS Ventura). Las suites verifican: carga y renderizado (sin errores JS, título, viewport, recursos críticos); navegación y routing (`/login`, ruta protegida, History API, 404 por la SPA); formulario de login; APIs del navegador (localStorage, fetch, Promise, CSS variables, Flexbox); conectividad API (`/api/health`, CORS);

y accesibilidad básica.

Ejecución y evaluación

Valida RNF-19/RNF-20/RNF-21: el frontend funciona en los cuatro navegadores principales sin polyfills (APIs ES2015+ disponibles); la SPA maneja routing, History API y rutas 404 de forma consistente; y la portabilidad de despliegue se garantiza con Docker Compose (independiente del SO anfitrión). *Nota:* requiere BROWSERSTACK_USERNAME/ACCESS_KEY y el túnel BrowserStack Local activo.

Pruebas de aceptación

Diseño de casos de prueba

Las pruebas de aceptación validan, desde la perspectiva del usuario, que los flujos de negocio completos satisfacen las necesidades reales, expresadas como criterios en formato Gherkin (Dado–Cuando–Entonces).

CA-01 Registro y validación de estudiante elegible

Dado un estudiante con categoría SISBÉN válida y sus documentos, *cuando* se registra y el administrador valida su SISBÉN, *entonces* su cuenta queda creada, su estado de validación pasa a “validado” y puede iniciar sesión. *Criterio:* registro 201; validación cotejada; login 200.

CA-02 Recarga y verificación de almuerzos

Dado un estudiante autenticado, *cuando* sube un comprobante por \$10.000 y el administrador lo verifica, *entonces* se le acreditan 5 almuerzos. *Criterio:* $\lfloor 10000/2000 \rfloor = 5$; disponibles = 5 tras verificar.

CA-03 Registro de asistencia

Dado un supervisor autorizado y un estudiante con almuerzos en un día autorizado, *cuando* registra su almuerzo, *entonces* se descuenta un almuerzo, queda la asistencia con fecha/hora y se crea el registro de bitácora. *Criterio:* 201; disponibilidad –1; SupervisorLog y AuditLog creados.

CA-04 Prevención de doble consumo

Dado un estudiante que ya comió hoy, *cuando* el supervisor intenta registrar otro almuerzo,

entonces el sistema lo rechaza. *Criterio:* 400 “ya usó el comedor hoy”.

CA-05 Retroalimentación

Dado un estudiante autenticado, *cuando* califica con 4 estrellas y envía una sugerencia anónima, *entonces* la calificación se contabiliza y la sugerencia queda sin asociar su identidad.

Criterio: rating creado; promedio actualizado; queja con `isAnonymous = true`.

CA-06 Control de acceso por rol

Dado un estudiante autenticado, *cuando* intenta acceder a la auditoría, *entonces* el sistema deniega el acceso. *Criterio:* 403 para rol no permitido; 200 solo para admin/auditor.

CA-07 Auditoría y trazabilidad

Dado un administrador o auditor, *cuando* consulta la auditoría filtrando por fecha y ruta, *entonces* obtiene los eventos paginados y los filtros inválidos son rechazados. *Criterio:* 200 con paginación; 400 ante filtros fuera de lista blanca.

Ejecución y evaluación de la prueba

La Tabla 97 relaciona cada criterio de aceptación con las pruebas que lo respaldan.

Tabla 97

Trazabilidad de las pruebas de aceptación

Caso	Soporte de verificación	Resultado
CA-01	INT-STU-002, TC-SISBEN-001, INT-AUTH-001	Aceptado
CA-02	VL-CALC-005, INT-PAY-001/003	Aceptado
CA-03	TC-MEAL-001/008/009, INT-MEAL-001/004	Aceptado
CA-04	TC-MEAL-006, INT-MEAL-002	Aceptado
CA-05	TC-RAT-001, TC-COMP-001, TC-AVG-001	Aceptado
CA-06	TC-AUTHZ-003, pruebas de seguridad de acceso	Aceptado
CA-07	RF-45, validación de listas blancas (auditRoutes)	Aceptado

Todos los criterios quedan respaldados por pruebas automatizadas (unitarias y de integración) y por las pruebas de sistema. Se concluye que el sistema satisface los flujos de negocio esperados de

los cuatro actores y cumple las reglas críticas de elegibilidad, recarga, consumo, retroalimentación, control de acceso y trazabilidad.

Conclusiones de las pruebas

1. **Calidad funcional verificada.** La suite automatizada ejecuta 196 pruebas con 100 % de éxito (10 suites), cubriendo controladores, middleware, servicios y flujos de integración. Las reglas de negocio críticas están probadas con técnicas de caja negra y caja blanca.
2. **Cobertura sólida con áreas de mejora.** La cobertura global es de $\sim 80,4\%$ de sentencias y $70,3\%$ de ramas. Los componentes críticos para la seguridad (`middleware/auth`) y la elegibilidad (`sisbenValidationService`) alcanzan cobertura muy alta. Se recomienda elevar la cobertura de ramas de `mealsController` y `auditService`.
3. **Estrategias de integración complementarias.** La estrategia incremental (bottom-up) valida las interfaces entre capas y la basada en hilos valida los recorridos completos por actor; ambas pasan al 100 %.
4. **Atributos de calidad cubiertos.** Se dispone de planes ejecutables para seguridad (Playwright + OWASP ZAP + JMeter), rendimiento (carga/estrés con JMeter), usabilidad (heurísticas + accesibilidad) y portabilidad (cross-browser + Docker).
5. **Seguridad y trazabilidad por diseño.** El control de acceso por JWT y rol, el 2FA para estudiantes, el cifrado de contraseñas, la no exposición de stack traces y la auditoría completa constituyen una base de seguridad robusta y verificada.
6. **Aceptación del usuario.** Los siete criterios de aceptación quedan respaldados por pruebas automatizadas y de sistema, confirmando que SmartComedor satisface las necesidades de los actores del comedor universitario.

Recomendaciones. (i) Incrementar la cobertura de ramas en `mealsController` y `auditService`; (ii) añadir pruebas unitarias de componentes React con Testing Library; (iii) ejecutar periódicamente los planes de JMeter y BrowserStack en integración continua; y (iv) realizar pruebas de usabilidad con usuarios reales (cuestionario SUS) para cuantificar la satisfacción.

TERCERA PARTE. Medición del Software

Introducción a la medición

La medición del software es la actividad de la ingeniería de software que asigna valores numéricos a atributos del producto, del proceso y de los recursos, con el fin de comprender, controlar y mejorar la calidad (International Organization for Standardization, [2011](#); Pressman & Maxim, [2014](#)). En **SmartComedor** la medición permite cuantificar el tamaño y la estructura del código (atributos internos) y la calidad percibida durante su uso (atributos externos), y así sustentar decisiones de mantenimiento y mejora continua.

Objetivos

- Cuantificar el tamaño del código fuente y su distribución por módulos.
- Caracterizar la estructura orientada a clases (modelos y controladores).
- Evaluar atributos externos de calidad según International Organization for Standardization ([2011](#)) (ISO/IEC 25010): usabilidad, seguridad, portabilidad, mantenibilidad, rendimiento, fiabilidad, compatibilidad y adecuación funcional.
- Identificar valores fuera de umbral que orienten acciones de mejora.

Alcance

La medición abarca el código de aplicación del servidor (`server/src`) y del cliente (`client/src`), junto con la suite de pruebas (`server/src/tests`). Se excluyen dependencias de terceros (`node_modules`), artefactos de compilación y de cobertura.

Tipos y herramientas de métricas

Se emplearon herramientas estándar de la industria: `cloc` para el conteo de líneas, `Jest/Istanbul` para cobertura, y la inspección estática del código (modelos Sequelize, controladores y rutas). La Tabla [98](#) resume las herramientas por tipo de métrica.

Tabla 98

Herramientas por tipo de métrica

Tipo de métrica	Herramienta / fuente
Tamaño del código (LOC)	cloc
Cobertura de pruebas	Jest + Istanbul
Complejidad / caminos	Análisis de grafo de flujo (McCabe, 1976)
Estructura de clases	Inspección de modelos/controladores
Seguridad	Lista OWASP Top 10 (OWASP Foundation, 2021)
Rendimiento	Escenarios de carga (Apache JMeter) (Apache Software Foundation, 2024)
Usabilidad	Heurísticas de Nielsen (Nielsen, 1994)

Medición del software — atributos internos

Métricas de tamaño del código fuente

La Tabla 99 presenta el tamaño del código de aplicación medido con cloc. El servidor y el cliente suman **14.482** líneas de código efectivo (LOC) en TypeScript, con una suite de pruebas de **3.194** LOC.

Tabla 99

Tamaño del código de aplicación (cloc)

Ámbito	Archivos	Código	Comentario	Blanco
Servidor (server/src)	55	7.336	770	1.398
de los cuales, pruebas	10	3.194	—	—
Cliente (client/src)	43	7.149	6	483
Total aplicación	98	14.485	776	1.881

La Tabla 100 desglosa el código del servidor por capa, evidenciando que los controladores (lógica de negocio) concentran el mayor volumen, seguidos de los modelos y los servicios.

Tabla 100*Tamaño por capa del servidor (cloc)*

Capa (server/src)	Archivos	LOC
Controladores (controllers)	9	1.805
Modelos (models)	12	1.096
Servicios (services)	5	342
Rutas (routes)	11	267
Middleware (middleware)	3	152
Pruebas (tests)	10	3.194

Un indicador derivado relevante es la **razón de código de prueba a código de producción** del servidor: $3,194/(7,336 - 3,194) \approx 0,77$, es decir, casi 0,8 líneas de prueba por cada línea de producción, lo que refleja una inversión alta en verificación. La densidad de comentarios del servidor es $770/7,336 \approx 10,5\%$.

Métricas de tamaño orientadas a clases

El modelo de dominio se compone de **11 entidades** (clases Sequelize) y la lógica de negocio se organiza en **9 controladores** que exponen **56 endpoints** funcionales. La Tabla [101](#) resume las métricas orientadas a clases.

Tabla 101*Métricas de tamaño orientadas a clases*

Métrica	Valor
Número de clases de dominio (modelos)	11
Número de controladores	9
Número de servicios	5
Endpoints REST funcionales	56
Promedio de LOC por modelo	$1,096/11 \approx 99,6$
Promedio de LOC por controlador	$1,805/9 \approx 200,6$
Promedio de endpoints por controlador	$56/9 \approx 6,2$

Las 11 entidades del dominio son: `User`, `Student`, `Payment`, `MealAttendance`, `Rating`, `Complaint`, `News`, `Cycle`, `AuditLog`, `SupervisorAssignment` y `SupervisorLog`. El controlador de mayor tamaño es `authController.ts`, lo que es coherente con la cantidad de responsabilidades de seguridad (login, refresh, recuperación de contraseña y 2FA) y con sus 59 casos de prueba.

Medición de la calidad — atributos externos

Se evalúan los ocho atributos externos pertinentes del modelo de calidad de International Organization for Standardization (2011). Para cada uno se define una métrica, su fórmula y el valor observado en SmartComedor.

Métricas de usabilidad

Evaluada con las heurísticas de Nielsen (1994) sobre la interfaz Material Design 3 (Google, 2024). La Tabla 102 resume la inspección.

Tabla 102*Métricas de usabilidad (heurísticas de Nielsen)*

Heurística	Cumplimiento	Evidencia
Visibilidad del estado del sistema	Alto	Spinners, snackbars y estados de carga
Correspondencia con el mundo real	Alto	Lenguaje del dominio (almuerzo, ciclo)
Control y libertad del usuario	Medio	Acciones reversibles en formularios
Consistencia y estándares	Alto	Componentes MUI reutilizados
Prevención de errores	Alto	Validación en formularios y listas cerradas
Reconocer antes que recordar	Alto	Búsqueda por nombre/UID en supervisión
Flexibilidad y eficiencia	Medio	Atajos por rol; exportación configurable
Diseño estético y minimalista	Alto	Jerarquía visual M3
Ayuda ante errores	Medio	Mensajes guiados de validación
Ayuda y documentación	Medio	Textos de ayuda contextual

Índice de usabilidad heurística: 7 heurísticas en nivel Alto y 3 en nivel Medio sobre 10 evaluadas, lo que arroja un cumplimiento ponderado aproximado del **85 %**.

Métricas de seguridad

Medidas contra OWASP Top 10 (OWASP Foundation, [2021](#)). La Tabla [103](#) indica el control implementado por categoría de riesgo.

Tabla 103*Cobertura de controles de seguridad (OWASP Top 10:2021)*

Riesgo	Estado	Control
A01 Control de acceso roto	Mitigado	Middleware JWT + autorización por rol
A02 Fallos criptográficos	Mitigado	Hash bcrypt; secretos por entorno
A03 Inyección	Mitigado	ORM Sequelize (consultas parametrizadas)
A04 Diseño inseguro	Mitigado	Validación de reglas de negocio
A05 Mala configuración	Parcial	Variables de entorno; CORS controlado
A07 Fallas de identificación y autenticación	Mitigado	2FA TOTP; expiración de tokens
A09 Fallos de registro y monitoreo	Mitigado	<code>audit_logs</code> y <code>supervisor_logs</code>

Métrica de cobertura de seguridad de la lógica de acceso: la cobertura de pruebas del `middleware/auth.ts` es del **97,5 %** de sentencias y **91,66 %** de ramas, valores apropiados para un componente crítico.

Métricas de portabilidad

La portabilidad se cuantifica como la capacidad de ejecutar el sistema en distintos entornos sin cambios de código. SmartComedor se distribuye con Docker, lo que produce un índice de portabilidad alto (Tabla 104).

Tabla 104

Métricas de portabilidad

Métrica	Valor
Adaptabilidad (entornos soportados)	Docker (dev/prod) y ejecución local
Facilidad de instalación	<code>docker compose -profile dev up</code> (un comando)
Independencia de plataforma	Node.js + contenedores (Linux/macOS/Windows)
Navegadores objetivo	Chrome, Firefox, Edge, Safari (SPA responsive)

Métricas de mantenibilidad

La mantenibilidad se estima con la modularidad, la analizabilidad (cobertura) y la capacidad de prueba. La Tabla 105 resume los indicadores.

Tabla 105

Métricas de mantenibilidad

Métrica	Valor
Modularidad	Separación en capas (rutas/controladores/servicios/modelos)
Densidad de comentarios (servidor)	$\approx 10,5\%$
Razón prueba/producción	$\approx 0,77$
Analizabilidad (cobertura global)	80,44 % sentencias
Capacidad de prueba	196 casos automatizados, deterministas

Métricas de rendimiento

El objetivo de rendimiento es que las operaciones críticas (autenticación y consultas) respondan en menos de 3 segundos (Apache Software Foundation, 2024). La Tabla 106 resume los indicadores de eficiencia de desempeño.

Tabla 106*Métricas de rendimiento*

Métrica	Objetivo / valor
Tiempo de respuesta (operaciones críticas)	< 3 s
Comportamiento temporal	Consultas indexadas por clave/UID
Utilización de recursos	Contenedores con <code>healthcheck</code>
Tiempo de ejecución de la suite	≈ 4,7 s (196 casos)

Métricas de fiabilidad

La fiabilidad se evidencia en la tasa de éxito de las pruebas y en los mecanismos de recuperación. La Tabla 107 resume los indicadores.

Tabla 107*Métricas de fiabilidad*

Métrica	Valor
Madurez (tasa de éxito de pruebas)	100 % (196/196)
Densidad de defectos abiertos	0 en la suite automatizada
Tolerancia a fallos	Manejo de errores con códigos HTTP controlados
Recuperabilidad	Respaldo diario con <code>pg_dump</code> (retención 30 días)

Métricas de compatibilidad

La compatibilidad mide la coexistencia e interoperabilidad. SmartComedor expone una API REST con JSON, consumible por el cliente web y por terceros autorizados (Tabla 108).

Tabla 108*Métricas de compatibilidad*

Métrica	Valor
Interoperabilidad	API REST/JSON con autenticación JWT
Coexistencia	Servicios desacoplados en contenedores
Formatos de exportación	Excel, CSV y PDF

Métricas de adecuación funcional

Mide en qué grado el sistema cubre las funciones requeridas. Con 45 requisitos funcionales y 56 endpoints implementados, la cobertura funcional es completa (Tabla 109).

Tabla 109*Métricas de adecuación funcional*

Métrica	Valor
Completitud funcional	45/45 requisitos funcionales implementados
Corrección funcional	Verificada por 196 casos de prueba
Pertinencia funcional	56 endpoints alineados a los casos de uso

Conclusiones de la medición

La medición confirma un producto de tamaño medio ($\approx 14,5$ KLOC de aplicación) con una fuerte inversión en pruebas (razón prueba/producción $\approx 0,77$ y cobertura global del 80,44%). Los atributos externos más críticos —seguridad y fiabilidad— presentan los mejores indicadores (cobertura del componente de acceso $\geq 97\%$ y 100% de éxito en pruebas). Las oportunidades de mejora se concentran en elevar la cobertura de `mealsController.ts` y `auditService.ts`, y en formalizar mediciones continuas de rendimiento en producción.

CUARTA PARTE. Estimación del Software

Introducción a la estimación

La estimación del software busca predecir el tamaño, el esfuerzo, el costo y la duración del proyecto a partir de sus características (Pressman & Maxim, 2014; Sommerville, 2011). En esta parte se estima SmartComedor mediante cuatro técnicas complementarias —puntos de función, puntos de caso de uso, puntos objeto y puntos de historia— y se contrastan con una estimación basada en líneas de código (COCOMO). Los conteos provienen del sistema real: 21 casos de uso, 11 entidades, 56 endpoints y 45 requisitos funcionales.

Estimación por puntos de función (PF)

Se aplica el método IFPUG, contando las funciones de datos y de transacción. La Tabla 110 muestra el conteo de puntos de función sin ajustar (PFSA).

Tabla 110

Conteo de puntos de función sin ajustar

Tipo de función	Cant.	Peso (medio)	Total
Entradas externas (EI)	22	4	88
Salidas externas (EO)	8	5	40
Consultas externas (EQ)	14	4	56
Archivos lógicos internos (ILF)	11	10	110
Archivos de interfaz externos (EIF)	2	7	14
Puntos de función sin ajustar (PFSA)			308

El factor de ajuste de valor (VAF) se obtiene de las 14 características generales del sistema (GSC). Con un grado total de influencia (TDI) estimado en 40 (promedio $\approx 2,9$ por característica, propio de un sistema web transaccional con seguridad y portabilidad relevantes):

$$\text{VAF} = 0,65 + 0,01 \cdot \text{TDI} = 0,65 + 0,01 \cdot 40 = 1,05$$

$$\text{PF ajustados} = \text{PFSA} \cdot \text{VAF} = 308 \cdot 1,05 = \mathbf{323,4}$$

Con una productividad de 8 horas por punto de función (proyecto TypeScript con equipo de experiencia media), el esfuerzo estimado es:

$$323,4 \times 8 \approx 2,587 \text{ horas} \approx 16,2 \text{ meses-persona.}$$

Estimación por puntos de caso de uso (PCU)

Se aplica el método de Karner. Primero se calcula el peso de los actores sin ajustar (UAW) y el de los casos de uso sin ajustar (UUCW).

Tabla 111

Peso de los actores sin ajustar (UAW)

Tipo de actor	Cant.	Peso	Total
Complejo (persona vía GUI: estudiante, supervisor, administrador, auditor)	4	3	12
Simple (otro sistema/API: cron, servicio de correo)	2	1	2
UAW			14

Tabla 112

Peso de los casos de uso sin ajustar (UUCW)

Complejidad (transacciones)	Cant.	Peso	Total
Simple (≤ 3)	0	5	0
Medio (4–7)	18	10	180
Complejo (> 7 : CU-01, CU-07, CU-14)	3	15	45
UUCW			225

$$\text{UUCP} = \text{UAW} + \text{UUCW} = 14 + 225 = 239$$

El factor de complejidad técnica (TCF) y el factor ambiental (ECF) ajustan el valor. Con

un total de factores técnicos $TF = 51,5$ y ambientales $EF = 20,5$:

$$TCF = 0,6 + 0,01 \cdot 51,5 = 1,115 \quad ECF = 1,4 - 0,03 \cdot 20,5 = 0,785$$

$$UCP = UUCP \cdot TCF \cdot ECF = 239 \cdot 1,115 \cdot 0,785 \approx \mathbf{209,2}$$

Con un factor de productividad de 20 horas-persona por UCP, el esfuerzo estimado es:

$$209,2 \times 20 \approx 4,184 \text{ horas} \approx 26,2 \text{ meses-persona.}$$

Estimación por puntos objeto

Los puntos objeto cuentan pantallas, informes y módulos 3GL ponderados por complejidad. La Tabla 113 resume el conteo.

Tabla 113

Conteo de puntos objeto

Objeto	Cant.	Peso (medio)	Total
Pantallas (vistas por rol)	20	2	40
Informes (reportes/exportaciones)	6	5	30
Módulos 3GL (servicios/integraciones)	5	10	50
Puntos objeto (bruto)			120

Aplicando un porcentaje de reúso estimado del 25 %, los nuevos puntos objeto (NOP) son $120 \cdot (1 - 0,25) = 90$. Con una productividad media ($PROD = 13 \text{ NOP/mes}$ para un equipo de experiencia y madurez *nominal*):

$$\text{Esfuerzo} = \frac{\text{NOP}}{\text{PROD}} = \frac{90}{13} \approx 6,9 \text{ meses-persona.}$$

Estimación por puntos de historia

Bajo el enfoque ágil (Scrum), el trabajo se estima en puntos de historia con la sucesión de Fibonacci. La Tabla 114 agrupa las épicas del producto.

Tabla 114*Estimación por puntos de historia (épicas)*

Épica	Historias	Puntos
Autenticación y cuentas (login, 2FA, recuperación)	6	21
Gestión de estudiantes (CRUD, importación, SISBÉN)	7	34
Gestión de supervisores e invitaciones	5	21
Pagos y comprobantes	4	13
Registro y control de almuerzos	5	21
Ciclos y automatización	3	13
Retroalimentación (calificaciones y quejas)	4	8
Noticias	2	5
Auditoría y analítica	3	13
Total	39	149

Con una velocidad de equipo estimada en 20 puntos por *sprint* de dos semanas, el proyecto requeriría $\lceil 149/20 \rceil = 8$ *sprints* (≈ 4 meses de calendario con el equipo completo).

Estimación con una herramienta (COCOMO básico)

Como contraste, se aplica COCOMO básico (modo orgánico) sobre el tamaño real medido: $KLOC \approx 11,29$ (código de aplicación de servidor y cliente, sin pruebas).

$$E = 2,4 \cdot (11,29)^{1,05} \approx 30,6 \text{ meses-persona}$$

$$T_{dev} = 2,5 \cdot (30,6)^{0,38} \approx 9,2 \text{ meses} \quad N = \frac{E}{T_{dev}} \approx 3,3 \text{ personas}$$

Análisis de los resultados

La Tabla 115 compara las técnicas. Los métodos basados en funcionalidad (PF y PCU) y el modelo COCOMO convergen en un esfuerzo del orden de **16 a 31 meses-persona**, mientras que los métodos de productividad agregada (puntos objeto y puntos de historia) proyectan duraciones de calendario de **4 a 7 meses** para un equipo pequeño.

Tabla 115*Comparación de técnicas de estimación*

Técnica	Resultado	Interpretación
Puntos de función	323,4 PF	$\approx 2,587$ h ($\approx 16,2$ meses-persona)
Puntos de caso de uso	209,2 UCP	$\approx 4,184$ h ($\approx 26,2$ meses-persona)
Puntos objeto	90 NOP	$\approx 6,9$ meses-persona
Puntos de historia	149 pts	8 <i>sprints</i> (≈ 4 meses)
COCOMO (11,29 KLOC)	30,6 MP	9,2 meses con $\approx 3,3$ personas

La diferencia entre PF y PCU se explica por los factores de ajuste: el método de Karner penaliza con un factor ambiental bajo (equipo en formación), elevando el esfuerzo. La coincidencia entre COCOMO (30,6 MP) y PCU (26,2 MP) refuerza la validez del rango superior, en tanto que PF y puntos objeto marcan el límite inferior. Para la planeación se recomienda adoptar el valor medio del rango y gestionar el riesgo con entregas incrementales (Scrum), monitoreando la velocidad real frente a la estimada.

Conclusiones de la estimación

Las cinco técnicas aplicadas sobre datos reales del sistema ofrecen un intervalo de esfuerzo coherente y permiten triangular la planeación. El sistema SmartComedor se clasifica como un proyecto de tamaño medio ($\approx 11,3$ KLOC de aplicación, 323 PF, 209 UCP), abordable por un equipo de 3 a 4 personas en un horizonte de 4 a 9 meses según el alcance comprometido por *sprint*.

QUINTA PARTE. Conclusiones Generales

Conclusiones generales

El presente documento integró, en cuatro grandes ejes, la ingeniería del sistema **SmartComedor**: la descripción y el diseño, la verificación y validación, la medición y la estimación. Las principales conclusiones son:

1. **Solución completa y trazable.** SmartComedor digitaliza la operación del comedor universitario (registro y validación de elegibilidad por SISBÉN, control de acceso por roles, gestión de pagos, registro de almuerzos, retroalimentación y auditoría) sobre una arquitectura web de tres capas desplegable con Docker, cubriendo los 45 requisitos funcionales y los 21 casos de uso especificados.
2. **Calidad verificada empíricamente.** La suite de 196 pruebas automatizadas se ejecuta con 100 % de éxito y una cobertura global del 80,44 % de sentencias, con los componentes críticos de seguridad y elegibilidad por encima del 97 %.
3. **Producto medido objetivamente.** La medición confirma un sistema de tamaño medio ($\approx 11,3$ KLOC de aplicación) con una fuerte inversión en pruebas (razón prueba/producción $\approx 0,77$) y buenos indicadores en seguridad, fiabilidad y adecuación funcional según ISO/IEC 25010.
4. **Esfuerzo estimado con múltiples técnicas.** Puntos de función (323,4 PF), puntos de caso de uso (209,2 UCP), puntos objeto, puntos de historia (149 pts) y COCOMO convergen en un proyecto abordable por un equipo de 3 a 4 personas en un horizonte de 4 a 9 meses.
5. **Mejora continua.** Las oportunidades identificadas —elevar la cobertura de `mealsController` y `auditService`, añadir pruebas de componentes del frontend y formalizar mediciones de rendimiento en producción— constituyen la hoja de ruta de evolución del producto.

En conjunto, el trabajo evidencia la aplicación rigurosa de las prácticas de la ingeniería de software —análisis, diseño, pruebas, medición y estimación— sobre un sistema real, alineado con los estándares y técnicas reconocidos de la disciplina (IEEE, 2008; International Organization for Standardization, 2011; Pressman & Maxim, 2014; Sommerville, 2011).

Referencias

- Apache Software Foundation. (2024). *Apache JMeter*. Consultado el 16 de junio de 2026, desde <https://jmeter.apache.org/>
- Beizer, B. (1990). *Software Testing Techniques* (2.^a ed.). Van Nostrand Reinhold.
- Google. (2024). *Material Design 3*. Consultado el 16 de junio de 2026, desde <https://m3.material.io/>
- IEEE. (2008). *IEEE Standard for Software and System Test Documentation (IEEE Std 829-2008)* (inf. téc.). Institute of Electrical y Electronics Engineers. New York, NY.
<https://doi.org/10.1109/IEEESTD.2008.4578383>
- International Organization for Standardization. (2011). *ISO/IEC 25010:2011 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models* (inf. téc.). ISO. Geneva, Switzerland.
- McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308-320. <https://doi.org/10.1109/TSE.1976.233837>
- Meta Open Source. (2024). *Jest: Delightful JavaScript Testing Framework*. Consultado el 16 de junio de 2026, desde <https://jestjs.io/>
- Microsoft. (2024). *Playwright: Fast and Reliable End-to-End Testing for Modern Web Apps*. Consultado el 16 de junio de 2026, desde <https://playwright.dev/>
- Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3.^a ed.). John Wiley & Sons.
- Nielsen, J. (1994). Heuristic Evaluation. En J. Nielsen & R. L. Mack (Eds.), *Usability Inspection Methods* (pp. 25-62). John Wiley & Sons.
- OWASP Foundation. (2021). *OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks*. Consultado el 16 de junio de 2026, desde <https://owasp.org/Top10/>
- Pressman, R. S., & Maxim, B. R. (2014). *Software Engineering: A Practitioner's Approach* (8.^a ed.). McGraw-Hill Education.
- Sommerville, I. (2011). *Software Engineering* (9.^a ed.). Addison-Wesley.